

Organisatorisches

Programmieren und Software-Engineering Theorie: Algorithmen, Formale Sprachen und Homomorphismen

26. Februar 2026

POS (Theorie)

Organisatorisches

1 / 12

Beurteilung

- 65% SLÜs (Theorie)
- 35% Programmieraufgabe
- **Für eine positive Note müssen sowohl die Theorie als auch die Programmieraufgabe positiv sein.**
- Die SLÜs müssen gemeinsam eine positive Note ergeben, es muss jedoch nicht unbedingt jede einzelne SLÜ positiv sein.
- Bei nachweislicher Verhinderung bei der SLÜ kann auf Wunsch in der darauffolgenden Stunde eine Ersatzprüfung erfolgen.

POS (Theorie)

Organisatorisches

3 / 12

Organisatorisches

Die Gesamtnote POS ergibt sich durch

- POS Programmieren (nicht dieser Kurs)
- POS Theorie: (dieser Kurs!)
 - ① Wintersemester: "Graphentheorie"
 - ② Sommersemester:
 - Algorithmen
 - Formale Sprachen, Syntaxanalyse
 - Homomorphismen
- Alle Teile müssen positiv bestanden werden um positive Gesamtnote zu erhalten.

POS (Theorie)

Organisatorisches

2 / 12

Notenschlüssel

Prozent	Note
[0, 50]	Nicht Genügend
(50, 62.5]	Genügend
(62.5, 75]	Befriedigend
(75, 87.5]	Gut
(87.5, 100]	Sehr Gut

POS (Theorie)

Organisatorisches

4 / 12

Programmieraufgabe

- Erstellen Sie ein Programm in Java, C#, Python, Javascript (andere Programmiersprachen nach Rücksprache möglich)
- Das Programm soll die Adjazenzmatrix eines Graphen aus einer Datei (csv) einlesen
- Weiters soll das Programm folgende Berechnungen ohne die Verwendung von Bibliotheken durchführen (Minimalanforderungen):
 - Bestimmung der Distanzen und Exzentrizitäten aller Knoten
 - Radius, Durchmesser, Zentrum
 - Komponenten, Artikulationen, Brücken
- Implementieren Sie auf jeden Fall die matrizenbasierten Algorithmen aus dem Unterricht (Wintersemester Foliensatz 7). Als Zusatzaufgabe können weitere Algorithmen umgesetzt werden.
- Wählen Sie eine dieser (oder andere) Erweiterungen für ein "Sehr Gut":
 - Grafische Benutzeroberfläche
 - Eulersche Linien/Zyklen
 - Spannbäume/Gerüste
 - Starke Zusammenhangskomponente
 - Blöcke (schwierig!)
 - Test auf Isomorphie

POS (Theorie)

Organisatorisches

5 / 12

Abgabegespräch

- Individuelle Terminvereinbarung
 - Für Sommersemester: bis 1. Juni
 - Für Kolloquium: bis 10. Jänner
- Bereiten Sie mindestens einen Testfall mit mindestens 10 Knoten und mindestens zwei Brücken vor. Eine Zeichnung des/der Graphen soll ebenso präsentiert werden.
- Nur sinnvoll viele Kommentare im Code (keine vollständigen Erklärungen zu jeder Code-Zeile)!
- Führen Sie zunächst das Programm aus: Demonstration der Funktionalität und Korrektheit.
- Geben Sie einen kurzen Überblick über den Aufbau des Programmes.
- Fragen zu ausgewählten Algorithmen und Codestücken

POS (Theorie)

Organisatorisches

7 / 12

Programmabgabe

- Fertigstellung des Programmes bis 1. Juni für Wertung im laufenden Semester
- Erstellen Sie ein Bildschirmvideo, indem sie Ihr Programm präsentieren.
- Für eine gute Benotung des Programmes ist ein Abgabegespräch notwendig.
- Ohne Abgabegespräch kann das Programm maximal mit "Befriedigend" bewertet werden.
- Quellcode als ZIP-Datei (keine Binärdateien), Video
- Namenskonvention (Beispiel): 2021-06-21_3BAIF_Gruber.zip

POS (Theorie)

Organisatorisches

6 / 12

Kolloquium, spätere Programmabgabe

- Für reine Kolloquien (Theorie) nutzen Sie bitte einen der Sammeltermine (Raum B4.17/B4.18a, oder B3.08-10)
 - letzter Freitag in den Sommerferien: 12:35
 - Montag nach den Herbstferien: 15:15
 - Montag vor den Semesterferien: 15:15
 - Montag nach den Semesterferien: 15:15
 - Montag nach den Osterferien: 15:15
 - Montag vor den Sommerferien: 15:15
- Terminvereinbarung per Mail (Betreff: Kolloquium 4XYIF POS/Theorie)
- Bitte Klasse und Schuljahr im Mail anführen!
- Für Abgabegespräche oder individuelle Termine: bitte zwei bis drei Termine (kompatibel mit unseren Stundenplänen) vorschlagen

POS (Theorie)

Organisatorisches

8 / 12

Stoffübersicht

- Algorithmen
 - Laufzeitanalyse
 - Rekursion
 - Sortierverfahren
 - Datenstrukturen
- Syntaxanalyse
 - Grammatiken
 - Backus-Naur Form
 - Syntaxdiagramme, Syntaxanalyse
 - ...
- Formale Sprachen
 - Chomsky Hierarchie
 - Reguläre Sprachen, endliche Automaten
 - Kontextfreie Sprachen
 - Komplexität und Berechenbarkeit
- Homomorphismen

POS (Theorie)

Organisatorisches

9 / 12

Weiterführende Inhalte (*)

- Manche Inhalte sind speziell für besonders Interessierte gedacht
- Diese Inhalte zählen nicht zum Kernstoffgebiet, und müssen somit nicht gelernt/gekonnt/verstanden werden
- Die Kennzeichnung dieser Inhalte erfolgt durch die blaue Titel- und Fußzeile in den Folien!

Unterlagen

- Die Vortragsfolien sowie ein zur Verfügung gestelltes Programm decken die Inhalte ab!
- Eigene Mitschrift zu Beispielen, Erklärungen
- Einige Übungsbeispiele werden nur im Unterricht an der Tafel präsentiert.

Achtung!

Die Präsentationsfolien enthalten Animationen die zu gewissen Beispielen/Algorithmen eine Schritt-für-Schritt Erklärung enthalten. Diese Animationen sind am Besten im Vollbildmodus anzusehen! Im gedruckten Handout finden sich lediglich verkürzte Darstellungen mit weniger Zwischenschritten.

POS (Theorie)

Organisatorisches

10 / 12

Literaturübersicht I

- [1] **Berger, Krieger, Mahr: "Grundlagen der elektronischen Datenverarbeitung", Skriptum**
- [2] Dirk W. Hoffmann: "Theoretische Informatik", Hanser, 3. Auflage
- [3] Gernot Salzer: "Einführung in die Theorie der Informatik", Skriptum, TU Wien, 2001
- [4] Wikipedia (Englisch): <https://en.wikipedia.org/>
- [5] Wikipedia (Deutsch): <https://de.wikipedia.org/>
- [6] HappyCoders (Deutsch): <https://www.happycoders.eu/de/algorithmen/>

Breiten- und Tiefensuche

Programmieren und Software-Engineering Theorie

26. Februar 2026

POS (Theorie)

Theorie

1 / 19

Traversierung von Bäumen

Zur Beschreibung der Suchverfahren werden folgende Begriffe benötigt:

- Ein Knoten wird **entdeckt**, wenn er das erste Mal besucht wird.
- Ein Knoten wird **fertiggestellt/abgeschlossen**, wenn er das letzte Mal verlassen wird.

Für manche Anwendungen ist es wichtig festzuhalten, wann ein Knoten *entdeckt*, bzw. *abgeschlossen* wurde. Dazu führen wir einen Zähler τ mit, der in jedem Schritt um 1 erhöht wird.

- Wird ein Knoten v das erste mal besucht ("entdeckt"), so setzen wir $\tau_d(v) = \tau++$
- Wird ein Knoten v das letzte mal verlassen ("abgeschlossen"), so setzen wir $\tau_f(v) = \tau++$

POS (Theorie)

Theorie

3 / 19

Traversierung von Bäumen

Wir betrachten im Folgenden Algorithmen zur Traversierung von Bäumen. Diese können jedoch auch zur Traversierung von Graphen im Allgemeinen verwendet werden.

Ziel: Wir wollen die Knoten eines Baumes systematisch durchlaufen, mit dem Ziel einen bestimmten Knoten zu finden.

- **Breitensuche (Breadth-First-Search (BFS)):** In jedem Schritt werden zunächst alle Nachbarknoten eines Knoten besucht, bevor von dort aus weitere Pfade gebildet werden.
- **Tiefensuche (Depth-First-Search (DFS)):** Ein Pfad wird vollständig in die Tiefe durchlaufen, bevor etwaige Abzweigungen verwendet werden.

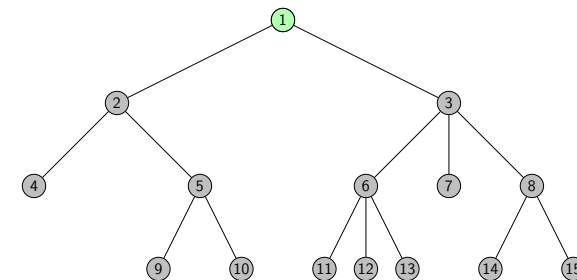
POS (Theorie)

Theorie

2 / 19

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.



$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

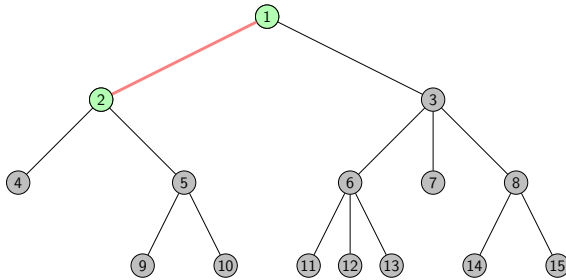
POS (Theorie)

Theorie

4 / 19

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

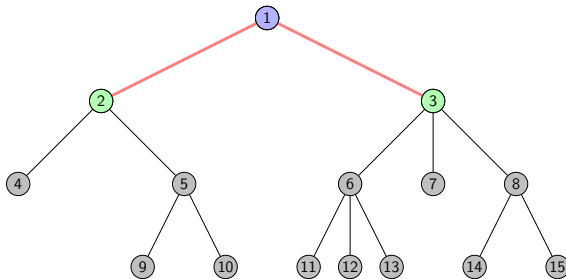


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

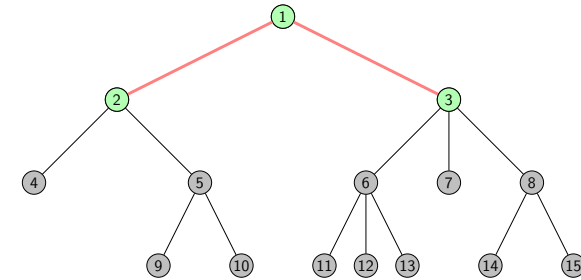


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

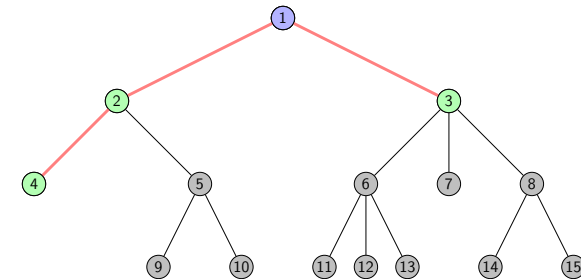


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

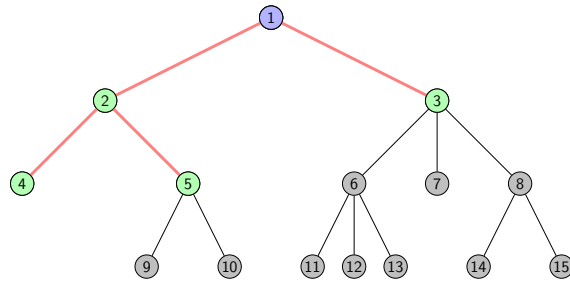


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

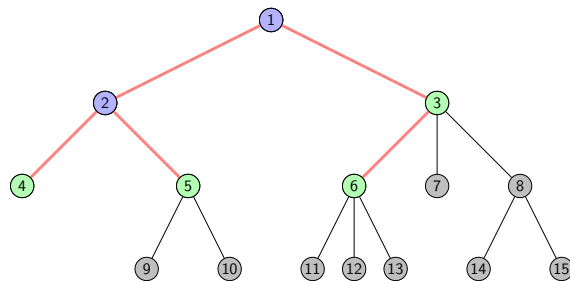


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

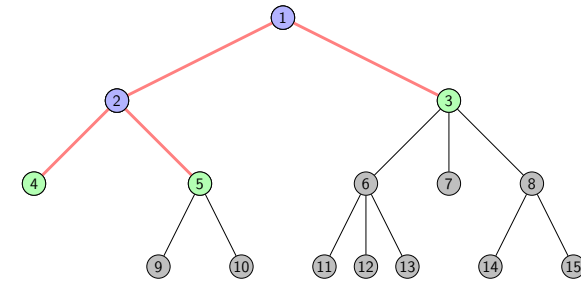


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

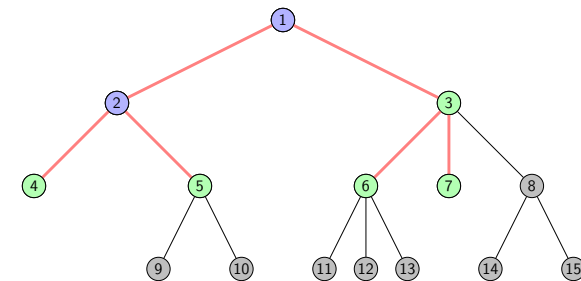


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

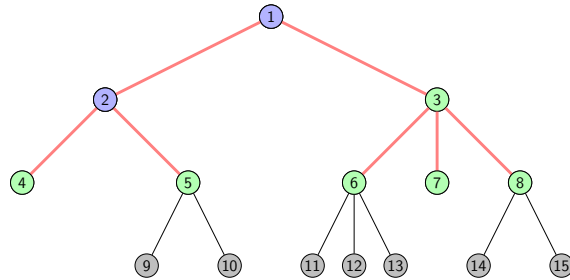


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

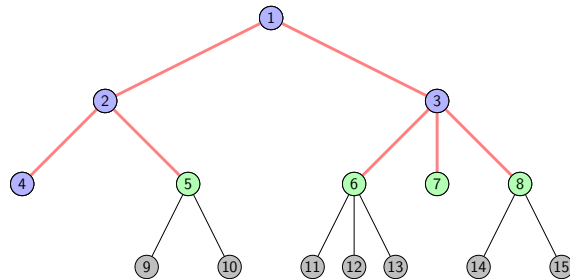


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

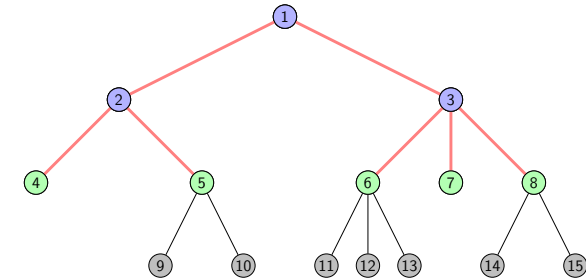


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.

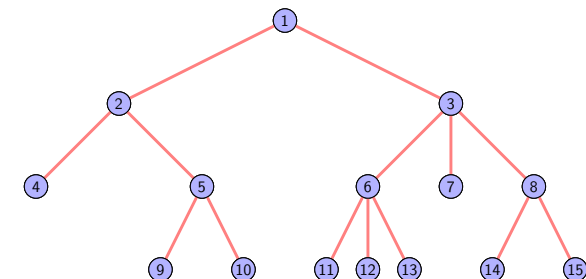


$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

Breitensuche

Beispiel: Beispiel einer Breitensuche. Die Suche kann abgebrochen werden, sobald der gesuchte Knoten gefunden ("entdeckt") wurde.



Achtung: einige Zwischenschritte sind in diesem Handout übersprungen! Alle Schritte sind im einzelnen Foliensatz als Animation verfügbar!

$\tau_d(1) = 1, \tau_d(2) = 2, \tau_d(3) = 3, \tau_d(4) = 5, \tau_d(5) = 6, \tau_d(6) = 8, \tau_d(7) = 9, \tau_d(8) = 10, \tau_d(9) = 13, \tau_d(10) = 14, \dots$

$\tau_f(1) = 4, \tau_f(2) = 7, \tau_f(3) = 11, \tau_f(4) = 12, \tau_f(5) = 15, \dots$

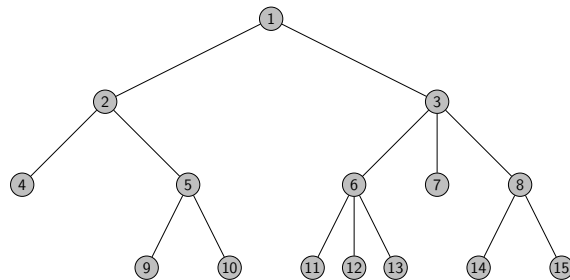
Breitensuche: Datenstruktur

In nahezu allen Programmiersprachen existiert eine Datenstruktur namens **Queue** (Warteschlange). Elemente können hinzugefügt werden ("hinten anstellen"), und werden geordnet abgespeichert. Das Element das als erstes hinzugefügt wurde, kann entnommen werden ("Nächster!")



Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



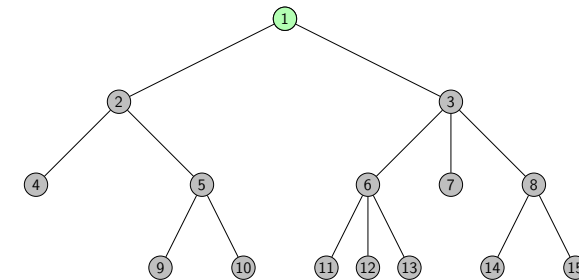
Breitensuche: Algorithmus

- 1 Füge Startknoten (Wurzel) in Queue (Warteschlange) ein und markiere ihn als entdeckt
- 2 Entnimm Knoten am Beginn der Queue
 - Markiere Knoten als entdeckt
 - Wenn dies der gesuchte Knoten ist $\Rightarrow$ Fertig!
 - Sonst: füge alle unbesuchten¹ Nachbarn dieses Knotens in die Queue ein
- 3 Wenn die Warteschlange leer ist wurden alle Knoten bereits besucht $\Rightarrow$ Fertig
- 4 Gehe zu Schritt (2)

¹nicht entdeckt, nicht abgeschlossen

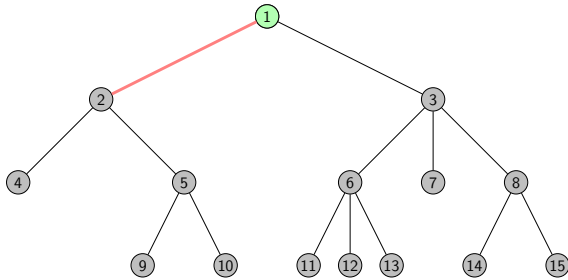
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



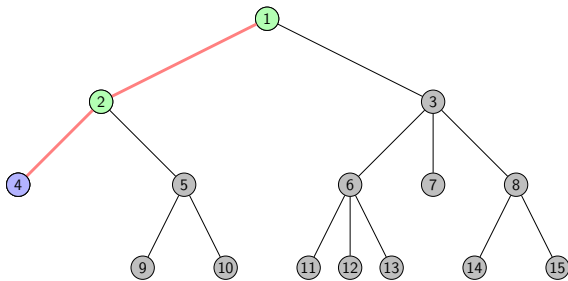
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



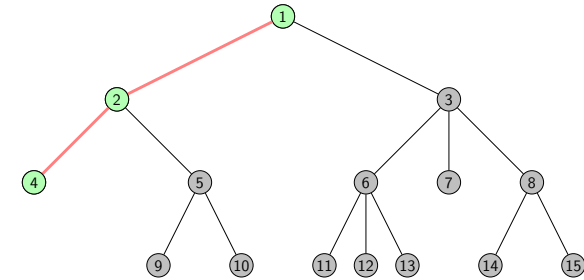
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



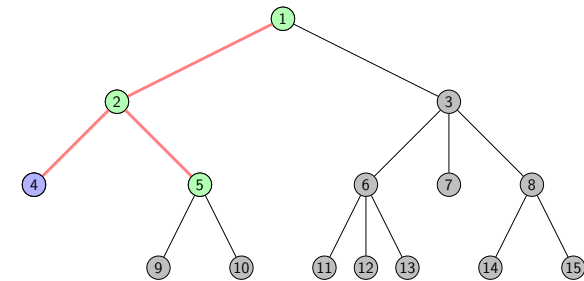
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



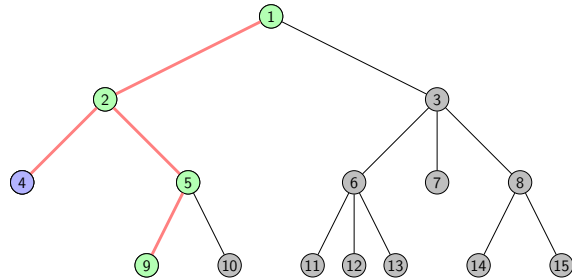
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



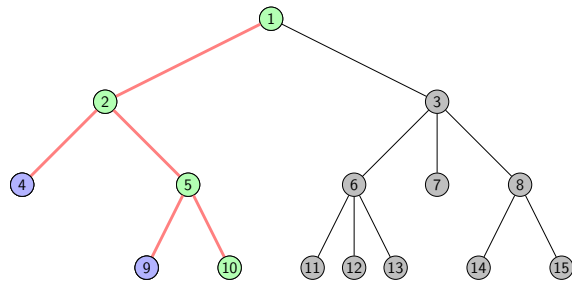
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



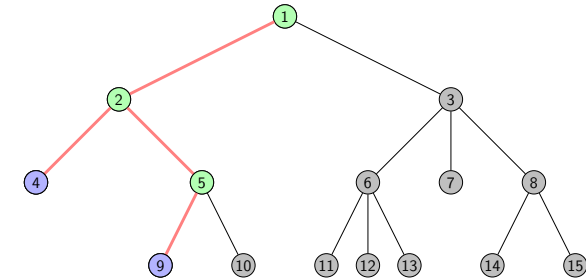
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



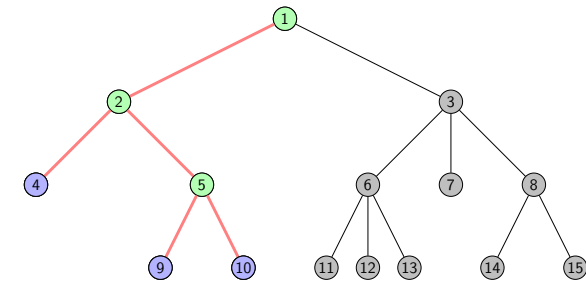
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



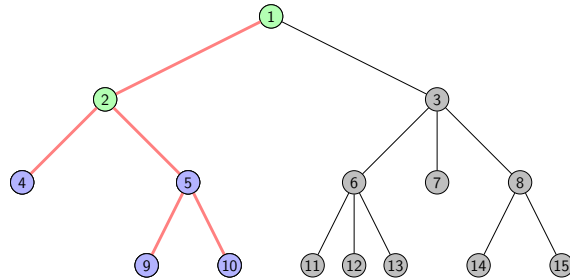
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



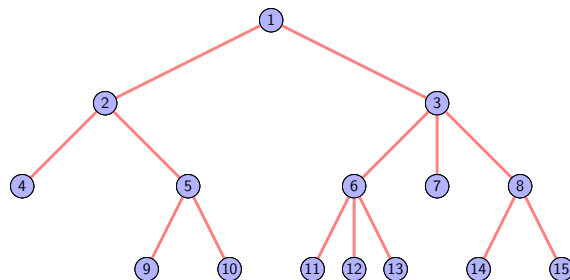
Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



Tiefensuche

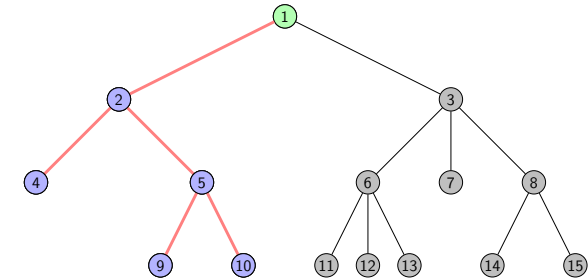
Beispiel: Beispiel einer Tiefensuche:



Achtung: einige Zwischenschritte sind in diesem Handout übersprungen! Alle Schritte sind im einzelnen Foliensatz als Animation verfügbar!

Tiefensuche

Beispiel: Beispiel einer Tiefensuche:



Tiefensuche: Datenstruktur

In nahezu allen Programmiersprachen existiert eine Datenstruktur namens **Stack** (Stapel).

Elemente können hinzugefügt werden ("oben drauflegen"), und werden geordnet abgespeichert. Das Element das als *letztes* hinzugefügt wurde, kann entnommen werden (oberstes Element vom Stapel nehmen).



Tiefensuche: Algorithmus (*)

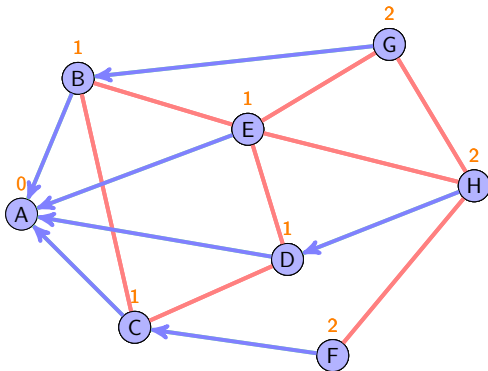
Algorithmus 1: Tiefensuche

```

1 Function DFS( $G = (V, E)$ , Startknoten  $v$ , Gesuchter Knoten  $s$ )
   Result: vertex  $s \in V$ , if exists
2   Stack  $S$ ;
3   markiere  $v$  als entdeckt;
4    $S.push(v)$ ; // Lege  $v$  auf Stapel
5   while  $S$  not empty do
6      $v = S.pop()$ ; // nimm obersten Knoten vom Stapel
7     if  $v$  gesuchter Knoten  $s$  then
8       return  $v$ ;
9     markiere  $v$  als abgeschlossen;
10    for all  $[v, u] \in E(G)$  do
11      if  $u$  noch nicht besucht then
12        markiere  $u$  als entdeckt;
13         $S.push(u)$ ;

```

Breitensuche auf allgemeinen Graphen



Start mit Knoten A B, C, D, E werden entdeckt (Distanz jeweils 1), A fertiggestellt

Fortsetzung bei B: [B, E] wird nicht betrachtet, da E schon besucht; G wird entdeckt (Distanz 2) B wird fertiggestellt, und Verweis auf Vorgänger A gespeichert.

Fortsetzung bei C: [C, D] wird nicht betrachtet, da D schon besucht. F wird

Anmerkungen

- Die Algorithmen können auch für allgemeine Graphen (und nicht nur Bäume) verwendet werden.
- Dabei werden schon besuchte Knoten *nicht* erneut besucht!
- Anwendungen BFS:
 - 2-färbbarkeit
 - Kürzester Pfad zwischen zwei Knoten
 - Kürzeste-Kreise-Problem
- Anwendungen DFS:
 - Test auf Kreisfreiheit
 - Topologische Sortierung
 - Starke Zusammenhangskomponente

Breitensuche auf allgemeinen Graphen

- Es entsteht ein (Baum) mit Startknoten als Wurzel
- Bei nicht erreichbaren Knoten im ersten Aufruf wird die Breitensuche erneut aufgerufen
- Dadurch entsteht ein Wald (=mehrere Bäume)
- Distanzberechnung durch Breitensuche möglich, da jeder Knoten um 1 höhere Distanz als sein Vorgänger hat.

Minimale Spannbäume

Definition (Spannbaum)

Ein *Spannbaum* T zu einem Graphen $G = (V, E)$ ist ein (auf-)spannender Teilgraph von G der ein Baum ist.

Wir betrachten nun ungerichtete, schlichte und zusammenhängende Graphen $G = (V, E)$ mit einer Gewichtsfunktion $w : E \rightarrow \mathbb{R}^+$ auf den Kanten $e \in E(G)$:

Definition (Minimaler Spannbaum)

Ein *Minimaler Spannbaum* T von $G = (V, E)$ mit $w : E \rightarrow \mathbb{R}^+$ für alle $e \in E(G)$ ist ein zusammenhängender Teilgraph mit $|E(T)| = |V(G)| - 1$ der alle Knoten enthält, und für den gilt:

$$\sum_{e \in E(T)} w(e) = \min$$

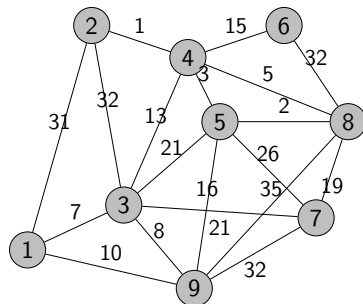
Ein Minimaler Spannbaum (Minimum Spanning Tree (MST)) ist also ein Baum mit minimalen Kantengewichten (=Kantenkosten).

POS (Theorie)

Theorie

13 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal

Algorithmus 2: Algorithmus von Kruskal

```

1 Function Kruskal(Graph  $G = (V, E)$ )
   Result: Minimum Spanning Tree  $T$ 
2   Ordne alle Kanten  $e \in E(G)$  aufsteigend nach  $w(e)$ ;
3    $\Rightarrow L = (e_1, e_2, \dots, e_n)$  mit  $w(e_1) \leq w(e_2) \leq \dots \leq w(e_n)$ 
4    $V(T) = V(G)$ ;
5    $E(T) = \emptyset$ ;
6   for  $e \in L$  do
7     if  $T = (V, E(T) \cup \{e\})$  ist kreisfrei then
8        $E(T) = E(T) \cup \{e\}$ ;
```

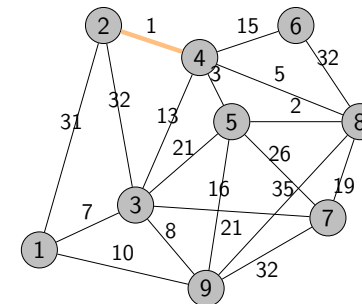
- Die Kreisfreiheit kann mit DFS berechnet werden.
- Nach Hinzufügen von $n - 1$ Kanten kann der Algorithmus beendet werden.

POS (Theorie)

Theorie

14 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

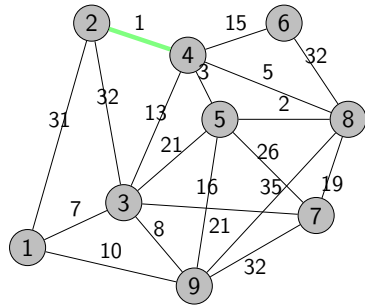
$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

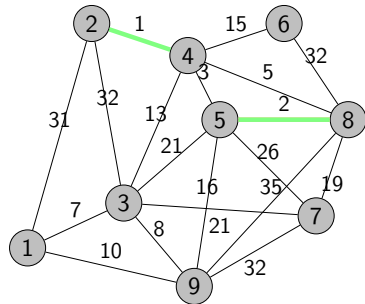
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

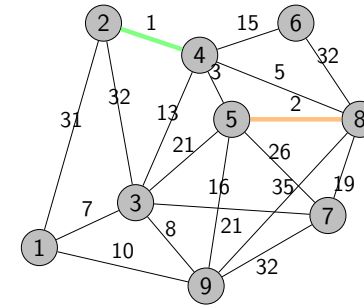
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

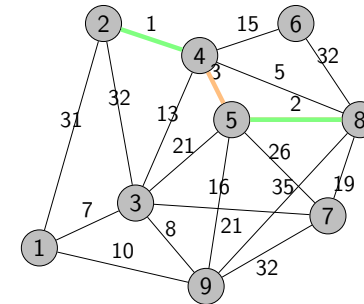
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

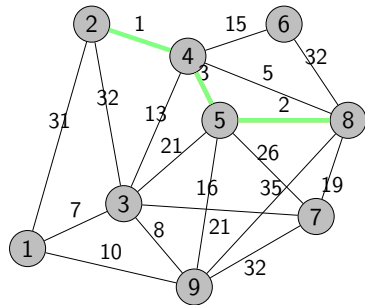
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

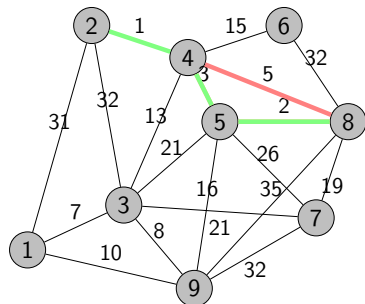
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

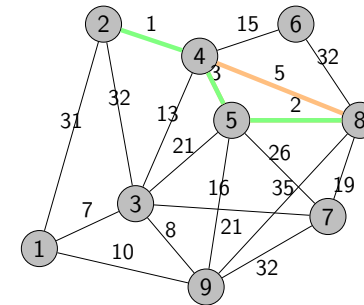
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

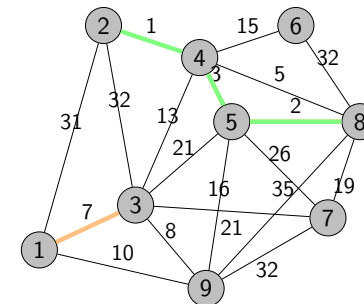
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

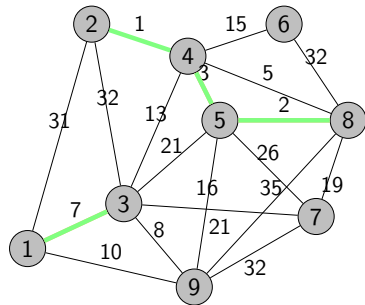
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

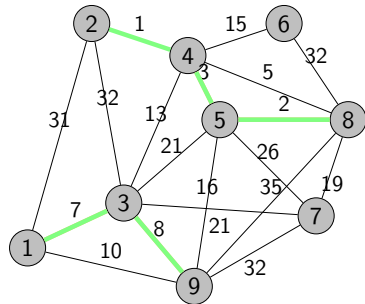
$w([2, 4]) = 1$, $w([5, 8]) = 2$, $w([4, 5]) = 3$, $w([4, 8]) = 5$
 $w([1, 3]) = 7$, $w([3, 9]) = 8$, $w([1, 9]) = 10$, $w([3, 4]) = 13$
 $w([4, 6]) = 15$, $w([5, 9]) = 16$, $w([7, 8]) = 19$, $w([3, 5]) = 21$
 $w([3, 7]) = 21$, $w([5, 7]) = 26$, ...

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

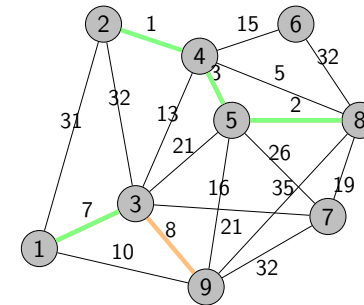
$w([2, 4]) = 1$, $w([5, 8]) = 2$, $w([4, 5]) = 3$, $w([4, 8]) = 5$
 $w([1, 3]) = 7$, $w([3, 9]) = 8$, $w([1, 9]) = 10$, $w([3, 4]) = 13$
 $w([4, 6]) = 15$, $w([5, 9]) = 16$, $w([7, 8]) = 19$, $w([3, 5]) = 21$
 $w([3, 7]) = 21$, $w([5, 7]) = 26$, ...

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

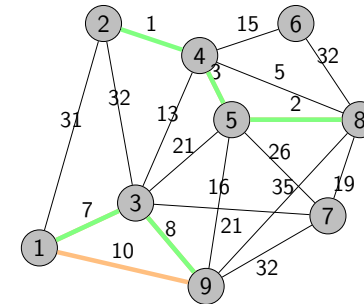
$w([2, 4]) = 1$, $w([5, 8]) = 2$, $w([4, 5]) = 3$, $w([4, 8]) = 5$
 $w([1, 3]) = 7$, $w([3, 9]) = 8$, $w([1, 9]) = 10$, $w([3, 4]) = 13$
 $w([4, 6]) = 15$, $w([5, 9]) = 16$, $w([7, 8]) = 19$, $w([3, 5]) = 21$
 $w([3, 7]) = 21$, $w([5, 7]) = 26$, ...

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

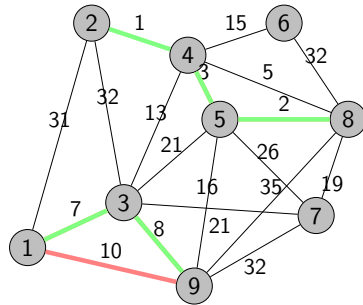
$w([2, 4]) = 1$, $w([5, 8]) = 2$, $w([4, 5]) = 3$, $w([4, 8]) = 5$
 $w([1, 3]) = 7$, $w([3, 9]) = 8$, $w([1, 9]) = 10$, $w([3, 4]) = 13$
 $w([4, 6]) = 15$, $w([5, 9]) = 16$, $w([7, 8]) = 19$, $w([3, 5]) = 21$
 $w([3, 7]) = 21$, $w([5, 7]) = 26$, ...

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

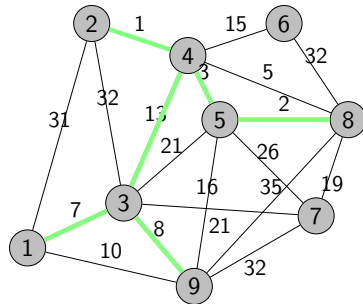
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

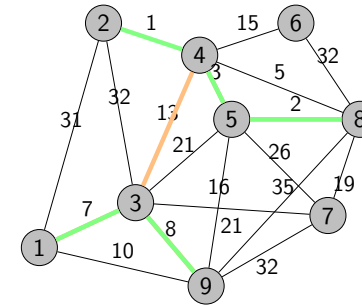
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

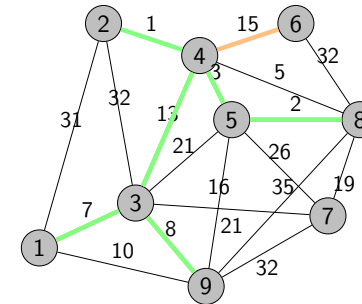
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

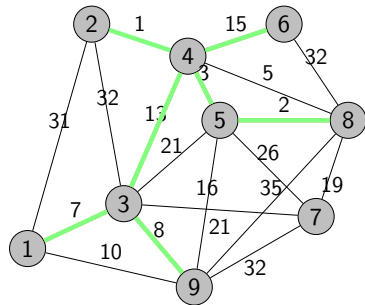
$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

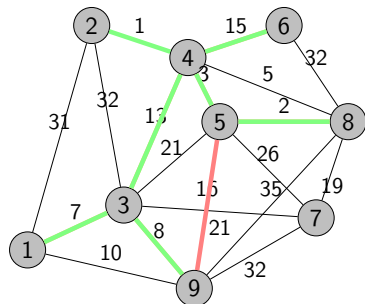
$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

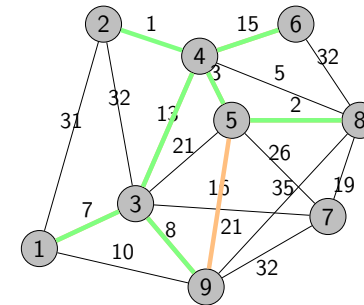
$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

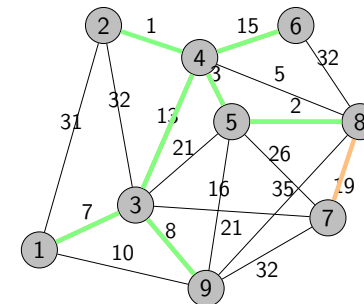
$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

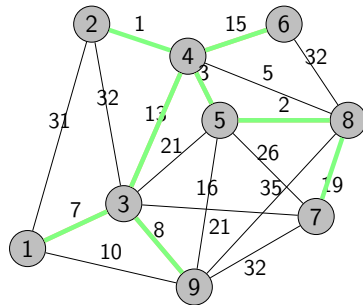
$w([2, 4]) = 1, w([5, 8]) = 2, w([4, 5]) = 3, w([4, 8]) = 5$
 $w([1, 3]) = 7, w([3, 9]) = 8, w([1, 9]) = 10, w([3, 4]) = 13$
 $w([4, 6]) = 15, w([5, 9]) = 16, w([7, 8]) = 19, w([3, 5]) = 21$
 $w([3, 7]) = 21, w([5, 7]) = 26, \dots$

POS (Theorie)

Theorie

15 / 19

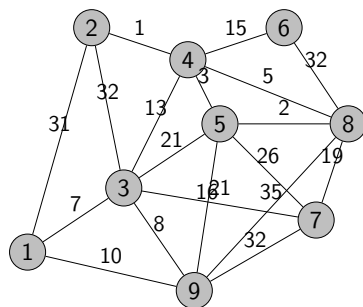
Algorithmus von Kruskal



Aufsteigend sortierte Kanten:

$w([2,4]) = 1, w([5,8]) = 2, w([4,5]) = 3, w([4,8]) = 5$
 $w([1,3]) = 7, w([3,9]) = 8, w([1,9]) = 10, w([3,4]) = 13$
 $w([4,6]) = 15, w([5,9]) = 16, w([7,8]) = 19, w([3,5]) = 21$
 $w([3,7]) = 21, w([5,7]) = 26, \dots$

Algorithmus von Prim



Algorithmus von Prim

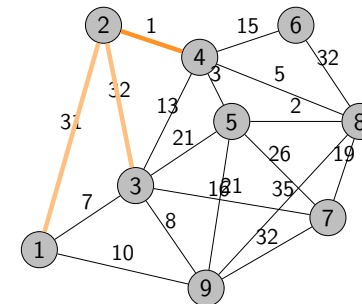
Der Algorithmus von Prim konstruiert ebenfalls einen MST.

Algorithmus 3: Algorithmus von Prim

```

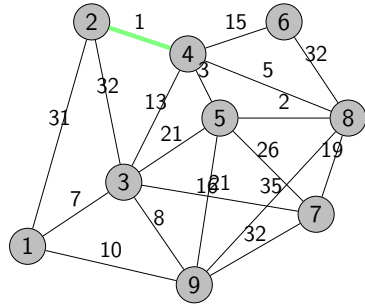
1 Function Prim(Graph G)
  Result: Minimum Spanning Tree T
2   $E(T) = \emptyset;$ 
3   $V(T) =$  ein zufaellig gewaehlter Startknoten;
4  while  $V(T) \subset V(G)$  do
5    Sei  $E'$  die Menge aller Kanten zwischen Knoten
6    aus  $V(T)$  und  $V(G) \setminus V(T)$ 
7     $e' = \operatorname{argmin}_{e \in E'} w(e);$  // Kante mit kleinstem Gewicht
8     $E(T) = E(T) \cup e';$ 
9    Füge neuen Knoten von  $e'$  zu  $V(T)$  hinzu;
```

Algorithmus von Prim



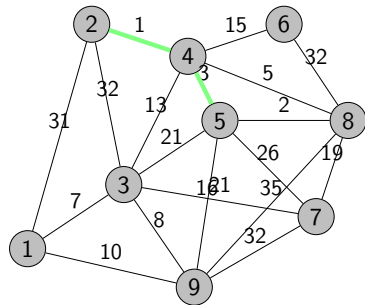
Startknoten: 2

Algorithmus von Prim



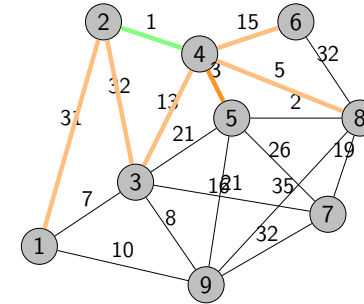
Startknoten: 2

Algorithmus von Prim



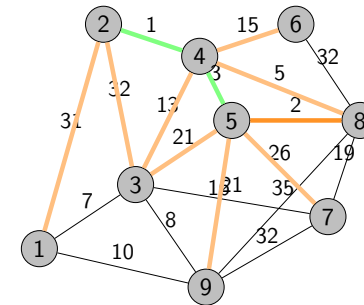
Startknoten: 2

Algorithmus von Prim



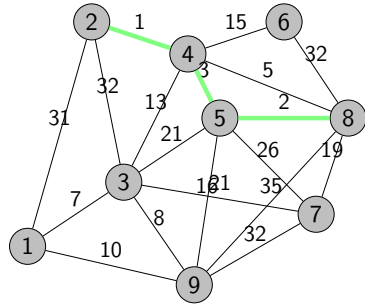
Startknoten: 2

Algorithmus von Prim



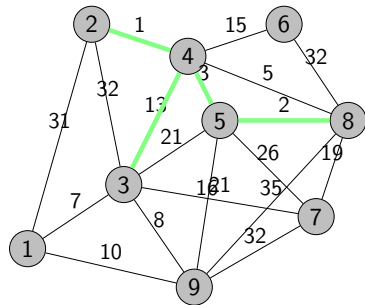
Startknoten: 2

Algorithmus von Prim



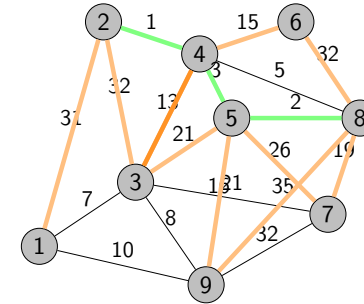
Startknoten: 2

Algorithmus von Prim



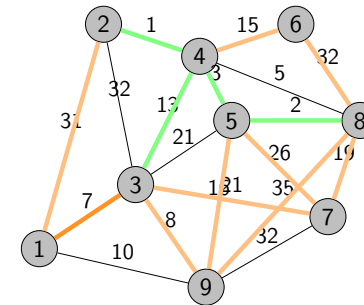
Startknoten: 2

Algorithmus von Prim



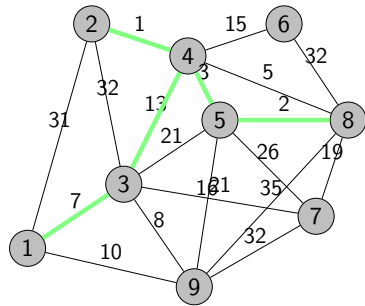
Startknoten: 2

Algorithmus von Prim



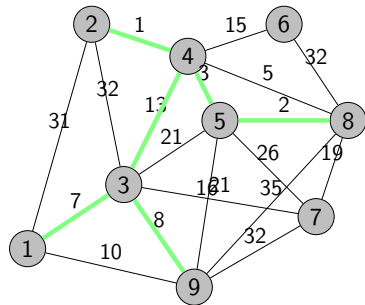
Startknoten: 2

Algorithmus von Prim



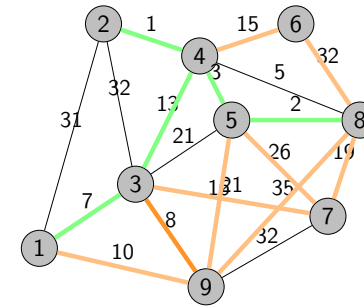
Startknoten: 2

Algorithmus von Prim



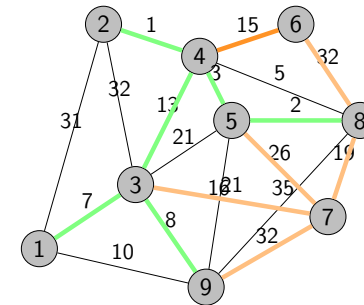
Startknoten: 2

Algorithmus von Prim



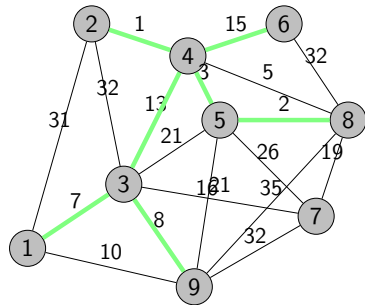
Startknoten: 2

Algorithmus von Prim



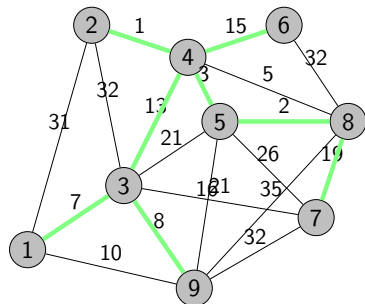
Startknoten: 2

Algorithmus von Prim



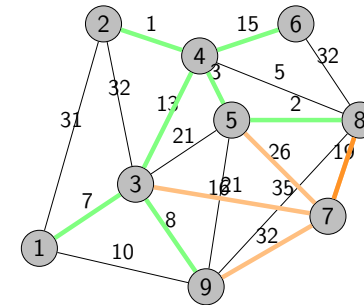
Startknoten: 2

Algorithmus von Prim



Startknoten: 2

Algorithmus von Prim



Startknoten: 2

Vergleich: Kruskal vs. Prim

- Beide Algorithmen (Kruskal und Prim) finden den MST in gewichteten Graphen.
- Bei *dicht*² besetzten Graphen ist der Algorithmus von Prim jedoch effizienter.
- Bei *dünn*³ besetzten Graphen ist Kruskal besser.
- Beim Algorithmus von Kruskal müssen die Kanten vorab sortiert werden, was bei vielen Kanten einen erheblichen Aufwand darstellt (sogar den Hauptaufwand des gesamten Verfahrens).
- Bei vergleichsweise wenigen Kanten überwiegen jedoch die Vorteile der einfacheren darauffolgenden Schritte.

² $|E| \in O(|V|^2)$, d.h. die Anzahl der Kanten liegt in der Größenordnung der Anzahl der Kanten eines vollständigen Graphen.

³ $|E| \in O(|V|)$, d.h. die Anzahl der Kanten liegt in der Größenordnung der Anzahl der Knoten; der Graph enthält also relativ wenige Kanten.

Zusammenfassung (MST)

- Beide Algorithmen (Kruskal, Prim) sind sogenannte **Greedy-Algorithmen**
- Sie wählen in jedem Schritt ("gierig") die nächst-beste Erweiterung der Teillösung
- Normalerweise erreicht man mit einer derartigen Strategie nur **Näherungslösungen** (d.h. nicht die insgesamt beste Lösung)
- In diesem Fall finden jedoch beide Greedy-Algorithmen das **globale Optimum**, d.h. die insgesamt beste Lösung
- ...der Minimale Spannbaum kann also (wie der Euler-Zyklus) leicht gefunden werden.
- Andere Spannbaum-Probleme (Varianten) sind jedoch wesentlich schwieriger!

Algorithmus von Dijkstra

Programmieren und Software-Engineering Theorie

26. Februar 2026

POS (Theorie)

Graphentheorie

1 / 6

Algorithmus von Dijkstra

Algorithmus 1: DIJKSTRA

Data: Graph G mit $w_{ij} \geq 0$ für alle $(i, j) \in E(G)$

Data: Startknoten s

```

1  $\forall v \in V : \delta_v \leftarrow \infty;$ 
2  $\delta_s \leftarrow 0;$ 
3 Prioritätswarteschlange  $Q$  befüllt mit allen Knoten ;
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow Q.\text{getMin}();$  // entnimmt Knoten  $u$  mit kleinstem  $\delta_u$ 
6   Fertigstellung von Knoten  $u$ ;
7   Speichere Verweis auf direkten Vorgänger von  $u$ ;
8   for all  $(u, v) \in E, v$  noch nicht fertiggestellt do
9     if  $\delta_v > \delta_u + w_{uv}$  then
10       $\delta_v \leftarrow \delta_u + w_{uv};$ 

```

Anmerkung: Die Prioritätswarteschlange Q kann durch ein einfaches Array umgesetzt werden. Um u mit kleinstem δ_u zu finden, muss es zur Gänze durchlaufen werden. Dies ist nachteilig für die Performance des Algorithmus, weshalb die Prioritätswarteschlange meist anhand eines *Heaps* umgesetzt wird.

POS (Theorie)

Graphentheorie

3 / 6

Algorithmus von Dijkstra

Der Algorithmus von Dijkstra berechnet die kürzesten Wege in einem gewichteten Graphen mit $w_{ij} \geq 0$, für alle $[i, j] \in E$.

Grundidee:

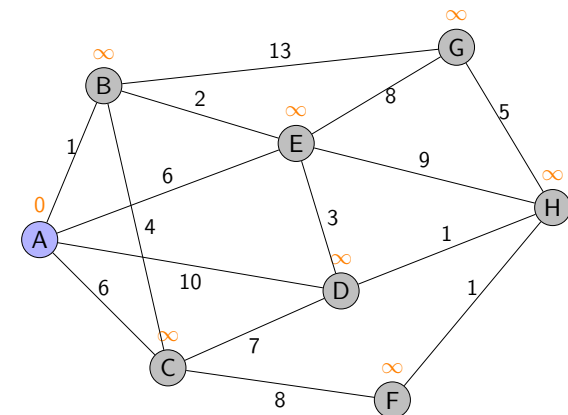
- Ähnlichkeit zu BFS, jedoch andere Regeln für die Auswahl des nächsten Knoten.
- Ein Aufruf von Dijkstra berechnet die kürzesten Wege von einem Startknoten zu allen anderen Knoten des Graphen.
- In jedem Schritt werden **Zwischenergebnisse** $\delta_k, k \in V$ berechnet, bzw. aktualisiert.
- Diese Zwischenergebnisse sind die Länge des kürzesten *bisher gefundenen* Weges bis zu diesem Knoten.
- Wiederhole (bis alle Knoten abgeschlossen):
 - 1 Wähle Knoten k mit minimalem δ_k und **schließe diesen ab**.
 - 2 Speichere **Verweis auf direkten Vorgänger**.
 - 3 Aktualisiere die Werte δ_k für noch nicht abgeschlossene Nachbarknoten von k .

POS (Theorie)

Graphentheorie

2 / 6

Algorithmus von Dijkstra: Beispiel



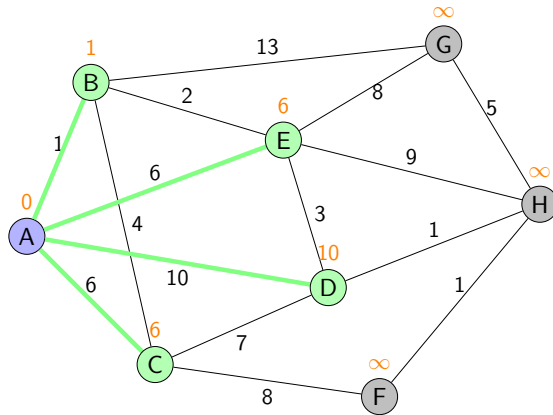
Wir suchen den kürzesten Weg vom Knoten A zum Knoten H. Im ersten Schritt wird der Startknoten **"fertiggestellt"**.

POS (Theorie)

Graphentheorie

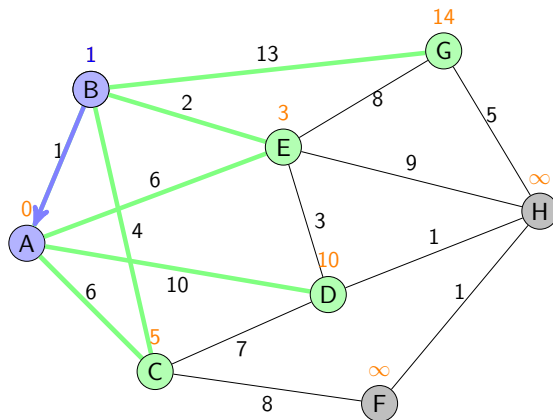
4 / 6

Algorithmus von Dijkstra: Beispiel



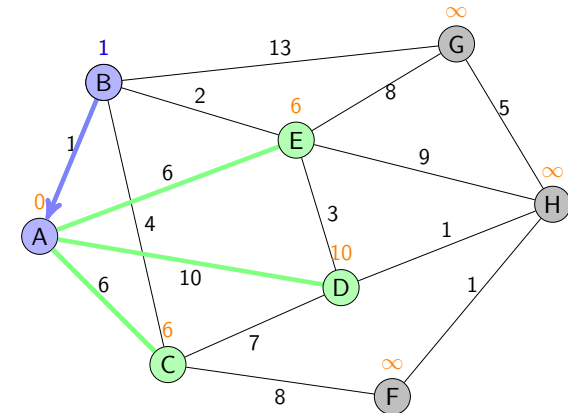
Im nächsten Schritt werden die Nachbarknoten B, C, D und E **entdeckt**. Die Zwischenwerte werden wie folgt berechnet:
 $\delta_A = 0, \delta_B = \delta_A + 1 = 1, \delta_E = \delta_A + 6 = 6, \delta_D = \delta_A + 10 = 10, \delta_C = \delta_A + 6 = 6$.

Algorithmus von Dijkstra: Beispiel



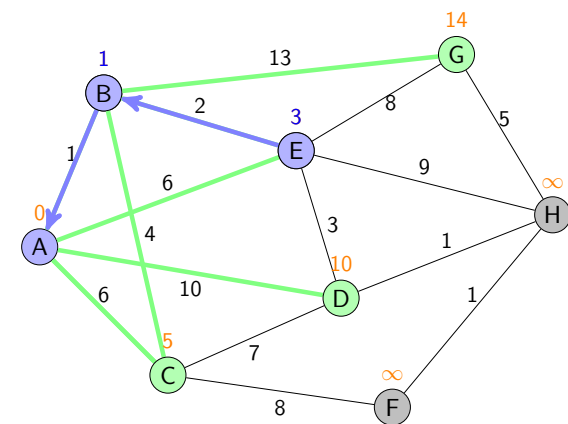
Ausgehend vom letzten fertiggestellten Knoten (B) werden nun neue Zwischenergebnisse für C, E und G berechnet. Wir erhalten $\delta_G = 1 + 13 = 14, \delta_E = 1 + 2 = 3, \delta_C = 1 + 4 = 5$. Für die Knoten C und E erhalten wir kleinere Werte als die bisher gefundenen.

Algorithmus von Dijkstra: Beispiel



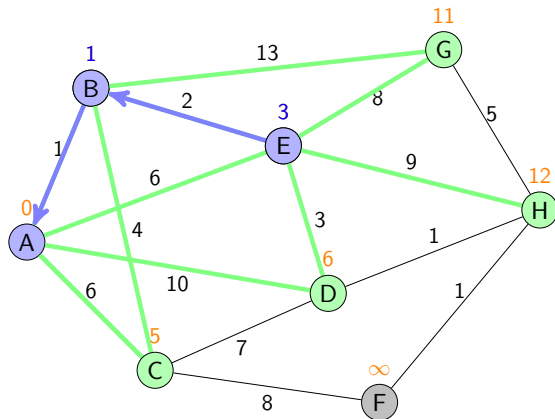
Nun wird der Knoten v mit kleinstem δ_v **fertiggestellt**. Im konkreten Beispiel ist dies der Knoten B .

Algorithmus von Dijkstra: Beispiel



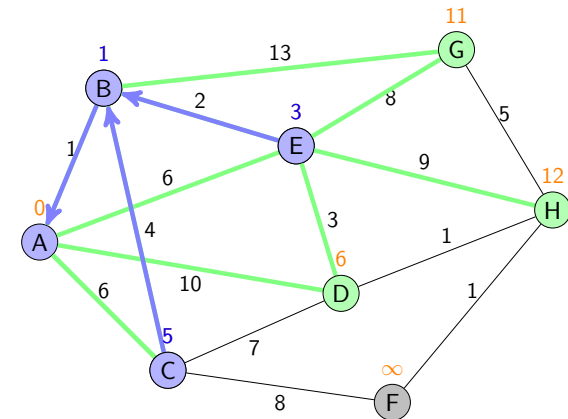
Knoten E wird fertiggestellt. Bei fertiggestellten Knoten merkt man sich wo man hergekommen ist (daher die blaue Kante).

Algorithmus von Dijkstra: Beispiel



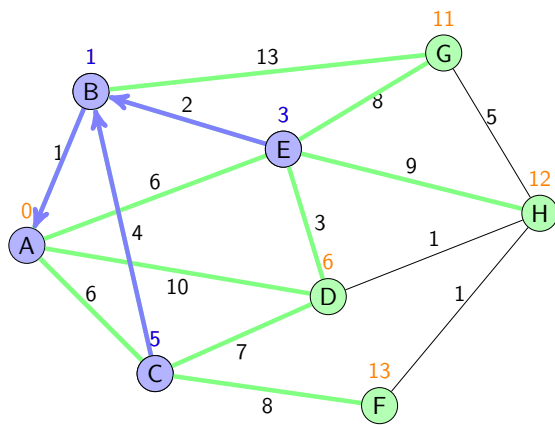
Berechnung neuer Zwischenergebnisse für D, G und H.

Algorithmus von Dijkstra: Beispiel



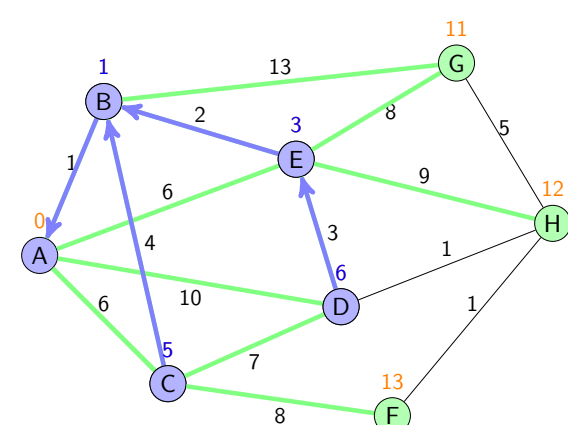
Knoten C wird fertiggestellt, da er nun den kleinsten Zwischenwert δ hat.

Algorithmus von Dijkstra: Beispiel



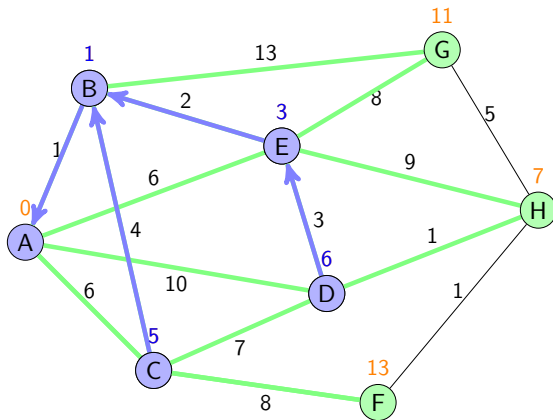
Ausgehend von C werden die Werte δ_D und δ_F berechnet. Da $5 + 7 > 6$ kommt es bei δ_D zu keiner Änderung.

Algorithmus von Dijkstra: Beispiel



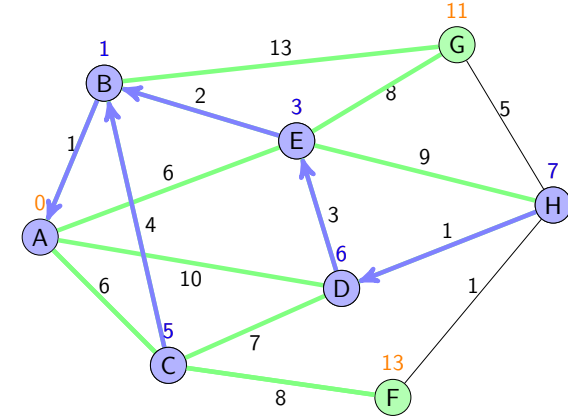
Fortgefahren wird mit Knoten D, da kleinstes δ .

Algorithmus von Dijkstra: Beispiel



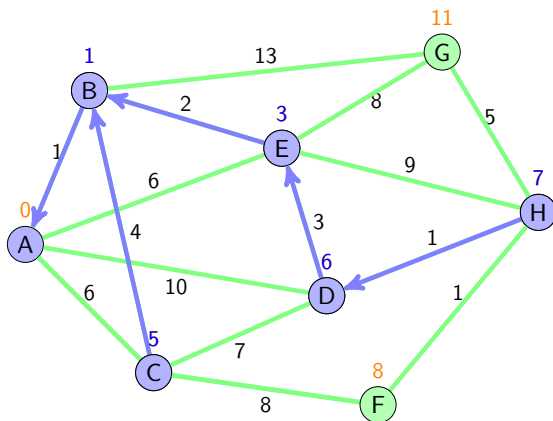
δ_H wird aktualisiert.

Algorithmus von Dijkstra: Beispiel



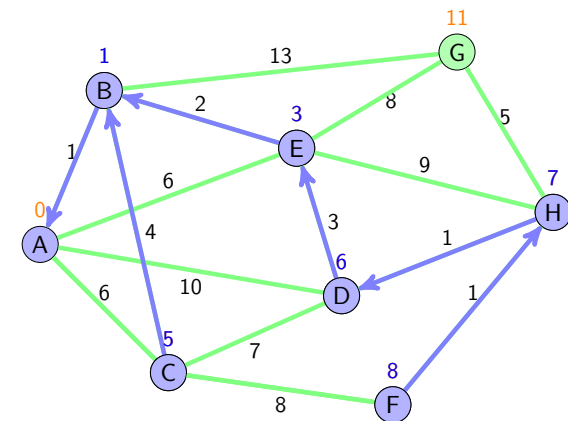
H wird fertiggestellt.

Algorithmus von Dijkstra: Beispiel



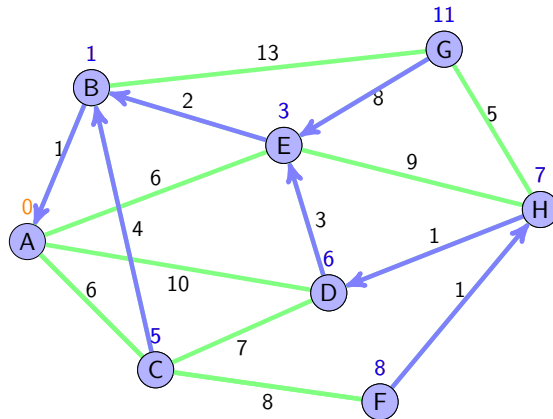
Update von δ_F und δ_G , wobei nur δ_F tatsächlich geändert wird.

Algorithmus von Dijkstra: Beispiel



F wird fertiggestellt.

Algorithmus von Dijkstra: Beispiel



G wird fertiggestellt (Vorgänger E).

Algorithmus von Dijkstra

- Die blauen Kanten bilden einen Wurzelbaum¹, der die kürzesten Wege vom Startknoten zu jedem Knoten enthält.
- Die Berechnung des kürzesten Weges von einem Startknoten zu einem Zielknoten beinhaltet also die Berechnung der kürzesten Wege vom Startknoten zu *allen* anderen Knoten.
- Ist nur der kürzeste Weg zu einem Zielknoten von Interesse, kann die Berechnung abgebrochen werden sobald dieser fertiggestellt wird.

¹streng genommen bilden die umgedrehten blauen Kanten einen Wurzelbaum
POS (Theorie) Graphentheorie

Algorithmus von Dijkstra – Laufzeitanalyse

Mit $n = |V|$ und $m = |E|$ können wir die Laufzeiteigenschaften angeben. Diese hängen von der konkreten Umsetzung der Prioritätswarteschlange Q ab.

Operation		Queue Implementierung		
Name	Anzahl	Liste	Heap	Fibonacci Heap ²
decreaseKey [10]	m	$O(1)$	$O(\log n)$	$O(1)$
getMin [5]	n	$O(n)$	$O(\log n)$	$O(\log n)$
create [3]	1	$O(n)$	$O(n)$	$O(n)$
Gesamt		$O(n^2 + m)$ $= O(n^2)$	$O((n + m) \log n)$	$O(n \log n + m)$

Anmerkung: In der Spalte ganz links ist in eckigen Klammern auf die Zeile der jeweiligen Operation im Pseudocode verwiesen!

²amortisierte Laufzeit

O-Notation

Programmieren und Software-Engineering Theorie

26. Februar 2026

POS (Theorie)

O-Notation

1 / 18

Komplexität von Algorithmen

Die theoretische Analyse von Algorithmen untersucht:

- (Lauf-)Zeit-Komplexität
- (Speicher-)Platz-Komplexität
- Evtl. weitere Parameter, z.B.:
 - Anzahl Vergleiche
 - Anzahl Bewegungen der Datensätze

Weitere Unterscheidung:

- **Worst-Case:** der ungünstigste Fall
- **Average-Case:** der zu erwartende, durchschnittliche Fall
- **Best-Case**

POS (Theorie)

O-Notation

3 / 18

Motivation

- Bestimmte Programmteile verursachen je nach Datenmenge unterschiedliche *Laufzeiten* bei der Ausführung.
- Bei geringen Datenmengen bleibt dies oft unbemerkt und spielt keine Rolle.
- Reale Programme arbeiten jedoch mit großen Datenmengen, umfangreichen Datenbanken, führen komplexe Berechnungen durch, etc.
- Durch die theoretische Laufzeitanalyse kann man etwaige Flaschenhälse und Performanceprobleme erkennen.
- Relevant ist hierbei weniger die konkrete Ausführungsgeschwindigkeit, sondern das Wachstum der Laufzeit in Abhängigkeit von der Anzahl der Eingabedaten.
- In der praktischen Arbeit werden *Profiler* verwendet, um zu analysieren wieviel Zeit in welchen Programmteilen verbracht wird.
- Für theoretische Analysen wird die *O-Notation* verwendet.

POS (Theorie)

O-Notation

2 / 18

Komplexität von Algorithmen

- Untersucht werden die Eigenschaften von Algorithmen in Abhängigkeit von den Eingabedaten.
- Wesentliche Kenngröße ist die Anzahl der Daten, oft mit n oder m bezeichnet.
 - *Beispiel:* Sortieren von n Zahlen
 - *Beispiel:* Berechnen eines Euler-Zyklus von einem Graphen mit n Knoten und m Kanten
 - *Beispiel:* Tiefensuche in Graphen mit n Knoten und m Kanten
 - *Beispiel:* Suchen eines Elementes in n vorhandenen Einträgen
- Zentrale Frage: wie hängt die Laufzeit von n (oder m) ab?
- Von Interesse ist lediglich die **Größenordnung**!
- Diese Größenordnung wird mit der Bachmann-Landau-Notation, oder kurz **O-Notation** dargestellt.

POS (Theorie)

O-Notation

4 / 18

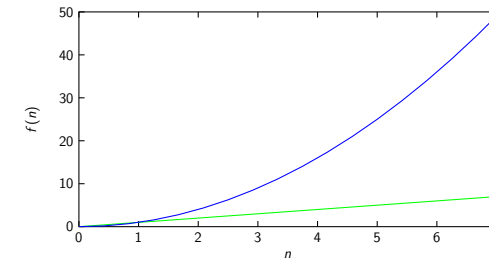
Komplexität von Algorithmen

- Ein sehr günstiges Laufzeitverhalten ist, wenn bei n Datensätzen **ca.** n Schritte ausgeführt werden.
 - Man nennt dies: **lineares** Laufzeitverhalten
 - Der Algorithmus durchläuft $O(n)$ Schritte
- Werden ungefähr n^2 viele Schritte ausgeführt:
 - **Quadratisches** Laufzeitverhalten
 - Der Algorithmus durchläuft $O(n^2)$ Schritte
- Höhere Laufzeiten sind oftmals in der Praxis problematisch, selbst bei quadratischer Laufzeit ist oft schon Vorsicht angebracht

Definition ((informell) Laufzeit eines Algorithmus anhand der O-Notation)

Ein Algorithmus A hat bei n Eingabedaten eine Laufzeit von $O(f(n))$, wenn bei seiner Ausführung *in etwa* ("in der Größenordnung von") $f(n)$ Schritte ausgeführt werden.

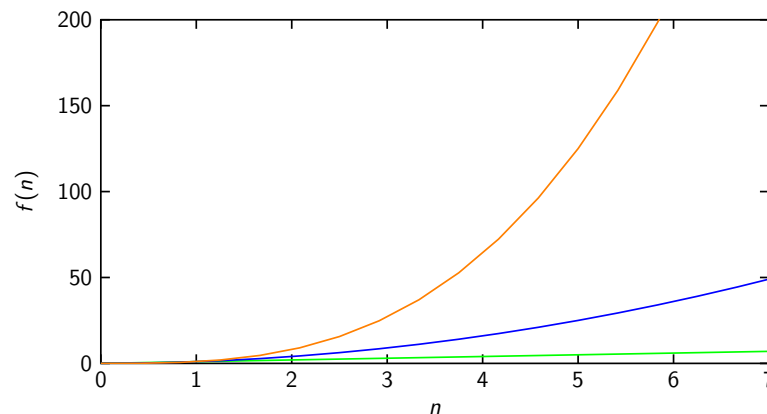
Wachstum von $f(n)$



$$f(n) = n, f(n) = n^2$$

Vergleich von linearem zu quadratischem Laufzeitverhalten

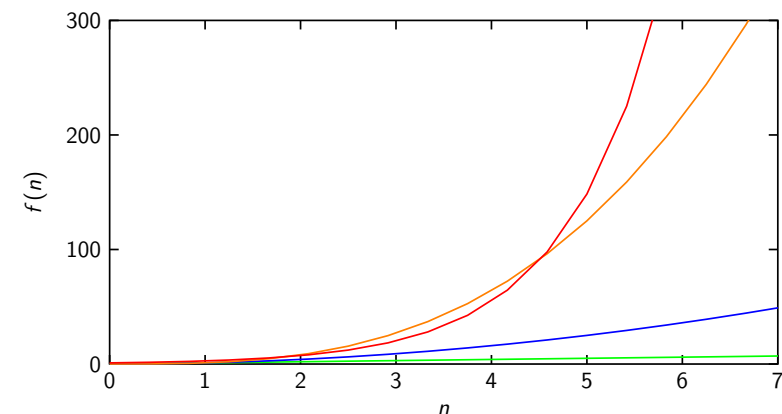
Wachstum von $f(n)$



$$f(n) = n, f(n) = n^2, f(n) = n^3$$

Vergleich von n , n^2 und n^3 (jeweils polynomiell)

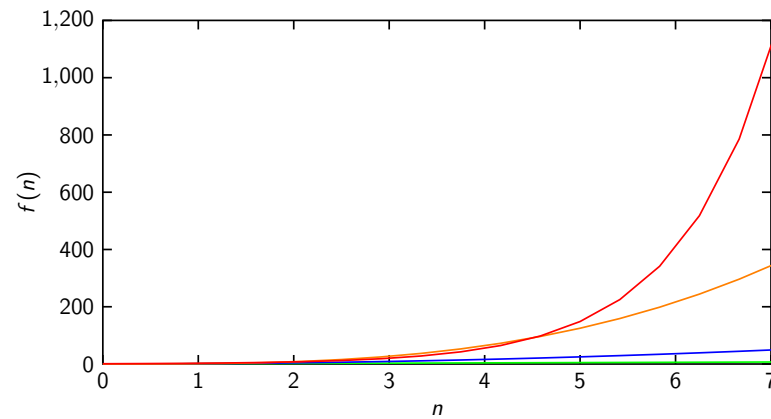
Wachstum von $f(n)$



$$f(n) = n, f(n) = n^2, f(n) = n^3, f(n) = e^n$$

Zusätzliche exponentielle Kurve, zunächst mit $y_{\max} = 300$

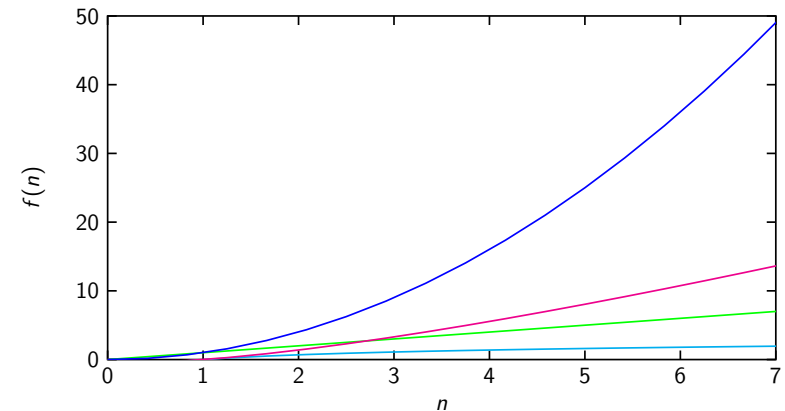
Wachstum von $f(n)$



$$f(n) = n, f(n) = n^2, f(n) = n^3, f(n) = e^n$$

Zusätzliche exponentielle Kurve, jetzt mit $y_{\max} = 1200$

Wachstum von $f(n)$



$$f(n) = n, f(n) = n^2, f(n) = \ln n, f(n) = n \ln n$$

Weiteres Beispiel: die Logarithmus-Funktion. Diese strebt zwar gegen Unendlich, aber extrem langsam! Wichtige Konsequenz: $n \ln n$ verhält sich "fast" wie n

O-Notation

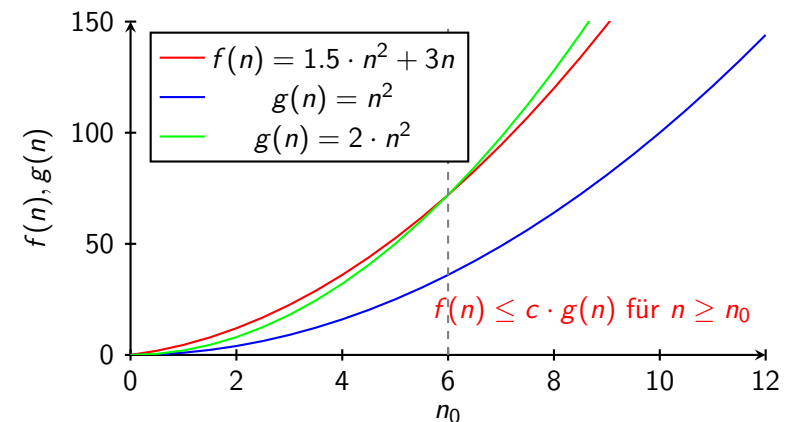
Eine Funktion $f(n)$ liegt in $O(g(n))$ wenn ab einem Wert n_0 und einer Konstante c die Funktionswerte von $f(n)$ kleiner sind als jene von $c \cdot g(n)$. Man schreibt hierfür auch $f(n) \in O(g(n))$.

Definition (Bachmann-Landau O-Notation)

$$O(g(n)) = \{f(n) \mid (\exists c, n_0 > 0), (\forall n \geq n_0) : 0 \leq f(n) \leq c g(n)\}$$

Somit ist $c \cdot g(n)$ ab n_0 eine **obere Schranke** für $f(n)$! Es müssen immer nur die Terme der höchsten Ordnung betrachtet werden, und diese ohne etwaige konstante Faktoren.

O-Notation



Funktion $f(n) \in O(g(n))$. x-Achse: n , y-Achse: Funktionswerte $f(n), g(n), c \cdot g(n)$.

Bachmann-Landau Notation (2)

- Obere Schranke:

$$O(g(n)) = \{f(n) \mid (\exists c, n_0 > 0), (\forall n \geq n_0) : 0 \leq f(n) \leq cg(n)\}$$

- Untere Schranke:

$$\Omega(g(n)) = \{f(n) \mid (\exists c, n_0 > 0), (\forall n \geq n_0) : 0 \leq cg(n) \leq f(n)\}$$

- Genaue Wachstumsrate:

$$\Theta(g(n)) = \{f(n) \mid (\exists c_1, c_2, n_0 > 0), (\forall n \geq n_0) : 0 \leq c_1g(n) \leq f(n) \leq c_2g(n)\}$$

Beispiele

Bei der O-Notation spielen also Koeffizienten und Terme niedrigerer Ordnung keine Rolle, und können generell weggelassen werden.

Beispiele:

- $3n^2 \in O(n^2)$
- $3n^2 \cdot 5n^3 \in O(n^5)$
- $\frac{3}{4}n^3 + 5n^2 \in O(n^3)$
- $42n + \ln n + 5150 \in O(n)$
- $n \log n \in O(n \log n)$
- $n \log n \notin O(n)$

Bei zwei unabhängigen Variablen müssen beide in der O-Notation berücksichtigt werden:

- $n + m^2 \in O(n + m^2)$

Die O-Notation ist im Allgemeinen eine beliebige obere Schranke. Somit gilt auch $n^2 \in O(n^3)$. Üblicherweise ist man aber an der kleinsten oberen Schranke interessiert!

Rechenregeln

Addition

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$$

Multiplikation

$$O(f(n)) \cdot O(g(n)) = O(f(n)g(n))$$

Beispiel

Beispiel: Verschachtelte Schleifen:

```
for i = 1 to n:           // O(n)
  for j = 1 to n:         // O(n)
    konstante Operation // O(1)
```

- Gesamt: $O(n) \cdot O(n) \cdot O(1) = O(n^2)$
- Ebenso ist die Laufzeit in $\Theta(n^2)$

Best-, Average-, Worst-Case

Beispiel: Lineare Suche: Suche nach Element x in Array der Größe n :

- Best-Case: x steht an erster Position $\Rightarrow O(1)$
- Worst-Case: x ist letzte Position oder nicht vorhanden $\Rightarrow O(n)$
- Average-Case (gleichverteilte Position): erwartete Position $n/2 \Rightarrow O(n)$

Im Worst-Case gilt auch $\Theta(n)$.

Amortisierte Analyse

- klassische Analyse: Kosten einer einzelnen Operation werden isoliert betrachtet
- Manche Datenstrukturen haben jedoch einzelne teure Operationen, die durch viele kostengünstige ausgeglichen werden.
- **Amortisierte Laufzeit** einer Operation: *durchschnittliche Laufzeit* pro Operation über eine Folge von Operationen, wobei teure Operationen durch viele günstigere ausgeglichen werden.

Rekursion

Programmieren und Software-Engineering Theorie

26. Februar 2026

POS (Theorie)

Rekursion

1 / 15

Fakultät

In der Mathematik bezeichnet $n!$ die Fakultät, und es gilt

$$n! = n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$$

für alle $n > 0$ und $0! = 1$.

Rekursive Formulierung der Fakultät:

$$n! = n \cdot (n-1)! \quad \text{für } n > 0$$

$$0! = 1$$

Direkte Umsetzung in Programmcode:

```
1 long factorial(long n) {
2     if (n == 0) return 1;
3     return n * factorial(n-1);
4 }
```

Achtung: der Aufruf `factorial(-1)` führt zu einer endlosen Rekursion!
Dieser Fall sollte im Code abgefangen werden.

POS (Theorie)

Rekursion

3 / 15

Definition

Definition (Rekursion)

Unter einer rekursiven Methode versteht man eine Methode die sich selbst aufruft.

- Die Definition gilt auch für *Funktionen* oder *Programme* (statt Methoden).
- Viele Problemstellungen/Algorithmen lassen sich sehr einfach und elegant mittels Rekursion formulieren.
- Abbruchbedingung: muss vorhanden sein um die rekursiven Aufrufe zu beenden.

POS (Theorie)

Rekursion

2 / 15

Negativbeispiel: Fibonacci-Zahlen

Die Folge der Fibonacci-Zahlen ist für $n \geq 2$ definiert als

$$f_n = f_{n-1} + f_{n-2},$$

weitere gilt $f_0 = f_1 = 1$.

Hierdurch erhalten wir die Folge:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, ...

POS (Theorie)

Rekursion

4 / 15

Negativbeispiel: Fibonacci-Zahlen

Die direkte Umsetzung als rekursives Programm ist jedoch sehr unvorteilhaft:

```

1  long fibonacci(long n) {
2      if (n <= 1) return 1;
3      return fibonacci(n-1) + fibonacci(n-2);
4  }
```

Frage

Wo genau liegt das Problem bei der rekursiven Implementierung der Fibonacci-Zahlen?

Negativbeispiel: Fibonacci-Zahlen

Die *iterative* Implementierung ist hier vorteilhaft, da wesentlich weniger Berechnungen ausgeführt werden:

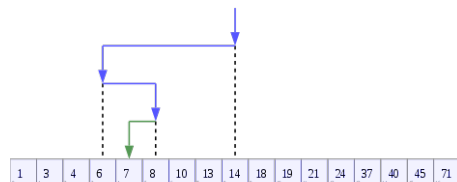
```

1  long fibonacci(long n) {
2      if (n < 0 || n > MAX) {
3          // Ausnahmebehandlung
4      }
5      long[] f = new long[n+1];
6      f[0] = 1; f[1] = 1;
7      for (long i=2; i<=n; i++) {
8          f[i] = f[i-1] + f[i-2];
9      }
10     return f[n];
11 }
```

Anmerkung: Die Berechnung des n -ten Folgigliedes ist auch ohne Array möglich!

Suche in sortierten Listen

- Aufgabenstellung: Suchen eines Elementes in einer sortierten Liste (Array)
- Naiver Zugang: $O(n)$ Schritte (Erwartungswert $n/2$)
- Mittels binärer Suche: nur $O(\log n)$ Schritte notwendig
- Vorgehensweise: analog zu Suche in Telefonbuch
 - Beliebige Seite aufschlagen
 - Steht Name davor oder danach?
 - Weitere Suche erfolgt nur mehr im verbleibenden Teil



Binäre Suche

Algorithmus 1: (Iterative) Binäre Suche

```

1  Function BinarySearch(sorted array a[], element e)
    Result: found element, or NULL if element not exists
2      l = 0;
3      r = a.length;
4      while l <= r do
5          m = l + (r-l)/2;
6          if (a[m] == e) then
7              return e;
8          else
9              if a[m] > e then
10                 r = m - 1;
11             else
12                 l = m + 1;
13     return NULL;
```

Rekursive Binäre Suche

Algorithmus 2: Rekursive Binäre Suche

```

1 Function RecBinarySearch(sorted array  $a$ [], element  $e$ ,  $l$ ,  $r$ )
   Result: found element, or NULL if element not exists
2   if  $l \leq r$  then
3      $m = l + \frac{(r-l)}{2}$ ;
4     if ( $a[m] == e$ ) then
5       return  $e$ ;
6     else
7       if  $a[m] < e$  then
8         return RecBinarySearch( $a$ ,  $e$ ,  $m+1$ ,  $r$ );
9       else
10        return RecBinarySearch( $a$ ,  $e$ ,  $l$ ,  $m-1$ );
11   else
12     return NULL;

```

Aufruf mit array a und gesuchtem Element e und RecBinarySearch(a , e , 0, $a.length-1$).

Tiefensuche

Bei der Tiefensuche ist eine rekursive Implementierung naheliegend. Diese Variante durchläuft alle vom ersten Aufruf mit Knoten $v \in V(G)$ (RecDFS(v)) aus erreichbaren Knoten des Graphen G .

Algorithmus 3: Rekursive Tiefensuche

```

1 Function RecDFS(Knoten  $v$ )
2   markiere  $v$  als besucht;
3   for all  $[v, u] \in E(G)$  do
4     if  $u$  noch nicht besucht then
5       RecDFS( $u$ );

```

In der rekursiven Version der Tiefensuche ist kein *Stack* notwendig, die Knoten werden durch die rekursiven Aufrufe automatisch in der richtigen Reihenfolge besucht.

Rekursive Binäre Suche

Eine rekursive Variante in Python:

```

1 def binaersuche_rekursiv(werte, gesucht, start, ende):
2     if ende < start:
3         return 'nicht gefunden'
4         # alternativ: return -start # bei (-Returnwert) waere
5         # die richtige Einfuege-Position
6
7     mitte = (start + ende) // 2
8     if werte[mitte] == gesucht:
9         return mitte
10    elif werte[mitte] < gesucht:
11        return binaersuche_rekursiv(werte, gesucht, mitte+1, ende)
12    else:
13        return binaersuche_rekursiv(werte, gesucht, start, mitte-1)
14
15 def binaersuche(werte, gesucht):
16     return binaersuche_rekursiv(werte, gesucht, 0, len(werte)-1)

```

Quelle: Wikipedia

Tiefensuche

Diese Variante sucht einen Knoten s und returniert ihn sobald gefunden:

Algorithmus 4: Rekursive Tiefensuche (2)

```

1 Function RecDFS(Knoten  $v$ , Gesuchter Knoten  $s$ )
2   if  $v$  gesuchter Knoten  $s$  then
3     return  $v$ ;
4   markiere  $v$  als besucht;
5   for all  $[v, u] \in E(G)$  do
6     if  $u$  noch nicht besucht then
7       if RecDFS( $u$ ,  $s$ ) ==  $s$  then
8         return  $s$ ;
9   return null;

```

Programmierbeispiel 12.1.1

Gegeben ist ein 2-dimensionales Boolean-Array, befüllt mit *wahr* und *falsch*. Ermitteln Sie mit einem Programm die Größe des größten zusammenhängenden Gebietes mit Wert wahr.

Beispiel: hier wird falsch mit einem '.' und wahr mit einem 'X' dargestellt.

```

...XXX..XX..
.XXX.XXXX...
X.XX..XX..XX
X...XXX..XXX
XXX...XXX..
XX.XX..X..XX
...X..X.XXX.
....XXXX...

```

In diesem Beispiel ist die Größe 19.

```

1 private static boolean[][] werte = {
2     {false,false,false,true,true,true,false,false,true,true,false,false},
3     {false,true,true,true,false,true,true,true,true,false,false,false},
4     {true,false,true,true,false,false,true,true,false,false,true,true},
5     {true,false,false,false,true,true,true,false,false,true,true,true},
6     {true,true,true,false,false,false,false,true,true,true,false,false},
7     {true,true,false,true,true,false,false,true,false,false,true,true},
8     {false,false,false,true,false,false,true,false,true,true,true,false},
9     {false,false,false,false,false,true,true,true,true,false,false,false}
10 };

```

Zusatzaufgabe 12.1.2

Füllen Sie ein zweidimensionales Boolean-Array (Größe 300x300) mit Zufallswerten, wobei mit einer Wahrscheinlichkeit von $\frac{1}{3}$ der Wert wahr, und mit $\frac{2}{3}$ der Wert falsch gesetzt werden soll. Wenden Sie den Algorithmus aus dem vorigen Beispiel an. Ist das Ergebnis plausibel? Was passiert wenn man die Wahrscheinlichkeiten tauscht?

Programmierbeispiel 12.2.1

Implementieren Sie Mergesort in Java.

Programmierbeispiel 12.2.2

Implementieren Sie Quicksort in Java.

Sortierv Verfahren

Programmieren und Software-Engineering Theorie

26. Februar 2026

- Wir betrachten einfache Sortialgorithmen als Beispiele für Algorithmen
- Ziel: Sortiere die Zahlen in einem Array (aufsteigend)

Array mit 10 Integer-Elementen

9	4	12	1	7	3	10	5	2	8
0	1	2	3	4	5	6	7	8	9

Abbildung: Array mit 10 Elementen (Index unter den Zellen).

POS (Theorie)

Sortierv Verfahren

1 / 25

Bubblesort

Algorithmus 1: Bubblesort

Input: Array a

Result: Sortiertes Array a

```
1 Function BUBBLESORT(a)
2   for i = 0; i < a.length - 1; i ++ do
3     swapped = false
4     for j = 0; j < a.length - 1 - i; j ++
5       do
6         if a[j] > a[j + 1] then
7           SWAP(a, j, j+1)
8           swapped = true
9   if notswapped then
10    Break
```

POS (Theorie)

Sortierv Verfahren

3 / 25

POS (Theorie)

Sortierv Verfahren

2 / 25

Bubblesort (Beispiel)

Durchlauf i	Array-Zustand
Initial	9 4 12 1 7 3 10 5 2 8
0	4 9 1 7 3 10 5 2 8 12
1	4 1 7 3 9 5 2 8 10 12
2	1 4 3 7 5 2 8 9 10 12
3	1 3 4 5 2 7 8 9 10 12
4	1 3 4 2 5 7 8 9 10 12
5	1 3 4 2 5 7 8 9 10 12
6	1 3 2 4 5 7 8 9 10 12
7	1 2 3 4 5 7 8 9 10 12
8	1 2 3 4 5 7 8 9 10 12 (sortiert)

POS (Theorie)

Sortierv Verfahren

4 / 25

Insertionsort

Algorithmus 2: Insertionsort**Input:** Array a**Result:** Sortiertes Array a

```

1 Function INSERTIONSORT(a)
2    $i = 1$ 
3   while  $i < a.length$  do
4      $j = i$ 
5     while  $j > 0 \wedge a[j-1] > a[j]$  do
6       SWAP(a, j, j-1)
7        $j = j - 1$ 
8      $i = i + 1$ 

```

Selectionsort

Algorithmus 3: Selectionsort**Input:** Array a**Result:** Sortiertes Array a

```

1 Function SELECTIONSORT(a)
2   for  $i = 0; i < a.length - 1; i++$  do
3      $jMin = i$ 
4     for  $j = i + 1; j < a.length; j++$  do
5       if  $a[j] < a[jMin]$  then
6          $jMin = j$ 
7     if  $jMin \neq i$  then
8       SWAP(a, i, jMin)

```

Insertionsort (Beispiel)

i (nächster Index)	Array-Zustand (linker Teil sortiert)
Initial	9 4 12 1 7 3 10 5 2 8
1	4 9 12 1 7 3 10 5 2 8
2	4 9 12 1 7 3 10 5 2 8
3	1 4 9 12 7 3 10 5 2 8
4	1 4 7 9 12 3 10 5 2 8
5	1 3 4 7 9 12 10 5 2 8
6	1 3 4 7 9 10 12 5 2 8
7	1 3 4 5 7 9 10 12 2 8
8	1 2 3 4 5 7 9 10 12 8
9	1 2 3 4 5 7 8 9 10 12 (sortiert)

Selectionsort

i	Array-Zustand
Initial	9 4 12 1 7 3 10 5 2 8
0	1 4 12 9 7 3 10 5 2 8
1	1 2 12 9 7 3 10 5 4 8
2	1 2 3 9 7 12 10 5 4 8
3	1 2 3 4 7 12 10 5 9 8
4	1 2 3 4 5 12 10 7 9 8
5	1 2 3 4 5 7 10 12 9 8
6	1 2 3 4 5 7 8 12 9 10
7	1 2 3 4 5 7 8 9 12 10
8	1 2 3 4 5 7 8 9 10 12 (sortiert)

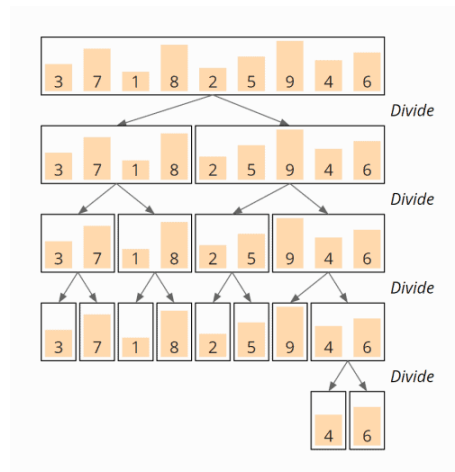
Laufzeitanalyse

Worst-Case Analyse einfacher Sortieralgorithmen bei n Elementen:

Algorithmus	Vergleiche	Swaps	Gesamt
Bubblesort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertionsort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selectionsort	$O(n^2)$	$O(n)$	$O(n^2)$

Die Gesamtlaufzeit von $O(n^2)$ ist in der Praxis problematisch, d.h. zu langsam. Daher kommen üblicherweise bessere Algorithmen zum Einsatz, die eine Gesamtlaufzeit von $O(n \log n)$ aufweisen.

Mergesort

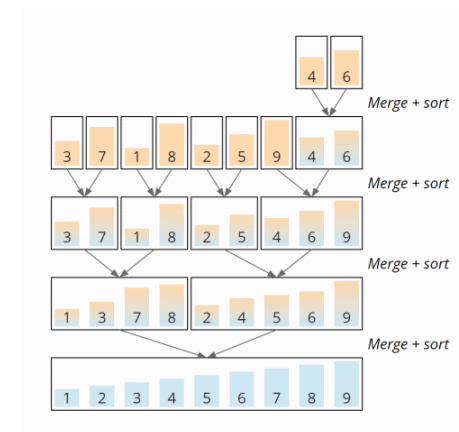


Quelle: <https://www.happycoders.eu/de/algorithmen/mergesort/>

Mergesort

- Der Algorithmus *Mergesort* sortiert die Daten nach dem Prinzip **Divide & Conquer** (dt.: Teile und Herrsche)
- Vorgehensweise bei Divide & Conquer:
 - Teile das Problem in kleinere Teilprobleme
 - Löse diese Teilprobleme
 - Füge Teillösungen zusammen
- Mergesort**
 - Zahlenfolge (Array) wird durch rekursive Aufrufe unterteilt.
 - Die Sortierung wird beim anschließenden Zusammenfügen der Arrays erreicht
 - Mergesort weist bessere Laufzeit-Eigenschaften als die bisher besprochenen Sortieralgorithmen auf!

Mergesort



Quelle: <https://www.happycoders.eu/de/algorithmen/mergesort/>

Mergesort

- Rekursive Mergesort-Implementierung
- Aufruf mit MERGESORT($A, 0, \text{len}(A)$)

Algorithmus 4: Mergesort

Input: Array $A[l..r]$

```

1 Function MERGESORT( $A, l, r$ )
2   if  $l = r$  then
3     return  $A[l]$ 
4    $m = l + \lfloor (r - l) / 2 \rfloor$ 
5    $L = \text{MERGESORT}(A, l, m)$ 
6    $R = \text{MERGESORT}(A, m + 1, r)$ 
7   return MERGE( $L, R$ )

```

POS (Theorie)

Sortierv Verfahren

13 / 25

Quicksort

- Effizienter Sortieralgorithmus von Tony Hoare (1959)
- Ebenso Divide & Conquer
- Array wird anhand von *Pivot-Element* rekursiv geteilt.
- Letztes Element im Teil-Array ist Pivot-Element¹.
- Trotz Worst-Case-Laufzeit von $O(n^2)$ in der Praxis schneller als Mergesort.
- Der Average-Case $O(n \log n)$ tritt so gut wie immer ein!

¹Es gibt viele Varianten zur Wahl des Pivot-Elements.

POS (Theorie)

Sortierv Verfahren

15 / 25

Merge-Methode

Algorithmus 5: Merge zweier sortierter Listen

Input: sortierte Arrays $L[0..a-1]$, $R[0..b-1]$

Output: zusammengefügt sortiertes Array $S[0..a+b-1]$

```

1 Function MERGE( $L, R$ )
2    $leftPos = 0$ 
3    $rightPos = 0$ 
4    $targetPos = 0$ 
5   while  $leftPos < a$  and  $rightPos < b$  do
6     if  $L[leftPos] \leq R[rightPos]$  then
7        $S[targetPos] = L[leftPos]$ 
8        $leftPos = leftPos + 1$ 
9     else
10       $S[targetPos] = R[rightPos]$ 
11       $rightPos = rightPos + 1$ 
12     $targetPos = targetPos + 1$ 
13  while  $leftPos < a$  do
14     $S[targetPos] = L[leftPos]$ 
15     $leftPos = leftPos + 1$ 
16     $targetPos = targetPos + 1$ 
17  while  $rightPos < b$  do
18     $S[targetPos] = R[rightPos]$ 
19     $rightPos = rightPos + 1$ 
20     $targetPos = targetPos + 1$ 
21  return  $S$ 

```

POS (Theorie)

Sortierv Verfahren

14 / 25

Quicksort

Algorithmus 6: Quicksort

Input: Array $A[l..r]$

```

1 Function QUICKSORT( $A, l, r$ )
2   if  $l \geq r$  then
3     return
4    $p = \text{PARTITION}(A, l, r)$ 
5   QUICKSORT( $A, l, p - 1$ )
6   QUICKSORT( $A, p + 1, r$ )

```

POS (Theorie)

Sortierv Verfahren

16 / 25

Algorithmus 7: Partition

Input: Array $A[l..r]$

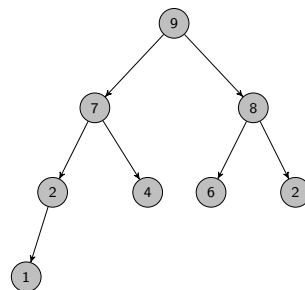
```

1 Function PARTITION( $A, l, r$ )
2    $pivot = A[r]$ 
3    $i = l$ 
4    $j = r - 1$ 
5   while  $i < j$  do
6     while  $A[i] < pivot$  do
7        $i = i + 1$ 
8     while  $j > l$  and  $A[j] \geq pivot$  do
9        $j = j - 1$ 
10    if  $i < j$  then
11       $SWAP(A, i, j)$ 
12     $i = i + 1$ 
13     $j = j - 1$ 
14  if  $i = j$  and  $A[j] < pivot$  then
15     $i = i + 1$ 
16  if  $A[i] \neq pivot$  then
17     $SWAP(A, i, r)$ 
18  return  $i$  // finale Pivot-Position

```

Heap

- (Max-)Heap: Vollständiger Binärbaum mit der Eigenschaft: Wert jedes Knotens größer oder gleich seiner Kinder.
- Max-Heap mit 8 Elementen: Die Werte in den Knoten stellen die gespeicherten Werte dar:



Da Baumstruktur implizit gegeben, können die Elemente (Ebene für Ebene betrachte) in einem Array $[9, 7, 8, 2, 4, 6, 2, 1]$ gespeichert werden. Da Baumstruktur implizit gegeben, können die Elemente (Ebene für Ebene betrachte) in einem Array $[(9), (7, 8), (2, 4, 6, 2), 1]$ gespeichert werden.

Quicksort (Beispiel)

Ablauf Zahlenfolge 3, 7, 1, 8, 2, 5, 9, 4, 6:

|> 3 7 1 8 2 5 9 4 6 <|

|> 3 7 1 8 2 5 9 4 (6) <|

|> 3 4 1 8 2 5 9 7 6 <|

|> 3 4 1 5 2 8 9 7 6 <|

|> 3 4 1 5 2 [6] 9 7 8 <|

|> 3 4 1 5 (2) <| 6 9 7 8

|> 1 4 3 5 2 <| 6 9 7 8

|> 1 [2] 3 5 4 <| 6 9 7 8

1 2 |> 3 5 (4) <| 6 9 7 8

1 2 |> 3 [4] 5 <| 6 9 7 8

1 2 3 4 5 6 |> 9 7 (8) <|

1 2 3 4 5 6 |> 7 9 8 <|

1 2 3 4 5 6 |> 7 [8] 9 <|

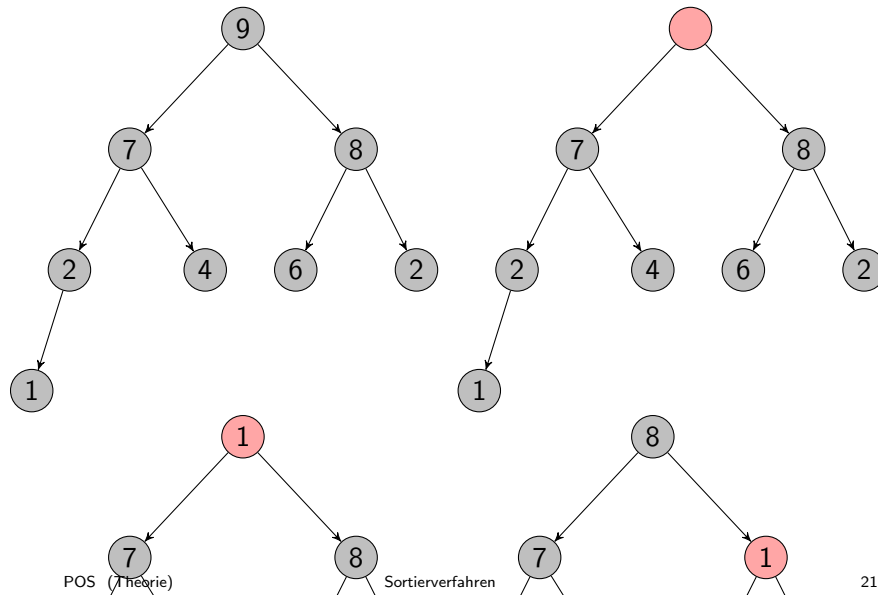
Anmerkung: |> und <| notieren die Grenzen (l und r). () notiert die Wahl des

Pivot-Elements, [] die Platzierung an der endgültigen Stelle.

Heapsort

- Nutzen für Sortierung: teilweise Ordnung
- Start mit gültigem Heap (kann in $O(n)$ aufgebaut werden)
- Entnahme Maximum: $O(1)$
- Platzierung des Maximums am Ende, und Verkleinerung des Heaps um ein Element
- Getaushtes Element von letzter Stelle ist nun in Wurzel
- → Versickern/Heapify

Versickern/Heapify



21 / 25

Ausführung von Heapsort

Detaillierte Zwischenschritte von Heapsort mit der Eingabefolge [9, 7, 8, 2, 4, 6, 2, 1]

Schritt	Arrayzustand	Kommentar
1	1, 7, 8, 2, 4, 6, 2 9	Vertausche Wurzel 9 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
2	2, 7, 6, 2, 4, 1 8, 9	Vertausche Wurzel 8 mit letztem Heap-Element 2 Heapify bei Index 0: versickern von 2
3	1, 4, 6, 2, 2 7, 8, 9	Vertausche Wurzel 7 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
4	2, 2, 1, 4 6, 7, 8, 9	Vertausche Wurzel 6 mit letztem Heap-Element 2 Heapify bei Index 0: versickern von 2
5	2, 2, 1 4, 6, 7, 8, 9	Vertausche Wurzel 4 mit letztem Heap-Element 2 Heapify bei Index 0: keine Veränderung nötig
6	1, 2 2, 4, 6, 7, 8, 9	Vertausche Wurzel 2 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
7	1 2, 2, 4, 6, 7, 8, 9	Vertausche Wurzel 2 mit letztem Heap-Element 1 Heapify nicht nötig
-	1, 2, 2, 4, 6, 7, 8, 9	Endergebnis: Array aufsteigend sortiert

Aufbau des initialen Heaps

- Bottom-Up-Aufbau eines Max-Heaps aus [1, 9, 8, 7, 4, 6, 2, 2].
- Versickern aller Nicht-Blätter

Arrayzustand	Kommentar
1, 9, 8, 7, 4, 6, 2, 2	Ausgangsfolge (unsortierte Anordnung)
	Heapify bei Index 3 (7 mit Kind 2): alles ok.
	Index 2 (8 mit Kindern 6 und 2): alles ok.
	Index 1 (9 mit Kindern 7 und 4): $9 \geq \max(7, 4)$, bleibt.
9, 1, 8, 7, 4, 6, 2, 2	Index 0 (1 mit Kindern 9 und 8): $\rightarrow$ tausche mit 9
9, 7, 8, 1, 4, 6, 2, 2	Jetzt 1 an Index 1 (Kinder 7, 4) $\rightarrow$ tausche mit 7
9, 7, 8, 2, 4, 6, 2, 1	Jetzt 1 an Index 3 (Kind 2) $\rightarrow 1 < 2 \rightarrow$ tausche
9, 7, 8, 2, 4, 6, 2, 1	Fertiger Max-Heap

POS (Theorie)

Sortierv Verfahren

22 / 25

Definition

Worst-Case Analyse einfacher Sortialgorithmen:

Algorithmus	Vergleiche	Swaps
Bubblesort	$O(n^2)$	$O(n^2)$
Insertionsort	$O(n^2)$	$O(n^2)$
Selectionsort	$O(n^2)$	$O(n)$

Analyse weiterer Algorithmen:

Algorithmus	Worst-Case	Average-Case
Heapsort	$O(n \log n)$	$O(n \log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$
Quicksort	$O(n^2)$	$O(n \log n)$

Quicksort:

- Worst-Case bei Quicksort wird nur selten erreicht
- In der Praxis schneller als Mergesort und Heapsort

Definition

In Java verwendet die Standardbibliothek verschiedene Sortierverfahren: Für primitive Arrays (z.B. `int[]` oder `double[]`) setzt die JDK-Implementierung seit Java 7 einen Dual-Pivot-Quicksort ein (leistungsfähig, jedoch instabil). Für Objekt-Arrays und für `List`-Implementierungen kommt eine stabile, adaptive Sortierung (Timsort) zum Einsatz.

Timsort ist ein stabiler, adaptiver Hybridalgorithmus, der bereits sortierte Teilfolgen (Runs) erkennt und durch Kombination von Insertionsort und einem effizienten Merge-Verfahren praktisch meist in $O(n \log n)$ Zeit sortiert.

Zusätzlich bietet `Arrays.parallelSort()` parallele Varianten (Fork/Join-basiert), die bei großen Datenmengen die Performance verbessern können.

Grundbegriffe

Datenstrukturen

Programmieren und Software-Engineering
Theorie

26. Februar 2026

- Einfachster Datentyp: Variable
- Komplexere Datentypen: Klassen und Objekte
- Organisation mehrerer Elemente: Datenstrukturen

POS (Theorie)

Datenstrukturen

1 / 46

Grundbegriffe

- Lineare Datenstrukturen
 - Array
 - Lists
 - Stack
 - Queue
- Dictionaries / Maps
- Sets (Mengen)
- Graphenbasierte Datenstrukturen:
 - Bäume
 - allgemeine Graphen

POS (Theorie)

Datenstrukturen

3 / 46

POS (Theorie)

Datenstrukturen

2 / 46

Array

- Zusammenhängende Anordnung mehrerer Elemente im Speicher
- Zugriff auf bestimmte Position mit Index $i = 0, 1, 2, \dots$
- Bei vielen Sprachen existiert Array als primitiver Datentyp
- Operationen wie *Einfügen* oder *Löschen* werden dann in höheren Datentypen umgesetzt.
- Beispiel Java: Array [] versus ArrayList

POS (Theorie)

Datenstrukturen

4 / 46

Array

Array mit 10 Elementen (Index unter den Zellen).

7	3	12	5	9	1	4	0	0	0
0	1	2	3	4	5	6	7	8	9

Array

Array nach dem Einfügen eines Elements 7 an der Stelle mit Index 4. Die Elemente 9, 14, und 4 wurden um eine Position nach rechts verschoben.

7	3	12	5	7	9	14	4	0	0
0	1	2	3	4	5	6	7	8	9

Array

Nun soll das Element an der Stelle mit Index 2 gelöscht werden.

7	3	12	5	7	9	14	4	0	0
0	1	2	3	4	5	6	7	8	9

Array

Die Elemente 12, 5, 7, 9, 14 und 4 wurden um eine Position nach links verschoben.

7	3	5	7	9	14	4	0	0	0
0	1	2	3	4	5	6	7	8	9

Array (Analyse)

Laufzeit der wichtigsten Operationen:

- Zugriff mit Index: $O(1)$
- Einfügen: $O(n)$
- Entfernen: $O(n)$

List

- Elemente liegen verteilt im Speicher
- Werden miteinander zur Zeiger/Pointer verkettet
- Einfach- vs. doppelt verketteter Liste



Array (Java)

Interne Umsetzung von ArrayList in Java

- Arrays existieren in Java als elementare Datenstruktur.
- `ArrayList<E>` bietet einen komfortableren Umgang mit Arrays an.
- `ArrayList<E>` nutzt intern ein `Object[]`.
- Beim Erweitern wird bei Bedarf ein größeres Array alloziert und die Elemente kopiert.
- Indexzugriff $O(1)$, Einfügen in der Mitte $O(n)$ wegen Verschieben der nachfolgenden Elemente.

Operationen und ihre Komplexität

- `add(e)` → Am Ende einfügen, amortisiert $O(1)$.
- `add(i, e)` → An Position `i` einfügen, $O(n)$.
- `get(i)` → Zugriff per Index, $O(1)$.
- `remove(i)` → $O(n)$ im Worst Case.

List (Analyse)

Laufzeit der wichtigsten Operationen:

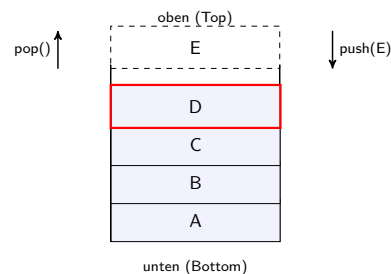
- Zugriff Index: $O(n)$
- Einfügen an beliebiger Stelle: $O(n)$
- Löschen: $O(n)$
- Zugriff/Einfügen/Löschen mit Iterator: $O(1)$

List (Java)

- Java stellt mit `LinkedList<E>` eine doppelt verkettete Liste bereit.
- `LinkedList`: Einfügen/Entfernen am Anfang/Ende und über Iterator $O(1)$, zufälliger Indexzugriff $O(n)$.
- `ArrayDeque<E>` ist eine effiziente Implementierung für Stack/Queue-Operationen an beiden Enden ($O(1)$).
- Für Random Access ist `ArrayList` die bessere Wahl ($O(1)$ per Index).

Stack

Schematischer Stack (LIFO): Nur das oberste Element (rot markiert) kann entfernt (pop) werden, bzw. kann auch nur oben hinzugefügt werden (push).



Stack

Ein **Stack** folgt dem Prinzip **Last In — First Out (LIFO)**: Das zuletzt eingefügte Element wird als erstes wieder entfernt. Typische Operationen sind:

- push: Einfügen eines Elements oben auf den Stack.
- pop: Entfernen des obersten Elements.
- peek/top: Zugriff auf das oberste Element, ohne es zu löschen.



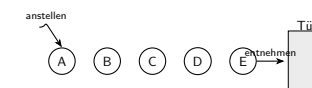
Stacks lassen sich einfach mit Arrays oder verketteten Listen realisieren und werden z.B. bei Funktionsaufrufen (Call Stack) oder in Undo-Mechanismen eingesetzt.

Queue

Eine **Queue** (Warteschlange) folgt dem Prinzip **First In – First Out (FIFO)**: Das zuerst eingefügte Element wird auch als erstes wieder entfernt. Typische Operationen sind:

- enqueue: Einfügen eines Elements am Ende der Queue.
- dequeue: Entfernen des ersten Elements.
- front: Zugriff auf das erste Element, ohne es zu löschen.

Queues werden oft in Betriebssystemen (Prozessplanung), Netzwerken (Datenpaketverwaltung) oder Druckersystemen verwendet.



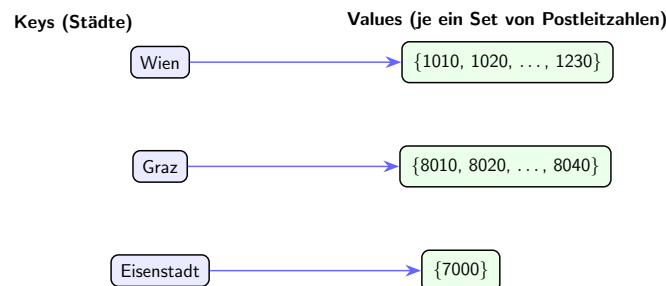
Schematische Warteschlange (Queue): vorne (rechts) wird entnommen, hinten (links) wird angestellt.

List/Queue (Java)

- In Java lassen sich Stack/Queue effizient mit der Deque-Schnittstelle umsetzen (z.B. ArrayDeque).
 - `push()/pop()` bzw. `addFirst()/removeFirst()` für Stack-Verhalten.
 - `add()/poll()` oder `addLast()/removeFirst()` für Queue-Verhalten.
- ArrayDeque ist in der Regel schneller als LinkedList für diese Einsatzzwecke und liefert $O(1)$ -Operationen an beiden Enden.

Dictionaries/Maps

Beispiel: Es werden Städten (=Schlüssel) die Mengen von Postleitzahlen (=Werte) zugeordnet. Der Wert ist hier also eine Menge (Datenstruktur Set).



Dictionaries/Maps

- Dictionaries oder Maps **Schlüssel – Wert**-Paare.
- Oft möchte man gespeicherte Objekte *schnell* aufgrund eines *Schlüssels* (id, name, ...) finden, bzw. darauf zugreifen.
- Beispiel:
 - Finde Mitarbeiter mit Personalnummer
 - Finde Immobilie mittels Anzeigennummer
- Das Suchen in Arrays, ArrayLists etc. ist aufwändig, da die Datenstruktur zunächst durchsucht werden muss.
- **Ziel:** Auffinden eines Elementes aufgrund des **Schlüssels (Key)** in (quasi) konstanter Zeit $O(1)$, d.h. ohne viele Elemente in der Datenstruktur betrachten zu müssen.

Hashfunktionen und Hashtabellen

- Mittels *Hashfunktionen* wird zu einem Schlüssel ein Hashwert berechnet, der als Index in einer *Hashtabelle* dient.
- Schlüssel (z.B. Strings) sollen schnell auf Speicherplätze (Buckets) einer Tabelle abgebildet werden.
- Dazu wird eine *Hashfunktion* h verwendet, die einen Schlüssel k auf einen ganzzahligen Index $h(k) \in \{0, \dots, m-1\}$ (Tabellengröße m) projiziert.
- Hashtabellen verbinden eine (große) Array-Struktur fester Länge m mit einer Hashfunktion. Jeder Eintrag (Bucket) kann (abhängig vom Verfahren) direkt ein Element oder eine kleine Sekundärstruktur (Liste/Baum) enthalten. Ziel: Erwartete $O(1)$ Zeit für Suchen, Einfügen und Löschen.

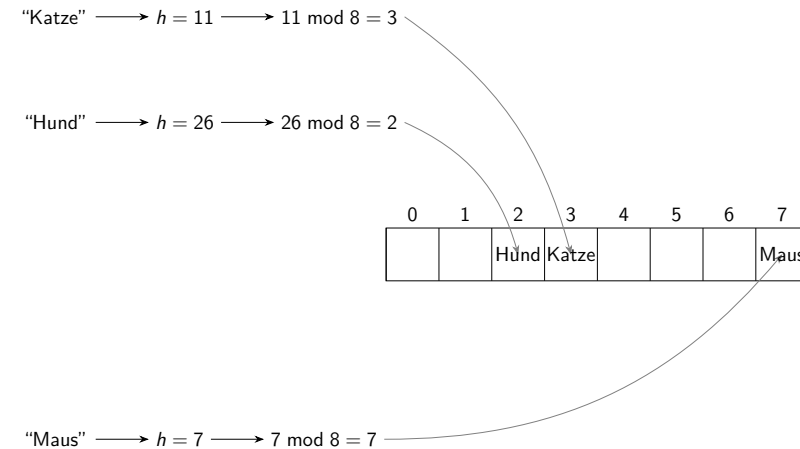
Anforderungen an eine Hashfunktion

- Deterministisch: Gleicher Schlüssel $\Rightarrow$ gleicher Hashwert.
- Effizient: Berechnung in (amortisiert) $O(1)$ und sehr schnell (wenige CPU-Operationen).
- Gleichmäßige Verteilung: Schlüssel sollen möglichst gleich über $\{0, \dots, m-1\}$ verteilt werden (minimiert Kollisionen).
- Ähnliche Schlüssel $\Rightarrow$ ähnliche Hashwerte (Vermeidung von Clustern).
- Stabil bzgl. Lebensdauer der Tabelle (aber: in manchen Sprachen kann sich Hash zwischen Programmläufen absichtlich ändern, z.B. Python wegen Sicherheit).

Hashtabellen: Grundbegriffe

- *Kollision*: Zwei verschiedene Schlüssel $k_1 \neq k_2$ mit $h(k_1) = h(k_2)$.
- *Lastfaktor* (load factor) $\alpha = \frac{n}{m}$ mit n = Anzahl gespeicherter Elemente.
- *Rehashing*: Vergrößern der Tabelle (typisch Faktor 2) und Neuverteilen aller Elemente, sobald α einen Schwellwert überschreitet.

Grundidee einer Hashtabelle



Hashfunktionen

Hashfunktion

$$h(k) = h_{\text{raw}}(k) \bmod m$$

Dabei produziert h_{raw} einen großen Integer (z.B. durch Kombination von Zeichen / Feldern), das Modulo faltet den Wert in den Tabellenbereich.

Beispiel: Einfache Beispiele:

- **Strings:** Polynomieller Rolling-Hash

$$h_{\text{raw}}(s_0 s_1 \dots s_{L-1}) = \sum_{i=0}^{L-1} s_i \cdot B^{L-1-i}$$
 mit Basis B (z.B. 131, 257) und ggf. Reduktion mit großem Prim P vor Modulo m .
- **Integer:** Mischung (Bit-Mixing), z.B. Multiplikation mit großer ungerader Konstante und Bit-Shifts.

Kollisionsbehandlung (Überblick)

- *Getrennte Verkettung* (Chaining): Jeder Bucket hält eine (kurze) Liste / Baum der kollidierenden Elemente.
- *Offene Adressierung* (Sondieren): Bei Kollisionen wird versucht das Element an einer anderen Stelle im Array abzulegen (Sondieren).

Methode 1: Verkettung der Überläufer

- Array $T[0..m-1]$; jeder Bucket hält z.B. eine (kurze) verkettete Liste der dort liegenden Schlüssel/Werte.
- Insert: Finde $i = h(k)$, hänge (k,v) an Liste $T[i]$ an (falls Schlüssel schon da: Wert ersetzen).
- Lookup: Suche in Liste $T[h(k)]$ nach Schlüssel.
- Delete: Entferne Element aus dieser Liste.
- Erwartete Länge einer Liste: $\alpha = n/m$ (Lastfaktor) bei guter Hashfunktion $\Rightarrow$ erwartete Kosten $O(1 + \alpha)$.

Hashfunktionen

Qualität der Hashfunktion:

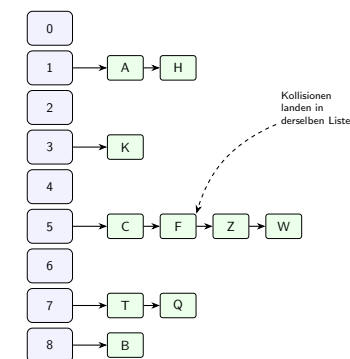
- Schlechte Verteilung führt zu langen Listen (Chaining) bzw. langen Suchfolgen (Sondieren) $\Rightarrow$ Performance-Verlust.
- Eine (annähernd) uniforme Verteilung hält erwartete Kosten pro Operation bei $O(1)$ solange α kontrolliert bleibt.

Immutabilität von Schlüsseln:

Ändert sich ein Schlüssel nach dem Einfügen (z.B. veränderbares Objekt mit hash-basiertem Wert), kann er nicht mehr korrekt gefunden werden. Daher müssen Schlüssel (logisch) unveränderlich sein oder ihr Hash / Vergleich darf sich nicht ändern.

Methode 1: Verkettung der Überläufer

Hashtabelle mit Verkettung (Chaining): Jeder **Bucket** zeigt (falls nicht leer) auf den Kopf einer Liste kollidierender **Elemente**. Die erwartete Listenlänge entspricht dem Lastfaktor $\alpha = n/m$.



Methode 2: Offene Hashverfahren (Sondieren)

Die offenen Hashverfahren verfolgen eine andere Strategie im Falle von Kollisionen. Statt die kollidierenden Elemente in einer Liste zu speichern, werden sie in der Tabelle selbst an der nächsten geeigneten freien Stelle abgelegt. Die nächste geeignete Stelle zu berechnen wird als Sondieren bezeichnet.

Bei belegtem Platz wird eine neuer Platz wie folgt berechnet

$$h_i(k) = (h(k) + f(i)) \bmod m, \quad i = 0, 1, 2, \dots$$

Verwendet man als Funktion $f(i) : i$, so nennt man dies *lineares Sondieren*. Verwendet man hingegen $f(i) = c_1 i + c_2 i^2$, wird dies als *quadratisches Sondieren* bezeichnet.

Methode 3: Double Hashing

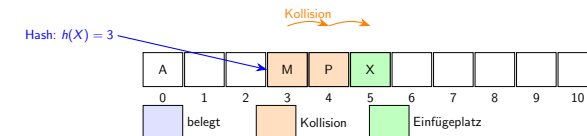
Eine weitere Möglichkeit ist Double Hashing: $h_i(k) = h_1(k) + i \cdot h_2(k)$. Hierbei werden zwei verschiedene Hashfunktionen h_1 und h_2 verwendet, um die Sondierfolge zu bestimmen. Dies kann helfen, Kollisionen weiter zu reduzieren und eine bessere Verteilung der Elemente in der Hashtabelle zu erreichen. Die Wahl von h_2 ist hierbei jedoch kritisch, ebenso ist das Verfahren aufwändiger als lineares oder quadratisches Sondieren.

Vor- und Nachteile der Verfahren:

- Chaining: Einfach, Performance robust, benötigt Zeiger/Overhead.
- Offene Adressierung: Cache-freundlich (alles im Array), aber sensibler gegenüber hoher Auslastung; Clusterbildung möglich.

Illustration: lineares Sondieren

Einfügen von Schlüssel X mit Hash 3: Position 3 ist belegt, lineares Sondieren prüft 4 (auch belegt), findet 5 frei und legt dort ab. Die Anzahl besuchter Buckets hängt vom Cluster vor der Zielposition ab; geringe Auslastung minimiert durchschnittliche Sondenlänge.



Beispiel: lineares Sondieren

$$m = 8, \quad h'(k) = k \bmod 8$$

Einzufügende Schlüssel (in dieser Reihenfolge): 10, 19, 31, 22, 14, 16

k	10	19	31	22	14	16
$h'(k)$	2	3	7	6	6	0

Schrittweises Einfügen (lineares Sondieren)

- 10: $h'(10) = 2 \Rightarrow$ Slot 2 frei $\Rightarrow$ einfügen.
- 19: $h'(19) = 3$ frei $\Rightarrow$ Slot 3.
- 31: $h'(31) = 7$ frei $\Rightarrow$ Slot 7.
- 22: $h'(22) = 6$ frei $\Rightarrow$ Slot 6.
- 14: $h'(14) = 6$ Kollision bei 6 (22) $\Rightarrow$ prüfe 7 (31, belegt) $\Rightarrow$ prüfe 0 (frei) $\Rightarrow$ Slot 0.
- 16: $h'(16) = 0$ Kollision bei 0 (14) $\Rightarrow$ prüfe 1 (frei) $\Rightarrow$ Slot 1.

Endzustand der Hashtabelle

Index	0	1	2	3	4	5	6	7
Wert	14	16	10	19			22	31

Beispiel: lineares Sondieren

Beobachtung: Die Clusterbildung zeigt sich daran, dass die Kollision von 14 (Start bei 6) die Sondenfolge bis zum Anfang (Wrap-Around) verlängert; 16 trifft danach sofort erneut auf eine belegte Zelle. Längere zusammenhängende belegte Abschnitte wachsen schneller (Primär-Cluster).

Beispiel: quadratisches Sondieren

Beobachtung: Kollisionen (z.B. Startindex 6 für 22 und 14) führen zu Sprüngen, die kein zusammenhängendes primäres Cluster erzeugen (Slots: $6 \rightarrow 0$). Dennoch teilen alle Schlüssel mit gleichem Start-Hash dieselbe Sondenfolge (sekundäre Cluster). Mit $m = 8$ (Potenz von 2) muss geeignete Wahl von c_1, c_2 getroffen werden, um genügend unterschiedliche Slots zu erreichen.

Beispiel: quadratisches Sondieren

$$m = 8, \quad h'(k) = k \bmod 8, \quad h_i(k) = (h'(k) + i + i^2) \bmod 8 \quad (i = 0, 1, 2, \dots; c_1 = c_2 = 1)$$

Einzufügende Schlüssel (in dieser Reihenfolge): 10, 19, 31, 22, 14, 16

k	10	19	31	22	14	16
$h'(k)$	2	3	7	6	6	0

Schrittweises Einfügen (quadratisches Sondieren)

- 10: $h'(10) = 2$ frei $\Rightarrow$ Slot 2.
- 19: $h'(19) = 3$ frei $\Rightarrow$ Slot 3.
- 31: $h'(31) = 7$ frei $\Rightarrow$ Slot 7.
- 22: $h'(22) = 6$ frei $\Rightarrow$ Slot 6.
- 14: $h'(14) = 6$ belegt (22). Sondenfolge: $i = 1 : 6 + 1 + 1 = 8 \equiv 0$ frei $\Rightarrow$ Slot 0.
- 16: $h'(16) = 0$ belegt (14). Sondenfolge: $i = 1 : 0 + 1 + 1 = 2$ belegt (10);
 $i = 2 : 0 + 2 + 4 = 6$ belegt (22); $i = 3 : 0 + 3 + 9 = 12 \equiv 4$ frei $\Rightarrow$ Slot 4.

Endzustand der Hashtabelle

Index	0	1	2	3	4	5	6	7
Wert	14		10	19	16		22	31

Java: Voraussetzungen

- Objekte sind als Schlüssel in HashMap nutzbar, wenn sie `hashCode()` und `equals()` konsistent implementieren.
- Eingebaute unveränderliche Typen (String, Integer, Long, ...) bieten diese Funktionalität bereits an.
- Generell für unveränderliche (*immutable*) Objekte sinnvoll, da sich Hashwert nicht ändert.

Zusammenfassung und Analyse

Laufzeitanalyse von Dictionaries:

- Erfolgreiche Suche: amortisiert $O(1)$
- Einfügen: amortisiert $O(1)$
- Löschen: amortisiert $O(1)$
- Worst Case (pathologisch viele Kollisionen): $O(n)$

Konsistenzanforderungen:

- Hash und Gleichheit müssen konsistent sein: $k_1 = k_2 \Rightarrow h(k_1) = h(k_2)$.
- Schlüssel dürfen sich nach Einfügen nicht (logisch) ändern.

Zusammenfassung:

Eine gute Hashfunktion ist schnell, verteilt Schlüssel gleichmäßig und minimiert so Kollisionen. Kollisionsbehandlung (Chaining oder offene Adressierung) plus kontrollierter Lastfaktor sichern amortisierte $O(1)$ -Operationen für Insert, Lookup und Delete.

POS (Theorie)

Datenstrukturen

37 / 46

Java-Maps

```
// Sicherer Zugriff
System.out.println(person.get("email")); // -> null
System.out.println(person.getDefault("email", "kein Eintrag"));

// Existenztest
if (person.containsKey("age")) { ... }

// Entfernen
person.remove("city"); // entfernt, kein Fehler
// bei fehlendem Key
Integer age = (Integer) person.remove("age"); // entfernt + liefert Wert

// Iteration
for (String k : person.keySet()) { ... }

for (Map.Entry<String, Object> e : person.entrySet()) {
    String k = e.getKey(); Object v = e.getValue();
}

for (Object v : person.values()) { ... }
```

POS (Theorie)

Datenstrukturen

39 / 46

Java-Maps

Java verwendet HashMap (bzw. LinkedHashMap, TreeMap):

```
// Erzeugen
Map<String, Object> d = new HashMap<>(); // leeres Map
Map<String, Object> person = new HashMap<>();
person.put("name", "Ada");
person.put("age", 37);

// Zugriff / Einfügen / Überschreiben
person.put("city", "London"); // Einfügen
person.put("age", 38); // Überschreiben
System.out.println(person.get("name")); // Direktzugriff
```

POS (Theorie)

Datenstrukturen

38 / 46

Sets

- Ein *Set* entspricht einer mathematischen Menge.
- Gleiche Elemente können in einem Set nur einmal enthalten sein.
- In einem Set kann man (im Gegensatz zur ArrayList) nicht mittels Index auf bestimmte Elemente zugreifen.
- Operationen auf Sets:
 - Hinzufügen eines Elementes
 - Entfernen eines Elementes
 - Überprüfen ob ein Element im Set enthalten ist

Anmerkung: Die konkrete Umsetzung der oben definierten Anforderungen an die Datenstruktur Set kann auf verschiedene Arten erfolgen. Eine sehr gebräuchliche und effiziente Methode ist die Verwendung von Hashtabellen aber auch baum-basierte Sets sind möglich.

POS (Theorie)

Datenstrukturen

40 / 46

Sets mit Hashtabelle

- Jedes Element wird durch seinen Hashwert in eine bestimmte Position (Bucket) der Hashtabelle eingefügt.
- Dies ermöglicht einen schnellen Zugriff auf die Elemente, da die Suche, das Hinzufügen und das Entfernen im Durchschnitt in konstanter Zeit $O(1)$ erfolgen können.
- Vergleiche mit naiver Implementierung als Liste: jeder Zugriff $O(n)$ Zeit

Eigenschaften:

Die Elemente sind im Set einzigartig. HashSet garantiert keine Reihenfolge; extttLinkedListHashSet erhält Einfüge-Reihenfolge; TreeSet liefert sortierte Reihenfolge. Zugriff/Einfügen/Entfernen im Durchschnitt $O(1)$ für HashSet, TreeSet liefert $O(\log n)$.

Datentyp Set (Standardimplementierung: HashSet)

Wichtige Operationen:

- `Set<Integer> s = new HashSet<>();` → Erzeugung
- `s.add(x);` → fügt ein Element hinzu (tut nichts, wenn schon vorhanden).
- `s.remove(x);` → entfernt ein Element (gibt boolean zurück).
- `s.clear();` → leert das Set.
- Iteration: `for (T e : s) ...`

Set mittels Bäumen

- Umsetzung des Set-Konzeptes mittels *Bäumen*.
- Elemente können in $O(\log n)$ gesucht, eingefügt und gelöscht werden.
- Zusatznutzen: die Elemente sind immer sortiert!
- Zugriff auf nächst- größeres/kleineres Element schnell möglich

In Java steht mit `java.util.TreeSet` eine standardisierte Baum-basierte Set-Implementierung zur Verfügung, die Elemente sortiert hält ($O(\log n)$ Operationen).

Laufzeitanalyse

Laufzeitanalyse verschiedener Operationen bei Größe n der Datenstruktur:

Operation	Array(List)	Verkettete Liste	Hash-Set	Tree-Set
add (end)	$O(1)$	$O(1)$	$O(1)$	$O(\lg n)$
remove	$O(n)$	$O(n)$	$O(1)$	$O(\lg n)$
remove (iterator)	$O(n)$	$O(1)$	$O(1)$	$O(\lg n)$
get (index)	$O(1)$	$O(n)$	$O(1)$	$O(\lg n)$
access (position)	$O(1)$	$O(n)$		
find (element)	$O(n)$	$O(n)$	$O(1)$	$O(\lg n)$
insert (position)	$O(n)$	$O(n)$		
insert (iterator)	$O(n)$	$O(1)$		

Laufzeitanalyse

- In der Tabelle scheint das Hash-Set dem Tree-Set eindeutig überlegen zu sein.
- Das Tree-Set bietet jedoch den Vorteil, dass die enthaltenen Elemente effizient sortiert ausgegeben werden können, und es benötigt wesentlich weniger zusätzlichen Speicher als das Hash-Set.
- Außerdem Bei Hashtabelle Reorganisation notwendig, wenn Auslastung zu groß wird. Aufwand $O(n)$, amortisiert $O(1)$.

Homomorphismen

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

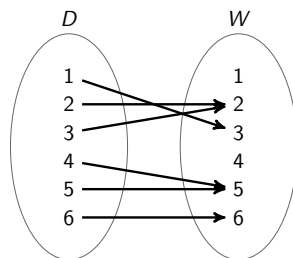
POS (Theorie)

Homomorphismen

1 / 36

Bild und Urbild

Beispiel: Definitionsmenge D , Wertemenge W :



- 3 ist das Bild von 1, hingegen ist 1 das Urbild zu 3
- 5 ist das Bild von {4, 5}, das Urbild von 5 ist {4, 5}
- {4, 5, 6} ist das Urbild von {5, 6}

POS (Theorie)

Homomorphismen

3 / 36

Funktionen und Abbildungen

Definition (Funktion, bzw. Abbildung)

Eine eindeutige Zuordnung von Elementen aus einer Menge A in eine Menge B

$$f : A \rightarrow B$$

heißt *Funktion* oder *Abbildung*.

Dabei bezeichnet A die *Definitionsmenge* oder *Urbildmenge*. Die Menge B wird als *Bildmenge* oder *Wertemenge* bezeichnet.

Konkrete Zuordnungen schreibt man als:

$$b = f(a)$$

POS (Theorie)

Homomorphismen

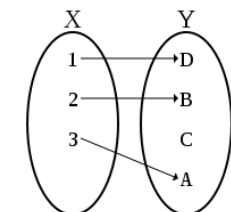
2 / 36

Injektivität, Surjektivität, Bijektivität

Injektivität: Eine Funktion heißt *injektiv*, wenn

- verschiedene Argumente auf verschiedene Funktionswerte abgebildet werden
- Jeder Funktionswert besitzt höchstens ein Urbild
- Ist $f(a) = f(b)$, dann ist $a = b$

Man nennt injektive Funktionen auch *links-eindeutig*.



Bildquelle: [?]

POS (Theorie)

Homomorphismen

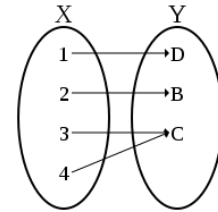
4 / 36

Injektivität, Surjektivität, Bijektivität

Surjektivität: Eine Funktion heißt *surjektiv*, wenn

- Jeder Funktionswert wird mindestens einmal durch die Abbildung getroffen.
- Jeder Funktionswert besitzt mindestens ein Urbild.

Man nennt surjektiv Funktionen auch *rechtsvollständig*.

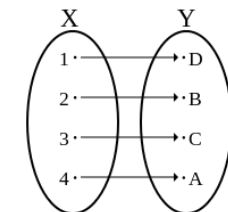


Bildquelle: [?]

Injektivität, Surjektivität, Bijektivität

Bijektivität: Eine Funktion heißt *bijektiv*, wenn

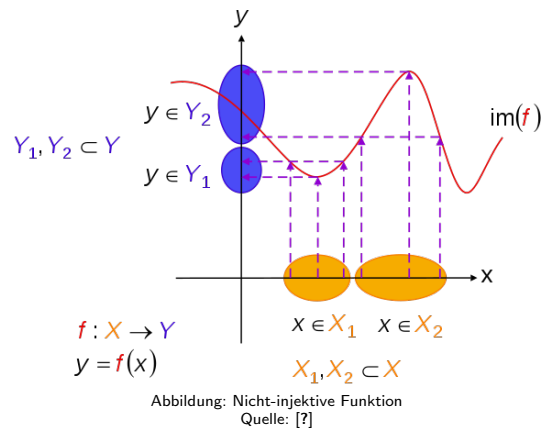
- Die Abbildung ist surjektiv und injektiv.
- Jeder Funktionswert besitzt genau ein Urbild.
- Es gibt für die Funktion eine Umkehrfunktion.



Bildquelle: [?]

Bijektive Funktionen sind *umkehrbar*.

Injektivität, Surjektivität, Bijektivität



Beispiele

Beispiel: Sei $f: \mathbb{N} \rightarrow \mathbb{N}$ (mit den natürlichen Zahlen $\mathbb{N}$) eine Funktion mit $f(n) = 2 \cdot n$.

$f(n)$ ist injektiv, aber nicht surjektiv.

Aufgaben 1.1.1 – 1.1.3

- 1 Wir betrachten die Menge $\mathbb{R}$ der reellen Zahlen und die Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$, definiert durch $f(x) = 5 \cdot x$. Welche Eigenschaften (injektiv, surjektiv, bijektiv) hat diese Funktion?
- 2 Welchen Bedingungen gehorcht die Funktion, die einem Kind seine Mutter zuordnet? Was sind dabei die Urbild- und die Bildmenge?
- 3 Welchen Bedingungen gehorcht die Funktion, die einem Land seine Hauptstadt zuordnet? Was sind dabei die Urbild- und die Bildmenge?

Aufgaben 1.1.4 – 1.1.8

- Sind die folgenden Funktionen $f : \mathbb{R} \rightarrow \mathbb{R}$ injektiv, surjektiv oder bijektiv? Begründung!
 - 1 $f(x) = x^3$
 - 2 $f(x) = |x|$
 - 3 $f(x) = \sin(x)$
 - 4 $f(x) = \pi \cdot x + 35$
 - 5 $f(x) = \tan(x)$

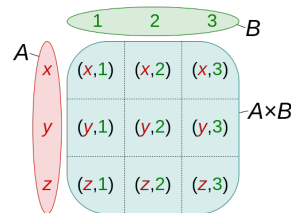
Kartesisches Produkt

Definition (Kartesisches Produkt)

Das *kartesische Produkt* (oder *Kreuzprodukt*) zweier Mengen $A \times B$ ist gegeben durch

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}.$$

Anmerkung: Das Kreuzprodukt ist also die Menge aller *geordneten Paare*, wobei das erste Element aus der ersten Menge stammt, und das zweite aus der zweiten.



Innere Verknüpfungen

Definition (Innere Verknüpfung, binäre Operation)

Eine zweistellige Funktion

$$f : A \times A \rightarrow A$$

heißt *innere Verknüpfung* oder *binäre Operation*, falls

$$\forall a, b \in A : f(a, b) \in A.$$

Diese Bedingung wird auch als **Abgeschlossenheit** der inneren Verknüpfung bezeichnet. Eine Menge mit einer inneren Verknüpfung heißt **Gruppoid** und wird vereinfacht durch $\langle A, f \rangle$ dargestellt.

Gruppoid, Halbgruppe, Monoid, Gruppe

Definition

- G1 Abgeschlossenheit: $a, b \in G \Rightarrow a * b \in G$ für alle $a, b \in G$
- G2 Assoziativgesetz: $(a * b) * c = a * (b * c)$, für alle $a, b, c \in G$
- G3 Einheitsselement: Es existiert ein Einheitsselement $e \in G$, sodaß für alle $a \in G$ gilt: $a * e = e * a = a$.
- G4 Inverses Element: Für jedes $a \in G$ existiert ein $a^{-1} \in G$ mit $a * a^{-1} = a^{-1} * a = e$.
- G5 Kommutativgesetz: $a * b = b * a$ für alle $a, b \in G$

Beispiele

- Gruppoid: $\langle \mathbb{R}^+, * \rangle$ mit $a * b = a^b$
- Halbgruppe: $\langle \mathbb{R}^+, + \rangle$
- Monoid: $\langle \mathbb{N}^+, + \rangle$, ($e=0$)
- Gruppe: Bewegungen der Ebene
- Abel'sche Gruppe: $\langle \mathbb{Z}, + \rangle$, $\langle \mathbb{R}, + \rangle$, $\langle \mathbb{R}_0 = \mathbb{R} \setminus \{0\}, \cdot \rangle$.

Gruppoid, Halbgruppe, Monoid, Gruppe

Definition (Gruppoid, Halbgruppe, Monoid, Gruppe)

Eine Struktur heißt:

- **Gruppoid**, wenn G1 erfüllt ist,
- **Halbgruppe**, wenn G1 und G2 erfüllt sind,
- **Monoid**, wenn G1, G2 und G3 erfüllt sind,
- **Gruppe**, wenn G1, G2, G3 und G4 erfüllt sind, und
- **Abel'sche Gruppe**, wenn G1 bis G5 erfüllt sind.

Beispiel

Wählen wir als Grundmenge die natürlichen Zahlen und als innere Verknüpfung die Addition oder als Grundmenge die ganzen Zahlen mit der Subtraktion als binäre Operation, die rationalen Zahlen mit der Multiplikation, die Menge der Aussagen mit Konjunktion, Disjunktion, Implikation oder Äquivalenz, die Menge der Wörter einer Sprache mit der Verkettung der Wörter oder die Menge aller Teilmengen einer Menge (Potenzmenge) mit Durchschnitt, Vereinigung oder Mengendifferenz als innere Verknüpfung, so erhalten wir jeweils Gruppoide.

Wählen wir als Grundmenge die reellen Zahlen und als zweistellige Funktion die Division, so erhalten wir kein Gruppoid. Warum?

Homomorphismen

Definition (Homomorphismus)

Seien $\langle A, * \rangle$ und $\langle B, \circ \rangle$ zwei Gruppoide. Dann heißt die Abbildung $\Phi : A \rightarrow B$ *Homomorphismus*, wenn für alle Elemente $a, b \in A$ gilt, daß

$$\Phi(a * b) = \Phi(a) \circ \Phi(b).$$

Man nennt Φ einen

- **Monomorphismus**, wenn Φ injektiv,
- **Epimorphismus**, wenn Φ surjektiv,
- **Isomorphismus**, wenn Φ bijektiv.

Wenn $A = B$, dann heißt ein Homomorphismus **Endomorphismus** und ein Isomorphismus **Automorphismus**.

Homomorphismen

Beispiel: Sei $A = \{0, 1, 2\}$ und $B = \{a, b, c\}$. Die inneren Verknüpfungen $*$ und $\circ$ für A und B seien gegeben durch:

$*$	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

$\circ$	a	b	c
a	b	c	a
b	c	a	b
c	a	b	c

Darstellung: Isomorphismus

Wir betrachten die Gruppoide $\langle A, * \rangle$ und $\langle B, \circ \rangle$, sowie die Abbildung $\Phi : A \rightarrow B$:

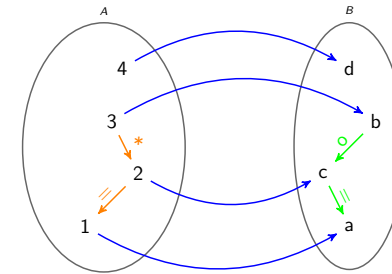


Abbildung $\Phi(1) = a, \Phi(2) = c, \Phi(3) = b, \Phi(4) = d$

Wenn in A gilt, daß $3 * 2 = 1$,

dann muss in B gelten: $b \circ c = a$.

Diese Eigenschaft muss für alle $x, y \in A$ gelten, damit Φ tatsächlich ein Isomorphismus ist!

Homomorphismen

Ein Homomorphismus (Isomorphismus) ist gegeben durch $\Phi(0) = c, \Phi(1) = b$ und $\Phi(2) = a$.

Beweis:

- $\Phi(0 * 0) = \Phi(0) = c = \Phi(0) \circ \Phi(0) = c \circ c$
- $\Phi(0 * 1) = \Phi(1) = b = \Phi(0) \circ \Phi(1) = c \circ b$
- $\Phi(0 * 2) = \Phi(2) = a = \Phi(0) \circ \Phi(2) = c \circ a$
- $\Phi(1 * 0) = \Phi(1) = b = \Phi(1) \circ \Phi(0) = b \circ c$
- $\Phi(1 * 1) = \Phi(2) = a = \Phi(1) \circ \Phi(1) = b \circ b$
- $\Phi(1 * 2) = \Phi(0) = c = \Phi(1) \circ \Phi(2) = b \circ a$
- $\Phi(2 * 0) = \Phi(2) = a = \Phi(2) \circ \Phi(0) = a \circ c$
- $\Phi(2 * 1) = \Phi(0) = c = \Phi(2) \circ \Phi(1) = a \circ b$
- $\Phi(2 * 2) = \Phi(1) = b = \Phi(2) \circ \Phi(2) = a \circ a$

Bestimmung aller Isomorphismen

o	a	b	c	d	e	*	1	2	3	4	5
a	d	b	a	c	b	1					
b	e	a	c	b	d	2				3	
c	e	a	d	b	a	3				2	
d	a	c	e	d	a	4			1		
e	c	c	b	d	e	5					

Wir halten zunächst fest: $2 * 4 = 3$, $3 * 4 = 2$, $4 * 3 = 1$

a	→	2	2	4		4	3		3		3	4		
b	→		3	2	2	1	4	3		4			4	3
c	→	3				2	2	2	3	4		3	3	4
d	→	4				3	5		4	2	2			
e	→		4	3	4	3		4		3	4	2	2	2
		?	?	?	?	✓	?	?	?	?	?	?	?	?

Die erste mögliche Abbildung wäre mit $a \rightarrow 2$ und $d \rightarrow 4$, und somit $c \rightarrow 3$. Die erste mögliche Abbildung wäre mit $a \rightarrow 2$ und $d \rightarrow 4$, und somit $c \rightarrow 3$.

Wir testen mit der zweiten Gleichung, $3 * 4 = 2$. Es müsste also gelten $c \circ d = a$. ? Die nächste mögliche Abbildung wäre mit $a \rightarrow 2$ und $e \rightarrow 4$, und somit $b \rightarrow 3$.

Die nächste mögliche Abbildung wäre mit $a \rightarrow 2$ und $e \rightarrow 4$, und somit $b \rightarrow 3$.

Wir testen $3 * 4 = 2$. Es müsste gelten $b \circ e = a$, tatsächlich gilt jedoch $b \circ e = d$. ? Auch hier erhalten wir einen Widerspruch ? Auch hier erhalten wir einen Widerspruch ? Wir testen nun $c \rightarrow 2$ und $a \rightarrow 4$. Somit gilt $2 * 4 = 3 \Rightarrow e \rightarrow 3$.

Wir prüfen mit der nächsten Gleichung $3 * 4 = 2$, und tatsächlich gilt $e \circ a = c$. Wir testen

Bestimmung aller Isomorphismen

o	1	2	3	4	5	*	a	b	c	d	e
1	4	2	3	5	5	a					
2	1	3	4	1	3	b				c	
3	2	4	5	3	1	c					
4	5	3	2	1	3	d		a		e	
5	4	4	2	1	3	e					

Wir erhalten: $b * d = c$, $d * b = a$, $d * d = e$

$1 \circ 2 = 2$ (derartige Fälle ignorieren wir künftig)

$1 \circ 4 = 5$, $4 \circ 1 = 5$? (da bezüglich * verschiedene Ergebnisse)

$2 \circ 3 = 4$, $3 \circ 2 = 4$? (da bezüglich * verschiedene Ergebnisse, derartige Fälle ignorieren wir künftig)

$2 \circ 4 = 1$, $4 \circ 2 = 3 \Rightarrow e \rightarrow 5 \Rightarrow d \rightarrow 3$? (da $d \in \{2, 4\}$)

$2 \circ 5 = 3$, $5 \circ 2 = 4 \Rightarrow e \rightarrow 1 \Rightarrow d \rightarrow 4$? (da $d \in \{2, 5\}$)

$3 \circ 5 = 1$, $5 \circ 3 = 2 \Rightarrow e \rightarrow 4 \Rightarrow d \rightarrow 1$? (da $d \in \{3, 5\}$)

$4 \circ 5 = 3$, $5 \circ 4 = 1 \Rightarrow e \rightarrow 2$? (da nicht in Diagonale)

Es existiert kein Isomorphismus!

POS (Theorie)

Homomorphismen

22 / 36

Bestimmung aller Isomorphismen

o	1	2	3	4	5	*	a	b	c	d	e
1						a	b	c	a	d	d
2			4			b	a	a	d	c	e
3						c	e	a	b	b	c
4	1					d	b	a	c	c	e
5				5		e	b	a	e	d	e

Lösung: (i) $4 \circ 1 = 1$, (ii) $2 \circ 3 = 4$ und (iii) $5 \circ 3 = 5$

*	a	b	c	d	e
a	b	c	a	d	d
b	a	a	d	c	e
c	e	a	b	b	c
d	b	a	c	c	e
e	b	a	e	d	e

a	→	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)
b	→	4	4		1		3	5		2		5		
c	→	3	3		4		4	4		5		2		
d	→	2	5		2		5	2		1		3		
e	→	1	1		3		2	3		4		4		1
		5	2		5		1	1		3		1		4
		?	?		?		?	?		?		?		?

- Nach Abbildung auf 1 und 4 betrachtet man bezüglich der verbleibenden Elemente in der Tabelle die Ergebnisse $\Phi(4)$, und kann dadurch viele Fälle ausschließen.
- Es gibt einen Isomorphismus

POS (Theorie)

Homomorphismen

23 / 36

Isomorphe Graphen

Definition (Isomorphe Graphen)

Zwei Graphen heißen *isomorph*, wenn es eine bijektive Abbildung zwischen den Knotenmengen der beiden Graphen gibt, und außerdem folgende Bedingung erfüllt ist: Zwei Knoten in einem Graphen sind genau dann miteinander verbunden, wenn auch die entsprechenden Bildknoten im anderen Graphen miteinander verbunden sind.

Folgerungen:

- Knoten mit Grad n müssen auf Knoten mit dem selben Grad n abgebildet werden (notwendige Bedingung)
- Alle Wege und Kreise müssen durch die Abbildung erhalten werden.

POS (Theorie)

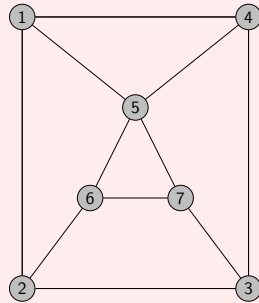
Homomorphismen

24 / 36

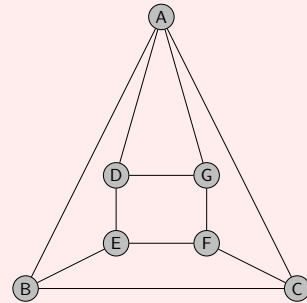
Aufgabe 1.8

Finden Sie alle isomorphen Abbildungen zwischen den folgenden Graphen:

Graph G_1 :

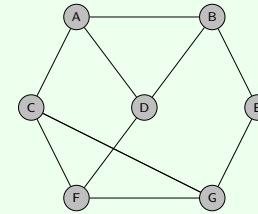


Graph G_2 :

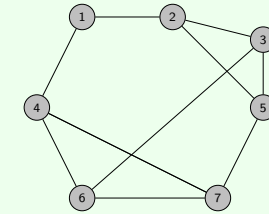


Isomorphe Graphen

G_1 :



G_2 :



Bestimmung aller möglichen Isomorphismen:

A	→	3	5	6	7
B	→	2	2	4	4
C	→	6	7	3	5
D	→	5	3	7	6
E	→	1	1	1	1
F	→	7	6	5	3
G	→	4	4	2	2

Es gibt nur einen Knoten mit Grad 2, nämlich E . Somit muss E auf den Knoten 1 abgebildet werden. Der Knoten E hat zwei Nachbarknoten B und G , die somit jeweils auf 2 oder 4 abgebildet werden müssen. Es sind also zwei Fälle zu unterscheiden. Für jeden Fall muss untersucht werden, ob er zu einer isomorphen Abbildung führt! (Fallunterscheidung (i) vs. (ii)) Im Fall (i) von $B \rightarrow 2$ kann nun $A \rightarrow 3$ oder $A \rightarrow 5$ abgebildet werden. Die Einträge $E \rightarrow 1$, $B \rightarrow 2$ sowie $G \rightarrow 4$ müssen hierfür in der Tabelle verdoppelt werden (weitere

Aufgabe 1.2

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	1	2	3	4	5
1	3	4	3	5	2
2	2	2	4	3	4
3	4	4	1	5	5
4	2	5	1	3	3
5	3	5	1	2	2

○	a	b	c	d	e
a	c				
b			b		
c					
d					
e					a

Aufgabe 1.3

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	1	2	3	4	5
1	3	2	5	1	4
2	4	4	3	4	5
3	1	1	4	2	4
4	3	4	3	1	2
5	5	5	1	3	3

○	a	b	c	d	e
a	c				
b			b		
c					
d					c
e					

Aufgabe 1.4

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	v	w	x	y	z
v	x	v	w	w	x
w	z	y	z	w	v
x	v	z	x	z	y
y	y	w	w	x	z
z	v	w	v	z	y

○	1	2	3	4	5
1			3		
2		5			
3					
4			4		
5					

Aufgabe 1.5

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, \circ \rangle$ und $\langle B, * \rangle$.

○	1	2	3	4	5
1					
2			4		
3					
4	1				
5			5		

*	a	b	c	d	e
a	b	c	a	d	d
b	a	a	d	c	e
c	e	a	b	b	c
d	b	a	c	c	e
e	b	a	e	d	e

Anmerkung: Durch Start mit der Gleichung $5 \circ 3 = 5$ kann man in die weitere Analyse auf drei Fälle ($a * c = a$, $c * e = c$ und $e * c = e$) beschränken!

Aufgabe 1.6

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	1	2	3	4	5
1	2	5	3	5	1
2	4	3	1	1	3
3	1	4	1	4	5
4	2	5	5	2	4
5	5	1	2	2	3

○	a	b	c	d	e
a		e		e	
b			c		
c					
d					
e					

Lösung:

	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
1	→	a	e	e		c	c	c	c
2	→	b	d	a	a	b	d	e	e
3	→	c	c	b	d	a	d	b	d
4	→	d	b	d	b	e	a	d	b
5	→	e	c	c	c	e	e	a	a
		✓	✓	✓	✓	✓	✓	✓	✓

Aufgabe 1.7

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	1	2	3	4	5
1	2	3	4	5	3
2	3	5	1	3	1
3	4	1	2	2	5
4	2	5	3	3	2
5	4	1	5	1	4

○	a	b	c	d	e
a				b	
b					
c					a
d	b				
e					

Aufgabe 1.8

Bestimmen Sie alle Isomorphismen zu den gegebenen Gruppoiden $\langle A, * \rangle$ und $\langle B, \circ \rangle$.

*	1	2	3	4	5
1	5	4	2	4	1
2	3	5	1	5	3
3	1	2	3	4	4
4	5	4	5	5	3
5	3	2	5	5	1

○	a	b	c	d	e
a		d			
b					
c			d		
d			c		
e					

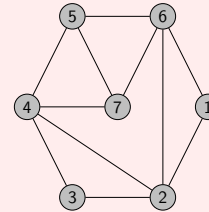
Lösung:

$1 \rightarrow e, 2 \rightarrow c, 3 \rightarrow b, 4 \rightarrow a, 5 \rightarrow d$
 $1 \rightarrow b, 2 \rightarrow c, 3 \rightarrow e, 4 \rightarrow a, 5 \rightarrow d$

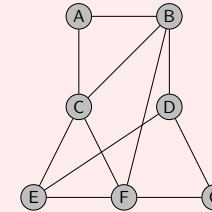
Aufgabe 1.8

- Welche der folgenden Graphen sind zueinander isomorph? Gegeben Sie gegebenenfalls *alle* Isomorphismen an!

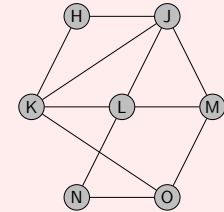
Graph X



Graph Y



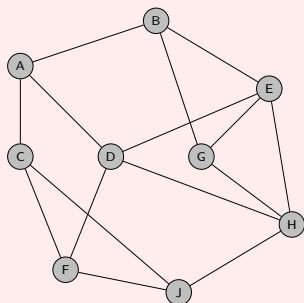
Graph Z



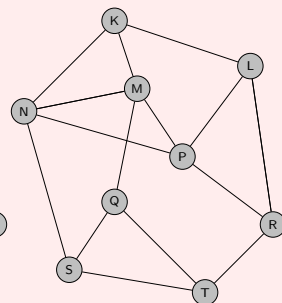
Aufgabe 1.9

- Welche der folgenden Graphen X, Y und Z sind zueinander isomorph? Gegeben Sie gegebenenfalls *alle* Isomorphismen an!

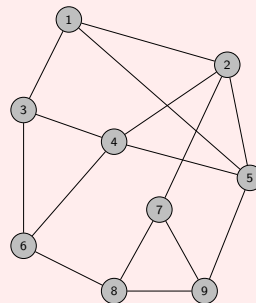
Graph X



Graph Y



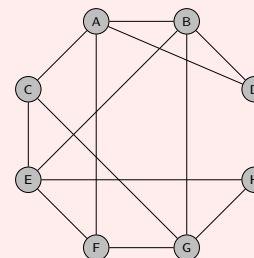
Graph Z



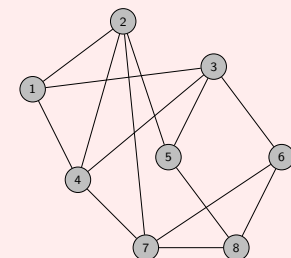
Aufgabe 1.10

- Welche der folgenden Graphen sind zueinander isomorph? Gegeben Sie gegebenenfalls *alle* Isomorphismen an!

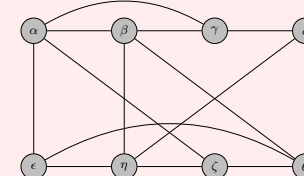
Graph G₁:



Graph G₂:



Graph G₃:



Formale Sprachen

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

POS (Theorie)

Formale Sprachen

1 / 16

Syntax

Gemeinsamkeiten von natürlichen und künstlichen Sprachen

Alphabet: vorgegebene endliche Menge an Zeichen (Symbolen) aus denen alle Elemente aufgebaut sind.

Grammatik: Regeln auf welche Weise diese Zeichen zu gültigen *Worten* kombiniert werden können.

- Mit *Wort* kann hierbei durchaus auch ein (natürlichsprachiger) Satz gemeint sein

Alphabet und Grammatik bilden die **Syntax** einer Sprache

POS (Theorie)

Formale Sprachen

3 / 16

Natürliche und künstliche Sprachen

Grobe Unterscheidung von Sprachen:

- **Natürliche Sprachen:** z.B. Deutsch, Englisch, Spanisch, Latein

- **Künstliche Sprachen:**

Beispiele:

- Chemie: Darstellung chemischer Reaktionen
- Musik: Notenschrift
- Mathematik, Logik: Formeln, etc.
- Informatik: Programmiersprachen, Ablaufdiagramme

In der Informatik finden künstliche Sprachen in der Praxis primär als Programmiersprachen Anwendung, jedoch auch in Protokollen. In der theoretischen Informatik spielen formale Sprachen eine zentrale Rolle zur Analyse von Komplexität, Berechenbarkeit etc.

POS (Theorie)

Formale Sprachen

2 / 16

Semantik

- Aus den syntaktischen Regeln allein können alle Sprachelemente hergeleitet werden, z.B.: Sätze, Kapitel, Bücher,...
- All diese Elemente werden als **Worte** der Sprache bezeichnet.
- Die *Bedeutung* der Worte wird **Semantik** genannt.
- Für Programmiersprachen wird durch die Semantik die Beziehung zwischen Zeichenketten und Aktionen des Rechners hergestellt.
- Die Zuordnung wird durch den Compiler/Interpreter umgesetzt.

POS (Theorie)

Formale Sprachen

4 / 16

Pragmatik

- Die **Pragmatik**: persönliche, subjektive Wahrnehmung oder Interpretation.
- Die Pragmatik einer Programmiersprache definiert ihren Einsatzbereich, d.h. sie gibt an, für welche Arten von Problemen die Programmiersprache besonders gut geeignet ist.
- **Beispiele**:
 - Kompatibilität
 - Verfügbare Bibliotheken
 - Erlernbarkeit
 - Eignung für Spezialanwendungen
 - Notation des Quellcodes
 - Integrierbarkeit

Beispiele

- **Beispiel**: Künstliche Sprache: Notenschrift:
 - Zeichenvorrat: Notensymbole, Schlüssel, Taktstriche,...
 - Syntax: Summe der (aufeinanderfolgenden) Notenwerte im Takt konstant, etc.
 - Semantik: Tonhöhe, Tonlänge,...
 - Pragmatik: Gefühlsmäßige Bestandteile, persönliche Empfindungen, Interpretationen;

Beispiele

- Es gilt: die Menge aller möglichen Worte ist größer als die Menge aller syntaktisch korrekten Worte, ist wiederum größer als die Menge aller semantisch korrekten Worte.
- **Beispiel**: Natürliche Sprache Deutsch:
 - **Menge der Symbole**: Großbuchstaben, Kleinbuchstaben, Ziffern, Satzzeichen
 - **Worte über dem Alphabet**: qwer4;:rwe?d 39fsd9Ä4fsd
 - **Worte der Sprache**: "Haus", "Er fährt mit dem Auto", Wörter, Sätze, Texte, Bücher,...
 - **Syntaktisch falsch**: Der wenn sein heute Projekt sprechen.
 - **Syntaktisch korrekt, semantisch falsch**: Der Tisch spricht gelb über Informatik.
 - **Syntaktisch korrekt, semantisch korrekt**: Der Student spricht oft über sein Projekt.

Beispiele

Die Beschreibung einer künstlichen (formalen) Sprache erfolgt durch die **Grammatik**.

Die Grammatik umfasst:

- Alphabet T der **Terminalsymbole**
- Menge N der **Nonterminalsymbole**: Hilfsvariablen zur Beschreibung der Sprache
- **Startvariable** $S \in N$.
- **Produktionsregeln** P , auch *Ersetzungsregeln* genannt

Eine Grammatik ist also gegeben durch

$$G = (N, T, P, S).$$

Beispiel: einfache Grammatik für die deutsche Sprache

Beispiel: Für die deutsche Sprache könnte eine primitive Grammatik andeutungsweise folgendes Aussehen haben:

- Alphabet T : 26 Groß- und Kleinbuchstaben, Sonderzeichen, Interpunktionszeichen
- Variablen $N = \{ \langle \text{Schriftstück} \rangle, \langle \text{Subjekt} \rangle, \langle \text{Objekt} \rangle, \langle \text{Substantiv im Nominativ} \rangle, \langle \text{Prädikat} \rangle, \langle \text{Substantiv im Akkusativ} \rangle, \dots \}$.

Anmerkung: Um diese Hilfssymbole als *metasprachliche* Größen zu kennzeichnen, werden sie in spitzen Klammern geschrieben.

- Als Oberbegriff S wählen wir $\langle \text{Schriftstück} \rangle$.
- Die Produktionsregeln P werden auf der folgenden Seite angegeben. Der Pfeil $\rightarrow$ deutet dabei eine *mögliche* Ersetzung an.

Beispiel: einfache Grammatik für die deutsche Sprache

Anmerkung: Betrachten wir die Definition von $\langle \text{Schriftstück} \rangle \rightarrow \langle \text{Hauptsatz} \rangle. \langle \text{Schriftstück} \rangle$. Diese erlaubt uns beliebig viele Hauptsätze aneinanderzureihen. Die *Rekursion* endet bei der Verwendung der Regel $\langle \text{Schriftstück} \rangle \rightarrow \langle \text{Hauptsatz} \rangle$.

Die konkrete Ableitung (tatsächliche Ersetzung) wird durch den Doppelpfeil $\Rightarrow$ notiert. Die dabei jeweils von einem Ableitungsschritt zum anderen entstehende, aus Terminal- und Nonterminalsymbolen aufgebaute Zeichenkette wird als **Satzform** bezeichnet.

Beispiel: einfache Grammatik für die deutsche Sprache

Produktionsregeln:

$\langle \text{Schriftstück} \rangle \rightarrow \langle \text{Hauptsatz} \rangle.$
 $\langle \text{Schriftstück} \rangle \rightarrow \langle \text{Hauptsatz} \rangle. \langle \text{Schriftstück} \rangle$
 $\langle \text{Hauptsatz} \rangle \rightarrow \langle \text{Subjekt} \rangle \langle \text{Prädikat} \rangle \langle \text{Objekt} \rangle$
 $\langle \text{Hauptsatz} \rangle \rightarrow \langle \text{Subjekt} \rangle \langle \text{Prädikat} \rangle$
 $\langle \text{Subjekt} \rangle \rightarrow \langle \text{Artikel im Nominativ} \rangle \langle \text{Substantiv im Nominativ} \rangle$
 $\langle \text{Objekt} \rangle \rightarrow \langle \text{Akkusativ-Objekt} \rangle$
 $\langle \text{Akkusativ-Objekt} \rangle \rightarrow \langle \text{Artikel im Akkusativ} \rangle \langle \text{Substantiv im Akkusativ} \rangle$

$\langle \text{Artikel im Nominativ} \rangle \rightarrow \text{der}$	$\langle \text{Prädikat} \rangle \rightarrow \text{liebt}$
$\langle \text{Artikel im Nominativ} \rangle \rightarrow \text{die}$	$\langle \text{Prädikat} \rangle \rightarrow \text{geht}$
$\langle \text{Artikel im Nominativ} \rangle \rightarrow \text{das}$	$\vdots$
$\langle \text{Artikel im Nominativ} \rangle \rightarrow \text{ein}$	$\langle \text{Artikel im Akkusativ} \rangle \rightarrow \text{den}$
$\langle \text{Artikel im Nominativ} \rangle \rightarrow \text{eine}$	$\langle \text{Artikel im Akkusativ} \rangle \rightarrow \text{die}$
$\vdots$	$\vdots$
$\langle \text{Substantiv im Nominativ} \rangle \rightarrow \text{Mensch}$	$\langle \text{Substantiv im Akkusativ} \rangle \rightarrow \text{Menschen}$
$\langle \text{Substantiv im Nominativ} \rangle \rightarrow \text{Luft}$	$\langle \text{Substantiv im Akkusativ} \rangle \rightarrow \text{Luft}$
$\langle \text{Substantiv im Nominativ} \rangle \rightarrow \text{Wald}$	$\langle \text{Substantiv im Akkusativ} \rangle \rightarrow \text{Wald}$

Beispiel: einfache Grammatik für die deutsche Sprache

Beispiel: Ableitung eines Satzes:

$\langle \text{Schriftstück} \rangle \Rightarrow \langle \text{Hauptsatz} \rangle.$
 $\Rightarrow \langle \text{Subjekt} \rangle \langle \text{Prädikat} \rangle \langle \text{Objekt} \rangle.$
 $\Rightarrow \langle \text{Artikel im Nominativ} \rangle \langle \text{Substantiv im Nominativ} \rangle \langle \text{Prädikat} \rangle \langle \text{Objekt} \rangle.$
 $\Rightarrow \dots \Rightarrow$
 $\Rightarrow \text{der Mensch} \text{ liebt } \langle \text{Akkusativ-Objekt} \rangle.$
 $\Rightarrow \text{der Mensch} \text{ liebt } \langle \text{Artikel im Akkusativ} \rangle \langle \text{Substantiv im Akkusativ} \rangle.$
 $\Rightarrow \Rightarrow \dots \Rightarrow$
 $\Rightarrow \text{der Mensch} \text{ liebt den Wald}.$

Es wäre auch möglich gewesen den Satz "der Mensch geht den Wald" abzuleiten. Dieser Satz wäre zwar syntaktisch korrekt, aber semantisch falsch.

Beispiel: Sprache der ganzen Zahlen

Beispiel: Wir definieren korrekte ganze Zahlen anhand einer Grammatik

$$\begin{aligned}
 T &= \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \\
 N &= \{ \langle \text{GanzeZahl} \rangle, \langle \text{VorlGanzeZahl} \rangle, \langle \text{Ziffer} \rangle, \langle \text{Vorzeichen} \rangle \} \\
 P &= \{ \langle \text{GanzeZahl} \rangle \rightarrow \langle \text{VorlGanzeZahl} \rangle, \\
 &\quad \langle \text{GanzeZahl} \rangle \rightarrow \langle \text{Vorzeichen} \rangle \langle \text{VorlGanzeZahl} \rangle, \\
 &\quad \langle \text{Vorzeichen} \rangle \rightarrow +, \\
 &\quad \langle \text{Vorzeichen} \rangle \rightarrow -, \\
 &\quad \langle \text{VorlGanzeZahl} \rangle \rightarrow \langle \text{Ziffer} \rangle, \\
 &\quad \langle \text{VorlGanzeZahl} \rangle \rightarrow \langle \text{Ziffer} \rangle \langle \text{VorlGanzeZahl} \rangle, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 0, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 1, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 2, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 3, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 4, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 5, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 6, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 7, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 8, \\
 &\quad \langle \text{Ziffer} \rangle \rightarrow 9 \\
 &\} \\
 S &= \langle \text{GanzeZahl} \rangle
 \end{aligned}$$

POS (Theorie)

Formale Sprachen

13 / 16

Beispiel: Sprache der ganze Zahlen

Eine Grammatik muss folgende Eigenschaften besitzen:

- Jedes gültige Wort muss ableitbar sein
- Ungültige Worte dürfen nicht ableitbar sein.

Übungsaufgabe

- Welches Problem besteht noch bei der vorangegangenen Definition der Sprache der ganzen Zahlen?
- Wie kann es behoben werden?

- $+/- 0$
- führende 0en

POS (Theorie)

Formale Sprachen

15 / 16

Beispiel: Sprache der ganzen Zahlen

Mit dem vertikalen Strich |, der verschiedene mögliche Ableitungen voneinander trennt, lassen sich die Produktionsregeln wesentlich kompakter anschreiben:

$$\begin{aligned}
 P = \{ & \langle \text{GanzeZahl} \rangle \rightarrow \langle \text{VorlGanzeZahl} \rangle \mid \langle \text{Vorzeichen} \rangle \langle \text{VorlGanzeZahl} \rangle, \\
 & \langle \text{Vorzeichen} \rangle \rightarrow + \mid -, \\
 & \langle \text{VorlGanzeZahl} \rangle \rightarrow \langle \text{Ziffer} \rangle \mid \langle \text{Ziffer} \rangle \langle \text{VorlGanzeZahl} \rangle, \\
 & \langle \text{Ziffer} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
 & \}
 \end{aligned}$$

Beispiel: Ableitung von -123

$$\begin{aligned}
 \langle \text{GanzeZahl} \rangle &\Rightarrow \langle \text{Vorzeichen} \rangle \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow - \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow - \langle \text{Ziffer} \rangle \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow -1 \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow -1 \langle \text{Ziffer} \rangle \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow -12 \langle \text{VorlGanzeZahl} \rangle \\
 &\Rightarrow -12 \langle \text{Ziffer} \rangle \\
 &\Rightarrow -123
 \end{aligned}$$

POS (Theorie)

Formale Sprachen

14 / 16

Beispiel: Sprache der ganze Zahlen

Lösung (Variante 1):

$$\begin{aligned}
 P = \{ & \langle \text{GanzeZahl} \rangle \rightarrow 0 \mid \langle \text{VorlGZ} \rangle \mid \langle \text{Vorzeichen} \rangle \langle \text{VorlGZ} \rangle, \\
 & \langle \text{Vorzeichen} \rangle \rightarrow + \mid -, \\
 & \langle \text{VorlGZ} \rangle \rightarrow \langle \text{PosZiffer} \rangle \mid \langle \text{PosZiffer} \rangle \langle \text{BelZifferZahl} \rangle, \\
 & \langle \text{BelZifferZahl} \rangle \rightarrow \langle \text{Ziffer} \rangle \mid \langle \text{Ziffer} \rangle \langle \text{BelZifferZahl} \rangle, \\
 & \langle \text{PosZiffer} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9, \\
 & \langle \text{Ziffer} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
 & \}
 \end{aligned}$$

Lösung (Variante 2):

$$\begin{aligned}
 P = \{ & \langle \text{GanzeZahl} \rangle \rightarrow \langle \text{Null} \rangle \mid \langle \text{VorlGZ} \rangle \mid \langle \text{Vorzeichen} \rangle \langle \text{VorlGZ} \rangle, \\
 & \langle \text{Vorzeichen} \rangle \rightarrow + \mid -, \\
 & \langle \text{VorlGZ} \rangle \rightarrow \langle \text{PosZiffer} \rangle \mid \langle \text{VorlGZ} \rangle \langle \text{PosZiffer} \rangle \mid \langle \text{VorlGZ} \rangle \langle \text{Null} \rangle, \\
 & \langle \text{PosZiffer} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9, \\
 & \langle \text{Null} \rangle \rightarrow 0 \\
 & \}
 \end{aligned}$$

POS (Theorie)

Formale Sprachen

16 / 16

Backus-Naur-Form

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

POS (Theorie)

Backus-Naur-Form

1 / 23

Backus-Naur-Form

Beschreibung des Sprachelementes $\langle \text{identifizier} \rangle$ der Programmiersprache ALGOL-60 mittels BNF:

$\langle \text{digit} \rangle ::= 0|1|2|3|4|5|6|7|8|9$

$\langle \text{letter} \rangle ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|$
 $r|s|t|u|v|w|x|y|z|A|B|C|D|E|F|G|$
 $H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|$
 $W|X|Y|Z$

$\langle \text{identifizier} \rangle ::= \langle \text{letter} \rangle | \langle \text{identifizier} \rangle \langle \text{letter} \rangle | \langle \text{identifizier} \rangle \langle \text{digit} \rangle$

Anmerkung: Das syntaktische Objekt $\langle \text{identifizier} \rangle$ wird aus einer beliebigen Kombination von Buchstaben und Ziffern gebildet, wobei an der ersten Stelle ein Buchstabe stehen muss. Dies wird durch den rekursiven zeichenweisen Aufbau von rechts nach links und durch das Verlassen der Rekursion mittels eines Buchstaben sichergestellt.

POS (Theorie)

Backus-Naur-Form

3 / 23

Backus-Naur-Form

Anlässlich der Definition der Programmiersprache ALGOL-58/ALGOL-60 wurde von John Backus und Peter Naur eine neue Darstellungsform (**Backus-Naur-Form, BNF**) für die formale Definition der Syntax von Programmiersprachen entwickelt, die folgende Metasprachlichen Symbole verwendet:

Definition (Backus-Naur-Form: Notation)

- $::=$ Definitions-, bzw. Ersetzungszeichen
- $|$ Entweder-Oder-Zeichen
- $\langle, \text{bzw.} \rangle$ Spitze Klammern zur Kennzeichnung von Variablen

POS (Theorie)

Backus-Naur-Form

2 / 23

Backus-Naur-Form

Beispiel

- Syntaktisch richtige $\langle \text{identifizier} \rangle$:
 - buchstabe
 - V17a
 - Feld27
- Syntaktisch falsche $\langle \text{identifizier} \rangle$:
 - 1bz
 - \$abc
 - f(x)

POS (Theorie)

Backus-Naur-Form

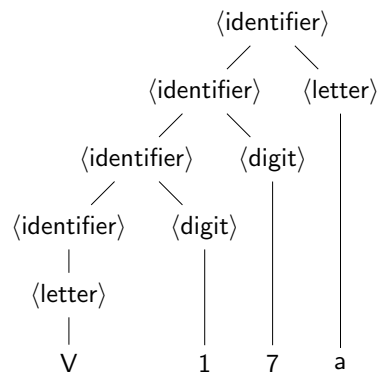
4 / 23

Ableitung (1)

Rechtsableitung von V17a

$\langle \text{identifier} \rangle \Rightarrow \langle \text{identifier} \rangle \langle \text{letter} \rangle$
 $\Rightarrow \langle \text{identifier} \rangle a$
 $\Rightarrow \langle \text{identifier} \rangle \langle \text{digit} \rangle a$
 $\Rightarrow \langle \text{identifier} \rangle 7a$
 $\Rightarrow \langle \text{identifier} \rangle \langle \text{digit} \rangle 7a$
 $\Rightarrow \langle \text{identifier} \rangle 17a$
 $\Rightarrow \langle \text{letter} \rangle 17a$
 $\Rightarrow V17a$

Syntaxbaum / Ableitungsbaum



- Jedem solchen Ableitungs-Prozess kann man einen **Ableitungsbaum**, bzw. **Syntaxbaum**, bzw. **Produktionsbaum** zuordnen.
- Wir zeigen anhand des Ableitungsbaumes, dass V17a ein gültiger $\langle \text{identifier} \rangle$ ist.
- Reihenfolge ist im Ableitungsbaum nicht nachvollziehbar $\Rightarrow$ somit klar, dass Reihenfolge unerheblich!

Ableitung (2)

Linksableitung von V17a

$\langle \text{identifier} \rangle \Rightarrow \langle \text{identifier} \rangle \langle \text{letter} \rangle$
 $\Rightarrow \langle \text{identifier} \rangle \langle \text{digit} \rangle \langle \text{letter} \rangle$
 $\Rightarrow \langle \text{identifier} \rangle \langle \text{digit} \rangle \langle \text{digit} \rangle \langle \text{letter} \rangle$
 $\Rightarrow \langle \text{letter} \rangle \langle \text{digit} \rangle \langle \text{digit} \rangle \langle \text{letter} \rangle$
 $\Rightarrow V \langle \text{digit} \rangle \langle \text{digit} \rangle \langle \text{letter} \rangle$
 $\Rightarrow V1 \langle \text{digit} \rangle \langle \text{letter} \rangle$
 $\Rightarrow V17 \langle \text{letter} \rangle$
 $\Rightarrow V17a$

Erweiterte Backus-Naur-Form (EBNF)

- Ergänzung um Zeichen { und } in der **Erweiterten Backus-Naur-Form (EBNF)**
- Darin stehende Ausdrücke können *beliebig oft* verwendet werden.
- Verwendung für die Definition von PASCAL

Beschreibung des Sprachelementes $\langle \text{identifier} \rangle$ der Programmiersprache PASCAL mittels EBNF:

$\langle \text{digit} \rangle ::= 0|1|2|3|4|5|6|7|8|9$

$\langle \text{letter} \rangle ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|$
 $r|s|t|u|v|w|x|y|z|A|B|C|D|E|F|G|$
 $H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|$
 $W|X|Y|Z$

$\langle \text{letter or digit} \rangle ::= \langle \text{letter} \rangle | \langle \text{digit} \rangle$

$\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{letter or digit} \rangle \}$

Anmerkung: Unterstrich war in der ursprünglichen Definition nicht vorgesehen.

Allgemeine Backus-Naur-Form (ABNF)

- Neben { und } können zusätzlich auch die Symbole [und] verwendet werden
- Die darin enthaltenen Ausdrücke können *höchstens einmal* verwendet werden.
- Anstatt der spitzen Klammern wird für Nonterminale (Variablen) **Fettdruck** verwendet.
- Die Programmiersprachen ADA und MODULA-2 sind in ABNF definiert.

Mehrdeutigkeit einer Grammatik

- Eine Grammatik G heißt **eindeutig**, wenn für jedes Wort w , das gemäß G aus S ableitbar ist, gilt: die mit unterschiedlichen Ableitungen von w verbundenen Ableitungsbäume sind identisch.
- ebenso, wenn es nur eine einzige Ableitung von w gibt (Spezialfall)
- Eine Grammatik heißt **mehrdeutig**, wenn sie nicht eindeutig ist.
- Ist G mehrdeutig, dann gibt es immer ein w und zwei Ableitungen von w , die unterschiedliche Ableitungsbäume erzeugen.
- In der Praxis von Programmiersprachen werden mehrdeutige Grammatiken nicht benutzt, da sie zu Problemen sowohl bei der Definition der Semantik als auch bei der Programmanalyse führen.
- Eine formale Sprache L kann im i. A. durch mehr als eine Grammatik beschrieben werden.
- Eine formale Sprache L heißt *inhärent mehrdeutig* genau dann, wenn alle Grammatiken für L mehrdeutig sind

Allgemeine Backus-Naur-Form (ABNF)

Beschreibung des Sprachelementes $\langle \text{identifier} \rangle$ der Programmiersprache ADA mittels ABNF:

digit ::= 0|1|2|3|4|5|6|7|8|9

letter ::= **lower_case_letter** | **upper_case_letter**

lower_case_letter ::= a | b | c | d | e | f | g | h | i | j | k |
l | m | n | o | p | q | r | s | t | u |
v | w | x | y | z

upper_case_letter ::= A | B | C | D | E | F | G | H | I | J |
K | L | M | N | O | P | Q | R | S | T |
U | V | W | X | Y | Z

letter_or_digit ::= **letter** | **digit**

underscore ::= _

identifier ::= **letter** | { [**underscore**] **letter_or_digit** }

Aufgabe 3.1

Die syntaktische Bildung von "sequence" gehorche folgender Menge P von Produktionsregeln:

$\langle \text{letter} \rangle ::= a | b | \dots | z$

$\langle \text{digit} \rangle ::= 0 | 1 | \dots | 9$

$\langle \text{word} \rangle ::= \langle \text{letter} \rangle \langle \text{word} \rangle | \langle \text{letter} \rangle \langle \text{digit} \rangle$

$\langle \text{assignment} \rangle ::= \langle \text{word} \rangle = \langle \text{digit} \rangle$

$\langle \text{sequence} \rangle ::= \langle \text{assignment} \rangle ; | \langle \text{assignment} \rangle , \langle \text{sequence} \rangle$

- Wie sehen die restlichen Bestimmungsstücke der Grammatik $G = (N, T, P, S)$ aus?
- Welche der folgenden Symbolketten sind keine richtige "sequence"? Lokalisieren Sie *alle* Fehler!

• x1=3,y1=7;

• aa1=bb1=0,cc1=1;

• i3=1,k2=i

• s5=25;

• klmno1=0

• a12b=3,

• a=1 bb1=7;

• r25=6

• A7=9;

SPL: Simple Programming Language

- Simple Programming Language (SPL) ist die Definition einer extrem einfachen Programmiersprache zur Veranschaulichung von Grammatiken formaler Sprachen.
- Die Sprache verfügt weder über ein *Typsystem*, noch *Methoden* oder gar *Klassen*.
- Dennoch ist die Sprache ausreichend um alle denkbaren Berechnungen durchzuführen, jedoch vergleichsweise umständlich und unkomfortabel.

SPL: Simple Programming Language

```

<comparison> ::= <expression> <relation> <expression>
<expression> ::= <term> | <expression> <weak operator> <term>
<term> ::= <element> | <term> <strong operator> <element>
<element> ::= <constant> | <variable> | (<expression>)
<constant> ::= <digit> | <constant> <digit>
<variable> ::= <letter> | <variable> <letter> | <variable> <digit>
<relation> ::= = | <= | < | > | >= | <>
<weak operator> ::= + | -
<strong operator> ::= * | /
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter> ::= a | b | c | d | e | f | g | h | i | j | k | l | m |
             n | o | p | q | r | s | t | u | v | w | x | y | z

```

SPL: Simple Programming Language

Produktionsregeln zur SPL in BNF:

```

<program> ::= <statement list>
<statement list> ::= <statement> | <statement list> ; <statement>
<statement> ::= <io statement> | <assignment statement> |
                <conditional statement> | <definite loop> | <indefinite loop>
<io statement> ::= read <variable list>
<io statement> ::= write <variable list>
<variable list> ::= <variable> | <variable list> , <variable>
<assignment statement> ::= <variable> := <expression>
<conditional statement> ::= if <comparison> then <statement list> fi |
                            if <comparison> then <statement list> else <statement list> fi
<definite loop> ::= to <expression> do <statement list> end
<indefinite loop> ::= while <comparison> do <statement list> end

```

SPL: Simple Programming Language

Aufgabe 3.2

- Gibt es Syntaxfehler in folgendem SPL-Programm?
- Was würde es tun wenn es korrekt wäre?

```

1  read n;
2  to n do
3      read x;
4      if x>0 then
5          y := 1;
6          z := 1;
7          while z<>x do
8              z := z+1;
9              y := y*z;
10         end;
11         write y
12     fi
13 end

```

SPL: Simple Programming Language

Aufgabe 3.3

Entwickeln Sie einen SPL-Dialekt, indem der Strichpunkt Bestandteil *jeder* Anweisung ist (und nicht nur Anweisungen voneinander trennt).

Definition (Perfekte Zahl)

Unter einer perfekten Zahl versteht man eine natürliche Zahl, deren Wert gleich der Summe *aller* ihrer echten Teiler ist.

Beispiel: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$, $496 = \dots$

Aufgabe 3.4

Entwickeln Sie ein kleines SPL-Programm, das eine positive ganze Zahl n einliest, und die kleinste perfekte Zahl größer n ausgibt.

POS (Theorie)

Backus-Naur-Form

17 / 23

Lösung (3.5): Wir beginnen die Ableitung mit $\langle \text{statement} \rangle$ (laut Angabe!). Bei der Ableitung werden einige Schritte übersprungen. Geben Sie bei Tests oder Abgaben aber stets alle Zwischenschritte an!

```

<statement>
⇒ <indefinite loop>
⇒ while <comparison> do <statement list> end
⇒ while <expr><rel><expr> do
    <statement list>;
    <statement>
end
⇒ ... ⇒
while z <> x do
    <statement>;
    <statement>
end

```

POS (Theorie)

Backus-Naur-Form

19 / 23

SPL: Simple Programming Language

Aufgabe 3.5

Zeigen Sie, dass es sich bei

```

while z <> x do
    z := z + 1;
    y := y * z
end

```

um ein gültiges SPL-Statement handelt.

POS (Theorie)

Backus-Naur-Form

18 / 23

Fortsetzung (Lösung 3.5)

```

⇒ ... ⇒
while z <> x do
    <assignment statement>;
    <assignment statement>
end
⇒ ... ⇒
while z <> x do
    <variable> := <expression>;
    <variable> := <expression>
end
⇒ ... ⇒
while z <> x do
    z := z + 1;
    y := y * z
end

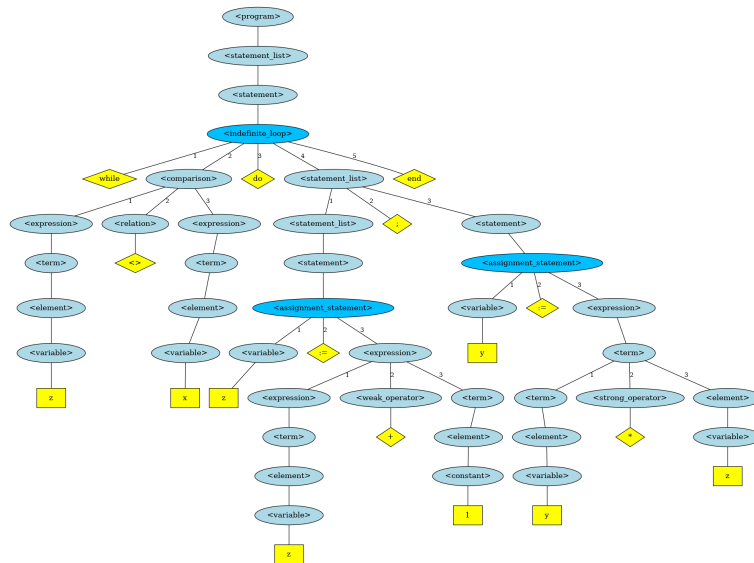
```

POS (Theorie)

Backus-Naur-Form

20 / 23

Alternative Lösung zu 3.5 mittels Syntaxbaum:



Aufgabe 3.6

- Gibt es Syntaxfehler in folgendem SPL-Programm?
- Was würde es tun wenn es korrekt wäre?

```

1      read n
2      if n>0 then
3          a = 1;
4          b = 1;
5          write 1;
6          if n>1 then
7              write 1;
8              to n-2 do
9                  c = a + b;
10                 write c;
11                 a = b;
12                 b = c;
13             end;
14         end
15     fi

```

Lösung zu Aufgabe 3.4

```

1      read n;
2      if n>0 then
3          sum := 0;
4          while n <> sum do
5              y := 2;
6              sum := 1;
7              n := n + 1;
8              while y*y < n do
9                  a := n/y;
10                 if n=y*a then
11                     sum := a + y + sum
12                 fi;
13                 y := y + 1
14             end;
15             if y*y=n then
16                 sum := sum + y
17             fi
18         end;
19         write n
20     fi

```

Grammatiken

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

POS (Theorie)

Grammatiken

1 / 16

Verkettung

Definition (Verkettung)

Sind $x = x_1x_2 \dots x_n$ und $y = y_1y_2 \dots y_m$ zwei beliebige Wörter aus der Menge T^+ , dann ist die **Zusammensetzung oder Verkettung** definiert als

$$xy = x_1x_2 \dots x_ny_1y_2 \dots y_m.$$

- Wird zu T^+ ein spezielles Wort ε (Epsilon) hinzugefügt, so erhält man die Menge T^* .

$$T^* = T^+ \cup \{\varepsilon\}$$

- ε wird als **Leerwort** bezeichnet. Es ist kein Terminalsymbol (und somit auch nicht im Alphabet enthalten).
- Es gilt:

$$x\varepsilon = \varepsilon x = x$$

POS (Theorie)

Grammatiken

3 / 16

Worte der Länge n

- Ein **Alphabet** ist eine nichtleere endliche Menge, deren Elemente Zeichen oder Symbole genannt werden.
- Ein **Wort der Länge n** ist eine Folge $x_1x_2 \dots x_n$ von n Symbolen des Alphabets A
- Jede beliebige geschlossene Gruppe von Zeichen innerhalb eines Wortes wird als **Teilwort** bezeichnet.
- Zwei Wörter x und y werden als *gleich* bezeichnet, wenn sie die gleiche Länge besitzen und auch die Reihenfolge der Zeichen gleich ist.
- Sei T das Alphabet, so bezeichnet T^+ die Menge aller daraus konstruierbaren Wörter.

POS (Theorie)

Grammatiken

2 / 16

Länge eines Wortes

- Die **Länge eines Wortes** x ist durch die Anzahl der Zeichen des Wortes bestimmt:

$$|x_1x_2 \dots x_n| = n$$

- Es gilt:

$$|xy| = |x| + |y|$$

- Für das Leerwort gilt:

$$|\varepsilon| = 0$$

POS (Theorie)

Grammatiken

4 / 16

Verkettung von Wortmengen

Definition (Verkettung von Wortmengen)

Seien M und N zwei Wortmengen. Die Verkettung MN ist definiert als

$$MN = \{x \mid x = yz, y \in M, z \in N\}.$$

- Die als Ergebnis entstehende Menge enthält alle jene Worte, die aus einem Wort der Menge M verkettet mit einem Wort der Menge N bestehen.
- Die Menge MN wird als das Produkt von M und N bezeichnet.
- Für die Produktoperation gilt das Kommutativgesetz **nicht!**
- Somit gilt im Allgemeinen: $MN \neq NM$

Verkettung von Wortmengen

Die Operationen “+” und “*” können auch auf Wortmengen angewandt werden.

Beispiel: Gegeben sei die Wortmenge $M = \{c, bba, bcacc\}$. Eine mögliche Teilmenge Z von M^* ist:

$$Z = \{\varepsilon, c, ccccc, cbbac, cbbabcacc, bbabbabcaccc, cccbbacc\}$$

Verkettung von Wortmengen

Beispiel: Gegeben seien zwei Alphabete Q und R sowie die beiden Wortmengen $M \subseteq Q^+$ und $N \subseteq R^+$.

$$\begin{aligned} Q &= \{a, b, c\} & M &= \{c, bba, bcacc\} \\ R &= \{D, E, F, G\} & N &= \{FE, DDF\} \end{aligned}$$

Man beachte den Unterschied

$$MN = \{cFE, cDDF, bbaFE, bbaDDF, bcaccFE, bcaccDDF\}$$

$$NM = \{FEc, FEbba, FEbcacc, DDFc, DDFbba, DDFbcacc\}$$

WH: Formale Sprache

- Eine *formale Sprache* L über einem Alphabet T ist eine beliebige Teilmenge von T^* .
- Grammatiken sind formale Systeme die zur Erzeugung formaler Sprachen dienen.
- Grammatiken sind definiert als

$$G = (N, T, P, S),$$

für die gilt:

- Alphabet der Nichtterminalsymbole N
- Alphabet der Terminalsymbole T mit $N \cap T = \emptyset$.
Weiters sei $V := N \cup T$.
- Menge der Produktionsregeln P
Eine Regel schreiben wir abstrakt als $\omega_1 \rightarrow \omega_2$, bzw. $(\omega_1, \omega_2) \in P$.
- Startvariable S mit $S \in N$

Produktionsregeln

- Wir untersuchen nun die linke Seite ω_1 und die rechte Seite ω_2 von Produktionsregeln $\omega_1 \rightarrow \omega_2$ genauer.
- Sei $\omega_1 \in N$, so besteht die linke Seite der Regel jeweils nur aus einem Nonterminal
- Sei $\omega_2 \in NT$, so ist die rechte Seite der Regel von der Form: ein Nonterminalsymbol gefolgt von einem Terminalsymbol
 - Hier wurde die Verkettung NT der Mengen N und T verwendet.
- Sei $\omega_2 \in T \cup TN$, so besteht die rechte Seite der Regel entweder aus einem Terminalsymbol, oder aus einem Terminalsymbol gefolgt von einem Nonterminalsymbol.

Chomsky-Hierarchie

- **(Rechts-)Reguläre Grammatik (Typ-3-Grammatik):**
Links von $\rightarrow$ steht genau ein Nonterminal. Rechts von $\rightarrow$ steht entweder ein Terminalsymbol oder ein Terminalsymbol gefolgt von genau einem Nonterminal
- **Kontextfreie Grammatik (Typ-2-Grammatik):**
Links von $\rightarrow$ steht genau ein Nonterminal, rechts davon eine beliebige Folge von Terminalen und Nonterminalen (genauer aus V^*)
- **Kontextsensitive (umgebungsabhängige) Grammatik (Typ-1-Grammatik):**
Hier sind zusätzlich Produktionsregeln erlaubt, auf deren linker Seite auch Terminalsymbole vorkommen, jedoch mindestens ein Nonterminal enthalten ist. Dieser "Kontext" der Terminalsymbole muss auf der rechten Seite der Produktionsregel erhalten bleiben.
- **Allgemeine Regelgrammatik (Typ-0-Grammatik):**
Die Einschränkungen für die rechte Seite bei der Typ-1-Grammatik gelten hier nicht.

Chomsky-Hierarchie

- Die **Chomsky-Hierarchie** unterscheidet verschiedene Arten von Grammatiken aufgrund der Beschaffenheit ihrer Produktionsregeln.
- Grammatiken vom **Typ-0**, **Typ-1**, **Typ-2** und **Typ-3** unterscheiden sich durch ihre Mächtigkeit.
- Die Typ-0 Grammatiken weisen die größte Mächtigkeit auf, während Typ-3 Grammatiken nur relativ einfache Sprachen erzeugen können.

Chomsky-Hierarchie (*)

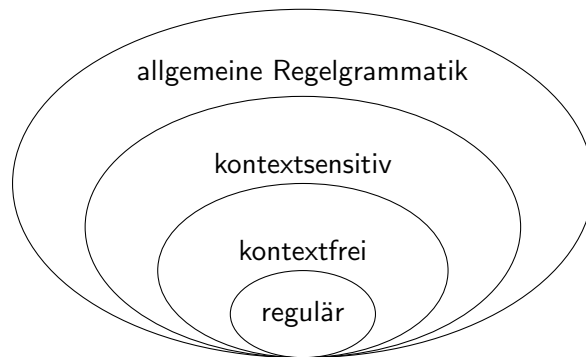
- **(Rechts-)Reguläre Grammatik (Typ-3-Grammatik):**
$$\forall (\omega_1 \rightarrow \omega_2) \in P : \omega_1 \in N \wedge \omega_2 \in T \cup TN \cup \{\varepsilon\}$$
- **Kontextfreie Grammatik (Typ-2-Grammatik):**
$$\forall (\omega_1 \rightarrow \omega_2) \in P : \omega_1 \in N \wedge \omega_2 \in V^*$$
- **Kontextsensitive (umgebungsabhängige) Grammatik (Typ-1-Grammatik):**
Alle Produktionsregeln sind von der Form:
$$\alpha A \beta \rightarrow \alpha \gamma \beta \text{ mit } A \in N \wedge \alpha, \beta, \gamma \in V^*,$$

oder $S \rightarrow \varepsilon$
- **Allgemeine Regelgrammatik (Typ-0-Grammatik):**
$$\forall (\omega_1 \rightarrow \omega_2) \in P : \omega_1 \in V^+ \wedge \omega_2 \in V^*$$

wobei ω_1 mindestens ein Nonterminal enthalten muss.

Chomsky-Hierarchie

Die Grammatiken höheren Typs sind in jenen niedrigeren Typs jeweils vollständig enthalten:



Chomsky-Hierarchie

Sprachen höheren Typs sind *echte Teilmengen* der Sprachklassen niederen Typs.

Grammatik	Sprache	Automat
Typ-0	rekursiv aufzählbar	nichtdeterministische Turingmaschine
Typ-1	kontextsensitiv	nichtdeterministische Turingmaschine mit linear beschränkter Bandlänge
Typ-2	kontextfrei	nichtdeterministischer Kellerautomat
Typ-3	regulär	endlicher Automat

Programmiersprachen sind im Allgemeinen kontextfreie Sprachen!

Beispiele zur Chomsky-Hierarchie

Beispiel:

T-3: $\langle A \rangle \rightarrow x$
 $\langle A \rangle \rightarrow x\langle B \rangle$

T-2: $\langle A \rangle \rightarrow x\langle A \rangle y\langle B \rangle c$
 $\langle \text{conditional statement} \rangle \rightarrow \text{if } \langle \text{expression} \rangle \text{ then } \langle \text{statementlist} \rangle \text{ fi}$

T-1: $x\langle A \rangle y \rightarrow x\langle B \rangle z\langle C \rangle w\langle D \rangle \langle E \rangle y$

T-0: $\dots \rightarrow \dots$ (mindestens ein Nonterminal auf der linken Seite)
 $x\langle A \rangle \rightarrow \langle B \rangle$

Aufgaben

Aufgabe 3.8

- Welche Regeln von SPL sind regulär?
- Wie können die anderen Regeln klassifiziert werden?

Reguläre Sprachen und Endliche Automaten

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

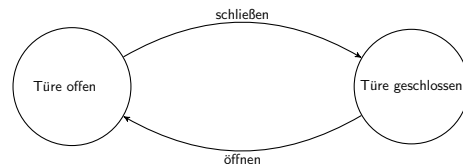
POS (Theorie)

Reguläre Sprachen

1 / 18

Echtwelt-Beispiel: Endlicher Automat

- Automaten *beschreiben* also reguläre Sprachen...
- ... jedoch auch Anwendungen/Maschinen ("Automaten") der "echten Welt"
- Beispiel eines Endlichen Automaten der eine Türe beschreibt:



- Zustände: Türe offen oder geschlossen
- "schließen" und "öffnen" sind Zustandsübergänge

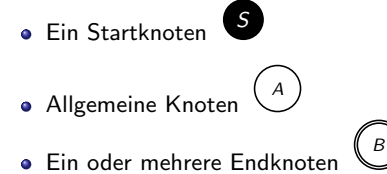
POS (Theorie)

Reguläre Sprachen

3 / 18

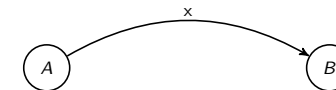
Einleitung

- Worte regulärer Sprachen können durch **Endliche Automaten** auf syntaktische Korrektheit überprüft werden.
- Ein *Endliche Automat* ist ein abstraktes Konzept, bestehend aus:
 - **Zuständen** (dargestellt durch Knoten):



- **Zustandsübergängen:**

- dargestellt durch Kanten mit Beschriftung (Aktion der Zustandsänderung, z.B. eingelesenes Zeichen)



- Ein endlicher Automat ist also ein gerichteter Graph

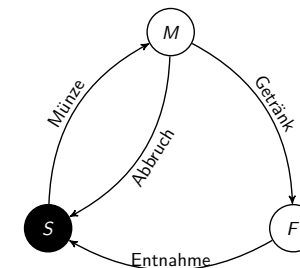
POS (Theorie)

Reguläre Sprachen

2 / 18

Echtwelt-Beispiel: Getränke-Automat

Beispiel: Getränkeautomat:



- Startzustand *S* (in dieser Abbildung durch einen Pfeil gekennzeichnet)
- Getränkeausgabe erst von Zustand *M* aus möglich
- Mit Münze gelangt man von *S* zu *M*
- Von *M* (also nach eingeworfener Münze) kann man entweder Abbrechen, oder ein Getränk wird ausgegeben.
- Nach ausgegebenem Getränk (Zustand *G*) ist die Entnahme möglich. Erst nach der Entnahme akzeptiert der Automat in *S* wieder weitere Münzen.

POS (Theorie)

Reguläre Sprachen

4 / 18

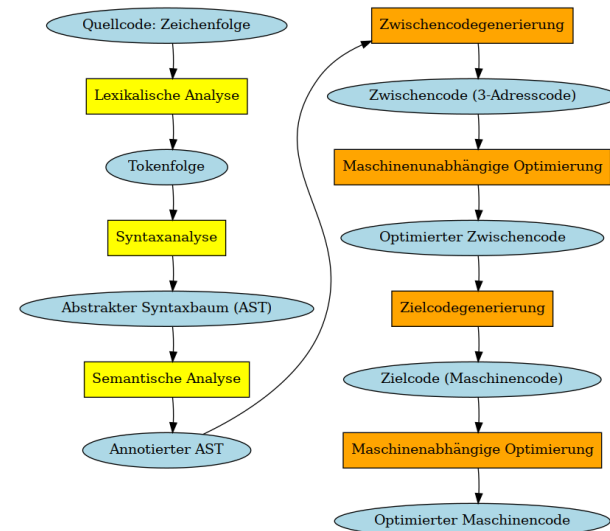
Syntaxanalyse in Compilern

- Viele Elemente von Programmiersprachen werden durch reguläre Sprachen beschrieben.
- Erster Schritt der Compilierung ist die *Syntaxanalyse*.
- Die erste Phase dabei ist die *lexikalische Analyse* ("Scanner", "Lexer")
- In der lexikalischen Analyse wird der Quelltext in zusammengehörige Einheiten "Tokens" zerteilt.
- Die Tokens werden dann *Schlüsselworten* (Keywords) oder *Bezeichnern* (Identifiers) zugeordnet.
- Bezeichner können anhand endlicher Automaten auf Korrektheit geprüft werden.
- Die Phasen eines Compilers sind auf der nächsten Folie dargestellt.

Automaten als Akzeptoren

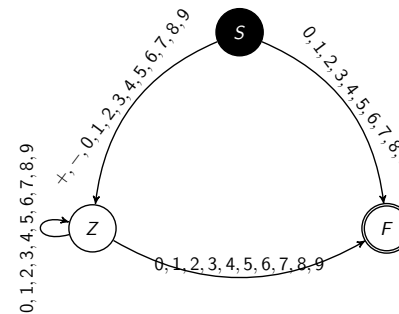
- Ein endlicher Automat überprüft Worte regulärer Sprachen auf Korrektheit, d.h. er **akzeptiert** syntaktisch korrekte Worte (syntaktisch falsche Worte werden nicht akzeptiert).
- Dabei liest der Automat Zeichen für Zeichen des Wortes, und führt entsprechende Zustandsübergänge durch.
- Ein Wort ist gültig, wenn sich der Automat nach dem letzten gelesenen Zeichen in einem Endzustand befindet.
- Ein Wort ist ungültig, wenn es zu einem gelesenen Zeichen keinen entsprechenden Zustandsübergang gibt, oder wenn nach dem letzten gelesenen Zeichen kein Endzustand erreicht wurde.

Phasen eines Compilers



Sprache der ganzen Zahlen (mit führenden 0en, oder ± 0)

Konvention: wenn Startsymbol nicht explizit angegeben, dann "S".



Beispiel:

Ableitung von -144

$\langle S \rangle \Rightarrow - \langle Z \rangle$

$\langle Z \rangle \Rightarrow 1 \langle Z \rangle$

$\langle Z \rangle \Rightarrow 4 \langle Z \rangle$

$\langle Z \rangle \Rightarrow 4$

$T = \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$N = \{S, Z\}$

$P = \{S \rightarrow +Z | -Z | 0Z | 1Z | 2Z | 3Z | 4Z | 5Z | 6Z | 7Z | 8Z | 9Z | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9, \\ Z \rightarrow 0Z | 1Z | 2Z | 3Z | 4Z | 5Z | 6Z | 7Z | 8Z | 9Z | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9\}$

Sprache der ganzen Zahlen (ohne führende 0en, ohne ± 0)

Aufgabe 3.10

Adaptieren Sie die Grammatik der Sprache der ganzen Zahlen mit führenden 0en, sodaß diese nicht mehr möglich sind, und ebenso ± 0 ... ausgeschlossen wird.

$$P = \{ S \rightarrow +A|-A|1B|2B|3B|4B|5B|6B|7B|8B|9B|0|1|2|3|4|5|6|7|8|9, \\ A \rightarrow 1B|2B|3B|4B|5B|6B|7B|8B|9B|1|2|3|4|5|6|7|8|9, \\ B \rightarrow 0B|1B|2B|3B|4B|5B|6B|7B|8B|9B|0|1|2|3|4|5|6|7|8|9 \}$$

Sprache der reellen Zahlen

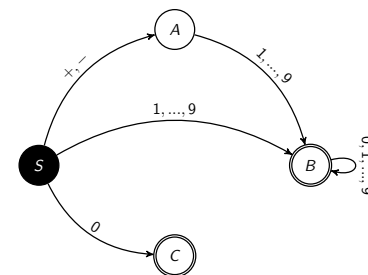
$$P = \{ S \rightarrow +A|-A|1B|2B|3B|4B|5B|6B|7B|8B|9B|0C|0|1|\dots|9; \\ A \rightarrow 1B|2B|3B|4B|5B|6B|7B|8B|9B|0D|1|\dots|9; \\ B \rightarrow 0B|1B|2B|3B|4B|5B|6B|7B|8B|9B|E; \\ C \rightarrow ,E; \\ D \rightarrow ,E; \\ E \rightarrow 0E|1F|2F|3F|4F|5F|6F|7F|8F|9F|1|\dots|9; \\ F \rightarrow 0E|1F|2F|3F|4F|5F|6F|7F|8F|9F|1|\dots|9 \\ \}$$

Anmerkung: Hier wird der Strichpunkt als Trennzeichen für die einzelnen Produktionsregeln verwendet.

Deterministischer Endlicher Automat

- Bei einem *deterministischen endlichen Automaten* sind die Zustandsübergänge *eindeutig*!
- Bei den Kantenbeschriftungen der von einem Knoten auslaufenden Kanten kommt eine Beschriftung also nur ein mal vor.

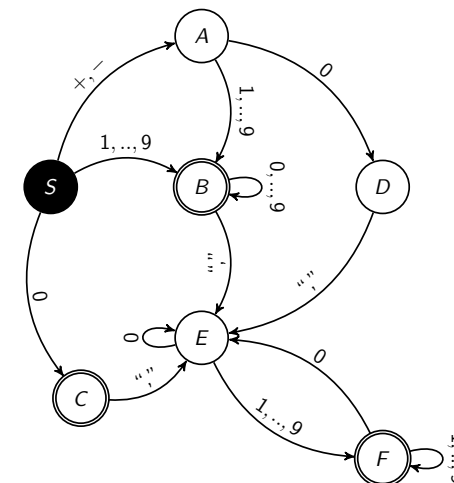
Beispiel: Deterministischer endlicher Automat (DEA) für die Sprache der Ganzen Zahlen ohne führende 0en und ohne ± 0 :



$$T = \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \\ N = \{S, A, B\} \\ P = \{ S \rightarrow +A|-A|1B|2B|\dots|9B|0|1|\dots|9, \\ A \rightarrow 1B|2B|\dots|9B|1|\dots|9, \\ B \rightarrow 0B|1B|2B|\dots|9B|0|1|\dots|9 \}$$

Endlicher Automat (Finite State Machine)

Erkennung einer rationalen Zahl (z.B. -0,425) durch einen endlichen Automaten. Nicht gültig wären ("00,4" oder "-,3" oder "3,000" oder "01,")



Beispiel: reguläre Sprache

Gesucht ist eine Sprache mit Worten $W \in \{a, b, c, d\}^+$ in der jedoch die Teilworte "dda" und "bdcb" nicht vorkommen dürfen.

$T = \{a, b, c, d\}$
 $N = \{S, T, A, B, C, D, E\}$
 $P = \{S \rightarrow aT \mid bA \mid cT \mid dB \mid a \mid b \mid c \mid d,$
 $T \rightarrow aT \mid bA \mid cT \mid dB \mid a \mid b \mid c \mid d,$
 $A \rightarrow aT \mid bA \mid cT \mid dC \mid a \mid b \mid c \mid d, \quad // G1W2$
 $B \rightarrow aT \mid bA \mid cT \mid dD \mid a \mid b \mid c \mid d, \quad // G1W1$
 $C \rightarrow aT \mid bA \mid cE \mid dD \mid a \mid b \mid c \mid d, \quad // G2W2 \wedge G1W1$
 $D \rightarrow bA \mid cT \mid dD \mid b \mid c \mid d, \quad // G2W1$
 $E \rightarrow aT \mid cT \mid dB \mid a \mid c \mid d \quad // G3W2$
 $\}$

GxWy ... "Gefahrenstufe x, Wort y"

Weiteres Beispiel regulärer Sprache (schwierig) (*)

- Wir betrachten nun schwierigeres Beispiel: Gesucht sei die reguläre Grammatik zur Sprache

$L = \{w \in \{0, 1\}^+ \mid w \text{ als Binärzahl aufgefasst ist durch 3 teilbar}\}.$

- Problemanalyse:** Wann ist eine Zahl durch 3 teilbar?
 - Genau dann, wenn Ziffernsumme durch 3 teilbar ist.
 - Begründung, anhand des Beispiels 4761:

Stelle durch 3 geteilt	Rest
1er	1
10er	6
100er	7
1000er	4

- Pro 10er, 100er, ... bleibt ein gewisser Rest. Ist die Summe dieser Reste durch 3 teilbar, dann auch die ursprüngliche Zahl!

Beispiel: reguläre Sprache

Gesucht ist die reguläre Grammatik der Sprache

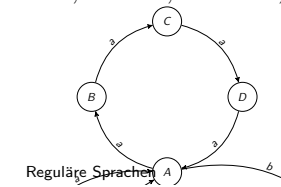
$$L = \{a^i b^j c^k \mid i \equiv 1(4), j \equiv 0(3), k \equiv 2(3)\}^*$$

$i \equiv 1(4)$, bzw. $i \equiv 1 \pmod{4}$ bedeutet, dass die Zahl i bei der ganzzahligen Division durch 4 den Rest 1 ergeben muss, also der Restklasse 1 angehört.

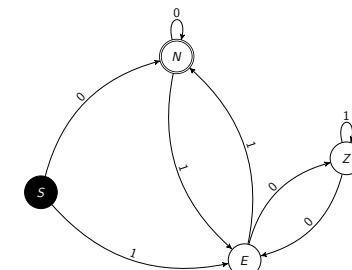
Beispiel: im obigen Beispiel kann i folgende Werte annehmen: 1, 5, 9, 13, ...

Beispiel: Die Worte der Sprache in aufsteigender Reihenfolge bezüglich ihrer Länge: $\varepsilon, acc, accccc, abbbcc, accacc, aaaaacc, \dots$

$P = \{S \rightarrow aA \mid \varepsilon,$
 $A \rightarrow aB \mid bE \mid cH,$
 $B \rightarrow aC,$
 $C \rightarrow aD,$
 $D \rightarrow aA,$
 $E \rightarrow aA,$
 $H \rightarrow aA,$
 $I \rightarrow aA,$
 $J \rightarrow aA,$
 $K \rightarrow aA,$
 $L \rightarrow aA,$
 $M \rightarrow aA,$
 $N \rightarrow aA,$
 $O \rightarrow aA,$
 $P \rightarrow aA,$
 $Q \rightarrow aA,$
 $R \rightarrow aA,$
 $S \rightarrow aA,$
 $T \rightarrow aA,$
 $U \rightarrow aA,$
 $V \rightarrow aA,$
 $W \rightarrow aA,$
 $X \rightarrow aA,$
 $Y \rightarrow aA,$
 $Z \rightarrow aA,$
 $\}$



Somit erhält man folgenden Automaten:



mit der Grammatik :

$P = \{S \rightarrow 0N \mid 1E \mid 0,$
 $N \rightarrow 0N \mid 1E \mid 0,$
 $E \rightarrow 0Z \mid 1N \mid 1,$
 $Z \rightarrow 0E \mid 1Z\}$

- Nach Start mit 0 $\rightarrow$ 0 Rest (N)
- Nach Start mit 1 $\rightarrow$ 1 Rest (E)
- In Zustand N (Restklasse 0): bei folgendem 1er Wechsel in Zustand E (Restklasse 1).
- Generell gilt: das Anhängen einer Ziffer verdoppelt die Zahl (Basis 2). Somit verdoppelt sich auch der Rest.
- Ziffer 0 in E: Rest wird auf 2 verdoppelt: Übergang nach Zustand Z (Restklasse 2)
- Ziffer 1 in E: Rest wird zunächst auf 2 verdoppelt, und ergibt mit +1 wieder 0 (Übergang nach Zustand N).

- Gesucht sei die Grammatik zur regulären Sprache

$$L = \{w \in \{a, b, c\}^* \mid w \neq \alpha cba\beta \wedge w \neq \alpha bcb\beta, \alpha, \beta \in T^*\}.$$

- Lösung:

$$\begin{aligned} P = \{ & S \rightarrow aT \mid bB \mid cA \mid a \mid b \mid c \mid \varepsilon, \\ & T \rightarrow aT \mid bB \mid cA \mid a \mid b \mid c, \\ & A \rightarrow aT \mid bC \mid cA \mid a \mid b \mid c, \quad //G1W1 \\ & B \rightarrow aT \mid bB \mid cD \mid a \mid b \mid c, \quad //G1W2 \\ & C \rightarrow bB \mid cD \mid b \mid c, \quad //G2W1 \wedge G1W2 \\ & D \rightarrow aT \mid bE \mid cA \mid a \mid b \mid c, \quad //G1W1 \wedge G2W2 \\ & E \rightarrow bB \mid b \} \quad //G2W1 \wedge G3W2 \end{aligned}$$

Aufgaben

Aufgabe 3.11

Gesucht ist eine Sprache mit Worten $W \in \{a, b, c, d\}^+$ in der jedoch die Teilworte "dda" und "bdb" *nicht* vorkommen dürfen. Geben Sie sowohl die reguläre Grammatik als auch den zugehörigen endlichen Automaten an.

Aufgabe 3.12

Gegeben sei die Sprache $L = \{a^i b^j c^k \mid i \equiv 2(5), j \equiv 1(3), k \equiv 3(4)\}^*$. Geben Sie die zugehörige reguläre Grammatik sowie den zugehörigen endlichen Automaten an.

Reguläre Sprachen, Endliche Automaten, und Reguläre Ausdrücke

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

POS (Theorie)

Reguläre Ausdrücke

1 / 8

Praxis: Basic Regular Expressions Standard

<code>c</code>	dass Zeichen <code>c</code>	BRE
<code>c₁c₂...</code>	Zeichenfolge <code>c₁c₂...</code>	BRE
<code>.</code>	ein beliebiges Zeichen	BRE
<code>\</code>	<i>escape character</i> : das folgende Zeichen ist <i>kein</i> Steuerzeichen	BRE
<code>\.</code>	Punkt-Zeichen	BRE
<code>[c₁c₂...]</code>	eines der Zeichen <code>c₁, c₂...</code>	BRE
<code>[c₁-c₂]</code>	eines der Zeichen zwischen <code>c₁</code> und <code>c₂</code> (inkl.)	BRE
<code>[^c₁c₂...]</code>	jedes andere Zeichen als <code>c₁, c₂...</code>	BRE
<code>[^c₁-c₂]</code>	jedes andere Zeichen als eines zwischen <code>c₁</code> und <code>c₂</code>	BRE
<code>\w</code>	Buchstabe, Ziffer, <code>_</code> : äquivalent <code>[A-Za-z0-9_]</code> <code>\W</code> : nicht ...	BRE
<code>\s</code>	Whitespace <code>\S</code> : nicht ...	BRE
<code>^</code>	Zeilenbeginn	BRE
<code>\$</code>	Zeilenende	BRE
<code>\b</code>	Wortgrenze: Wechsel <code>\w ↔ \W</code> <code>\B</code> : nicht ...	BRE
<code>*</code>	Wiederholung beliebig oft inkl. 0-mal (<i>lazy: *? PCRE</i>)	BRE

POS (Theorie)

Reguläre Ausdrücke

3 / 8

Reguläre Ausdrücke: Theorie

Regulärer Ausdruck über einer Menge Σ von Zeichen

- 1 jedes Zeichen $z \in \Sigma$ ist ein regulärer Ausdruck
- 2 sind r und s reguläre Ausdrücke, gilt
 - Aneinanderreihung: rs ist ein regulärer Ausdruck
 - Alternative: $r | s$ ist ein regulärer Ausdruck:
 - Wiederholung: r^* ist ein regulärer Ausdruck:
 - Klammerung: (r) ist ein regulärer Ausdruck:

Beispiel: $\Sigma = \{a,b\}$

- $a, b, aa, ab, ba, abba, abbbaaba$
- $a|b, abb|ba$
- $a^*, (a|b)^*$
- $a(a|b)^*b$
- $(a|b)^*abb$
- a^*b^*
- aa^*bb^*
- $a^*(a|b)^+$
- $(aa)^*(bb)^*b$
- $(aa|b)^*(a|bb)$

POS (Theorie)

Reguläre Ausdrücke

2 / 8

Praxis: Extended und Perl Compatible Regular Expressions

<code>+</code>	Wiederholung, beliebig oft, mindestens 1-mal (<i>lazy: +? PCRE</i>)	ERE
<code>{n}</code>	Wiederholung genau n -mal	ERE
<code>{m, n}</code>	Wiederholung m - bis n -mal	ERE
<code>{m,}</code>	mindestens m -mal	ERE
<code>{, n}</code>	höchstens n -mal	ERE
<code>?</code>	optional (0-mal oder 1-mal)	ERE
<code> </code>	oder (Alternative)	ERE
<code>\d</code>	Ziffer: äquivalent <code>[0-9]</code> <code>\D</code> : nicht ...	PCRE
<code>(...)</code>	Gruppierung ¹ und <i>back references</i> <code>\1, \2, ...</code>	ERE

¹hauptsächlich für Ersetzungen

POS (Theorie)

Reguläre Ausdrücke

4 / 8

Beispiele Zahlendarstellung

Ganze Zahl: 1, 123, +123, -123, 100, 001, 0, +0, -0

`[+-]?[0-9]+`

ohne führende Nullen, inkl. Null: 1, 123, +123, -123, 100, 0

`[+-]?[1-9][0-9]*|0`

Kommazahl: 3.14, +3.14, -3.14, 3.

`[+-]?[0-9]+\.[0-9]*`

ohne Vorkommateil: .14159

`[+-]?([0-9]+\.[0-9]*|\.[0-9]+)`

mit Exponent: 3.14E12, 3.14e12, 3.14e-12, 3.14e+12

`[+-]?([0-9]+\.[0-9]*|\.[0-9]+)([eE][+-]?[0-9]+)?`

Beispiel: Aufbau einer Textzeile (2)

z. B. Meier, Paul.F.: DBI2.3.14:00-16:30.C3.15.

Zeilenbeginn	^
Familienname (Buchstaben, 1. groß)	[A-Z][a-z]+
Beistrich	,
kein oder mehrere Leerzeichen	_*
Vorname (Buchstaben, 1. groß)	[A-Z][a-z]+
ein oder mehrere Leerzeichen	_*
2. Vorname Abkürzung (Großbuchstabe, Punkt)	[A-Z]\.
Doppelpunkt	:
kein oder mehrere Leerzeichen	_*
Kürzel (3 Großbuchstaben, Ziffer zwischen 1 und 4)	[A-Z]{3}[1-4]
ein Leerzeichen	_
Wochentag (1-5)	[1-5]
ein Leerzeichen	_
Uhrzeit von (als 2 Ziffern, Doppelpunkt, 2 Ziffern)	[0-9]{2}:[0-9]{2}
Bindestrich	-
Uhrzeit bis	[0-9]{2}:[0-9]{2}
ein oder mehrere Leerzeichen	_*
Raum (Buchstabe A-D, Ziffer 1-5, Punkt, Ziffer, Ziffer 1-9)	[A-D][1-5]\.[0-9][1-9]
kein oder mehrere Leerzeichen	_*
Zeilenende	\$

Beispiel: Aufbau einer Textzeile

z. B. Meier, Paul.F.: DBI2.3.14:00-16:30.C3.15.

- 1 Zeilenbeginn
- 2 Familienname (Buchstaben, 1. groß)
- 3 Beistrich
- 4 kein oder mehrere Leerzeichen
- 5 Vorname (Buchstaben, 1. groß)
- 6 ein oder mehrere Leerzeichen
- 7 2. Vorname Abkürzung (Großbuchstabe, Punkt)
- 8 Doppelpunkt
- 9 kein oder mehrere Leerzeichen
- 10 Kürzel (3 Großbuchstaben, Ziffer zwischen 1 und 4)
- 11 ein Leerzeichen
- 12 Wochentag als Nummer (1-5)
- 13 ein Leerzeichen
- 14 Uhrzeit von (als 2 Ziffern, Doppelpunkt, 2 Ziffern)
- 15 Bindestrich
- 16 Uhrzeit bis
- 17 ein oder mehrere Leerzeichen
- 18 Raum (Buchstabe A-D, Ziffer 1-5, Punkt, Ziffer, Ziffer 1-9)
- 19 kein oder mehrere Leerzeichen
- 20 Zeilenende

Beispiel: Aufbau einer Textzeile (3)

z. B. Meier, Paul.F.: DBI2.3.14:00-16:30.C3.15.

Regulärer Ausdruck für die ganze Zeile

`^[A-Z][a-z]+,[_*][A-Z][a-z]+,*[A-Z]\.:[_*][A-Z]{3}[1-4]-[1-5]-[0-9]{2}:[0-9]{2}-[0-9]{2}:[0-9]{2}_[A-D][1-5]\.[0-9][1-9]_*$`

Mockaroo: Generierung statt Match, d. h. $*$ $\Rightarrow$ $\{0,9\}$, $+$ $\Rightarrow$ $\{1,9\}$

`[[:upper:]][:lower:]{1,9},_{0,9}[:upper:][:lower:]{1,9}_{1,9}[:upper:]._{0,9}[:upper:]{3}(1|2|3|4)-(1|2|3|4|5)_{d{2}:d{2}}_{d{2}:d{2}}_{1,9}(A|B|C|D)(1|2|3|4|5)_{d(1|2|3|4|5|6|7|8|9)}_{0,9}`

Zeichenklassen (BRE)

`[[:upper:]]`: Großbuchstaben, `[[:lower:]]`: Kleinbuchstaben, `[[:alpha:]]`: Buchstaben, `[[:digit:]]`: Ziffern, `[[:alnum:]]`: Buchstaben und Ziffern, `[[:blank:]]`: Leerzeichen oder Tabulator, `[[:punct:]]`: Sonderzeichen, ...

Kontextfreie Sprachen

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$\{a^n b^n \mid n > 0\}.$$

- *Beispiel:* $\{ab, aabb, aaabbb, aaaabbbb, \dots\}$
- Sprache ist nicht regulär, da “Mitzählen” nicht möglich
- Lösung:

$$P = \{S \rightarrow aSb \mid ab\}$$

POS (Theorie)

Kontextfreie Sprachen

1 / 16

Beispiele für Kontextfreie Sprachen
○●○○○○○○○

Weitere Grammatiken
○○○

Syntaxdiagramme
○○○

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$\{a^{n+1} b^{3n} \mid n \geq 0\}.$$

- *Beispiel:* $\{a, aabbb, aaabbbbbb, \dots\}$

- Lösung:

$$P = \{S \rightarrow aSbbb \mid a\}$$

POS (Theorie)

Kontextfreie Sprachen

3 / 16

POS (Theorie)

Kontextfreie Sprachen

2 / 16

Beispiele für Kontextfreie Sprachen
○●○○○○○○○

Weitere Grammatiken
○○○

Syntaxdiagramme
○○○

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$\{a^n b^{n+2} c^m d^{m+1} \mid n, m \geq 0\}.$$

- *Beispiel:* $aabbbbccccddd(n = 2, m = 3)$

- Lösung:

$$\begin{aligned} P = \{ & S \rightarrow AB, \\ & A \rightarrow aAb \mid bb, \\ & B \rightarrow cBd \mid d \} \end{aligned}$$

POS (Theorie)

Kontextfreie Sprachen

4 / 16

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$\{i^n + i^m = i^{n+m} \mid n, m > 0\}, \text{ und}$$

$$T = \{i, +, =\}.$$

- **Beispiel:** $iiii + ii = iiiii$
- Lösung:

$$\begin{aligned} P &= \{S \rightarrow iSi \mid i + Ai, \\ A &\rightarrow iAi \mid i = i\} \end{aligned}$$

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$L = \{w \in \{0, 1\}^* \mid n_0(w) = n_1(w)\}.$$

- Dabei gibt die Funktion $n_x(w)$ die Anzahl der Vorkommnisse von x in w an.
- **Beispiel:** 00111101100001 mit $n_0(w) = n_1(w) = 7$.
- Lösung:

$$\begin{aligned} P &= \{S \rightarrow 0E \mid 1N \mid \varepsilon \\ E &\rightarrow 0EE \mid 1S, \\ N &\rightarrow 0S \mid 1NN\} \end{aligned}$$

- Aufgrund der Ableitungsregeln der Form $S \rightarrow tAB$ mit $t \in T$ und $S, A, B \in N$ ist ersichtlich, dass einfache endliche Automaten für kontextfreie Sprachen nicht funktionieren.
- Es ist unmöglich von *einem* Knoten in einem Schritt über eine Kante zu *zwei* (oder mehreren) Knoten zu gelangen.
- Endliche Automaten können somit nicht zur Überprüfung, ob ein Wort Element der kontextfreien Sprache ist, herangezogen werden.
- Der Automat für kontextfreie Sprachen ist der **Kellerautomat**
 - Kellerautomaten verfügen über einen Last-In First-Out Speicher
 - Die Zustandsübergänge hängen vom gelesenen Zeichen, dem Wert im Speicher (und gegebenenfalls dem Automatenzustand) ab.
- Zur grafischen Veranschaulichung, sowie zur Überprüfung (Ableitung) von Gültigkeit von Worten können **Syntaxdiagramme** verwendet werden.

Ableitung

Ableitung des Wortes 00111101100001:

00111101100001	00111101100001
0E	0011110N
00EE	0011110N
001SE	00111101NN
001E	001111011NNN
0011S	001111011000S
00111N	0011110110000E
001111NN	00111101100001S
0011110SN	00111101100001

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$L = \{a^i b^{2j+3} c^k \mid k > 0, j \geq 0, k \leq i \leq 3k\}.$$

- **Beispiel:** aaaaaabbbccc

- Lösung:

$$P = \{S \rightarrow aSc \mid aaSc \mid aaaSc \mid aAc \mid aaAc \mid aaaAc, \\ A \rightarrow bbAc \mid bbb\}$$

Kontextsensitive Grammatik

- Gesucht sei die Grammatik zur Sprache

$$\{a^n b^n c^n \mid n > 0\}.$$

- **Beispiel:** $\{abc, aabbcc, aaabbbccc, aaaabbbbcccc, \dots\}$
- Die Sprache ist nicht kontextfrei, da die in den Beispielen gezeigten Mechanismen zur Generierung zweier Elemente mit der selben Anzahl nicht auf den Fall von drei Elementen übertragbar ist.
- Die Grammatik ist *kontextsensitiv* (kontextabhängig, umgebungsabhängig)

- Gesucht sei die Grammatik zur kontextfreien Sprache

$$L = \{w \in \{a, b, c\}^* \mid n_a(w) = n_b(w) + 2n_c(w)\}.$$

- **Beispiel:** aaacabccabaaa

- Lösung:

$$P = \{S \rightarrow \varepsilon \mid aA \mid bB \mid cC, \\ A \rightarrow aD \mid bS \mid cB, \\ B \rightarrow aS \mid bC \mid cBC, \\ C \rightarrow aB \mid bC \mid cCC, \\ D \rightarrow aAD \mid aDA \mid bA \mid cS\}$$

- Bemerkungen:

- A ... ein a zuviel
- B ... ein b zuviel, bzw. 1 a zuwenig
- C ... zwei a's zuwenig
- D ... zwei a's zuviel

Kontextsensitive Grammatik

Die Produktionsregeln der kontextsensitiven Gr. zu $\{a^n b^n c^n \mid n > 0\}$ lauten:

$$P = \{S \rightarrow aSAB \mid aAB \\ BA \rightarrow AB \\ aA \rightarrow ab \\ bA \rightarrow bb \\ bB \rightarrow bC \\ CB \rightarrow cc\}$$

Ableitung von aabbcc

$$S \Rightarrow aSAB \\ \Rightarrow aaABAB \\ \Rightarrow aabBAB \\ \Rightarrow aabABB \\ \Rightarrow aabbBB \\ \Rightarrow aabbCB \Rightarrow aabbcc$$

$a^n b^n c^n \mid n > 0$ mittels Typ-0 GrammatikAbleitung von $aaabbbccc$:

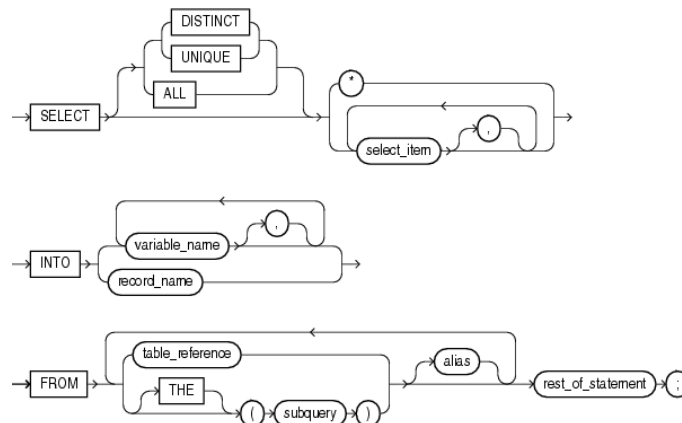
Produktionsregeln:

$$P = \{ \begin{array}{l} S \rightarrow abc \\ S \rightarrow A \\ A \rightarrow aABc \\ A \rightarrow abc \\ cB \rightarrow Bc \\ bB \rightarrow bb \end{array} \}$$

$$\begin{array}{l} S \Rightarrow A \\ \Rightarrow aABc \\ \Rightarrow aaABcBc \text{ (mittels } A \rightarrow aABc) \\ \Rightarrow aaabcBcBc \text{ (mittels } A \rightarrow abc) \\ \Rightarrow aaabBccBc \text{ (mittels } cB \rightarrow Bc) \\ \Rightarrow aaabBcBcc \text{ (mittels } cB \rightarrow Bc) \\ \Rightarrow aaabbcBcc \text{ (mittels } bB \rightarrow bb) \\ \Rightarrow aaabbBccc \text{ (mittels } cB \rightarrow Bc) \\ \Rightarrow aaabbbccc \text{ (mittels } bB \rightarrow bb) \end{array}$$

Syntaxdiagramme: PL/SQL

select into statement ::=



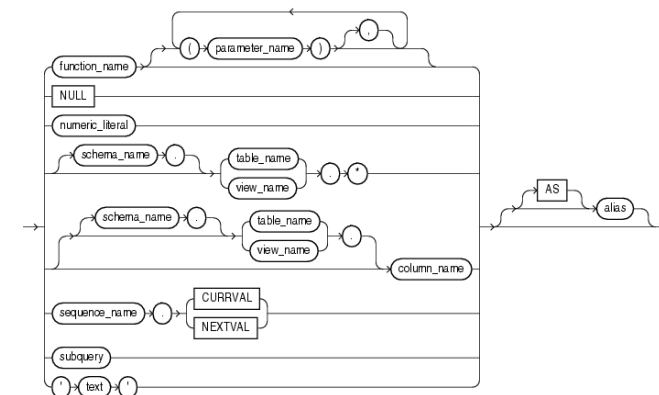
Quelle: [?]

Syntaxdiagramme

- Für eine ausführliche Darstellung sei auf das Skriptum [?] Seiten 24/25 verwiesen!
- Die Terminalsymbole sind durch Kreise oder Ellipsen dargestellt.
- Die Nonterminale sind durch Rechtecke dargestellt.

Syntaxdiagramme: PL/SQL

select item ::=



Quelle: [?]

Automaten

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

POS (Theorie)

Automaten

1 / 9

Keller-Automat

- Der Keller-Automat erweitert das Konzept des endlichen Automaten um einen Speicher
- Speicher: "Keller-Speicher" (=Stack)
- Englische Bezeichnung: pushdown automaton
- Bestandteile:
 - Zustandsmenge N
 - Eingabealphabet T
 - Kelleralphabet K
 - Übergangsfunktion $\delta : K \times (T \cup \{\varepsilon\}) \times K \rightarrow 2^{S \times K^*}$
 - Startzustand S
- Der Kellerspeicher wird mit $\perp$ initialisiert.
- In jedem Schritt wird ein Zeichen σ des zu verarbeitenden Wortes eingelesen.
- Ebenso wird das oberste Zeichen γ aus dem Kellerspeicher gelesen.
- Zusätzlich zum Zustandsübergang (δ) wird nun eine Zeichenkette aus K^* in den Kellerspeicher geschrieben.

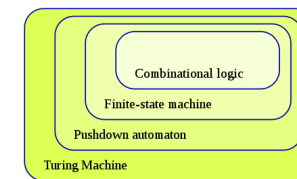
POS (Theorie)

Automaten

3 / 9

Automaten-Theorie

- Endliche Automaten können die syntaktische Korrektheit von Wörtern bezüglich regulärer Sprachen überprüfen.
- Das Ergebnis bezüglich eines Wortes lautet: akzeptiert oder nicht akzeptiert $\Rightarrow$ man nennt solche Automaten daher auch *Akzeptoren*.
- Der *Keller-Automat* (engl. Pushdown-Automaton) nutzt einen zusätzlichen Speicherbereich um auch Wörter kontextfreier Sprachen überprüfen zu können.
- Eine weitere Verallgemeinerung zur *Turing-Maschine* ermöglicht tatsächliche Berechnungen mittels des Automaten-Konzeptes vorzunehmen.



POS (Theorie)

Automaten

2 / 9

Wir betrachten die reguläre Sprache

$$L = \{a^n b^n \mid n \in \mathbb{N}^+\}.$$

Der Kellerautomat $A = (N, T, K, \delta, S)$ ist nun wie folgt definiert:

$$N = \{A, B\}$$

$$T = \{a, b\}$$

$$K = \{\perp, X\}$$

Die Übergangsfunktion lautet:

$$\delta(A, a, \perp) \rightarrow \{(A, X\perp)\}$$

$$\delta(A, a, X) \rightarrow \{(A, XX)\}$$

$$\delta(A, b, X) \rightarrow \{(B, \varepsilon)\}$$

$$\delta(B, b, X) \rightarrow \{(B, \varepsilon)\}$$

$$\delta(B, \varepsilon, \perp) \rightarrow \{(B, \varepsilon)\}$$

POS (Theorie)

Automaten

4 / 9

Beispiel: Wort *aabb* wird akzeptiert:

Zustand	(Rest-)Wort	Keller
A	aabb	⊥
A	abb	X⊥
A	bb	XX⊥
B	b	X⊥
B	ε	⊥
B	ε	ε

Beispiel: Wort *aaabb* wird *nicht* akzeptiert:

Zustand	(Rest-)Wort	Keller
A	aaabb	⊥
A	aabb	X⊥
A	abb	XX⊥
A	bb	XXX⊥
B	b	XX⊥
B	ε	X⊥

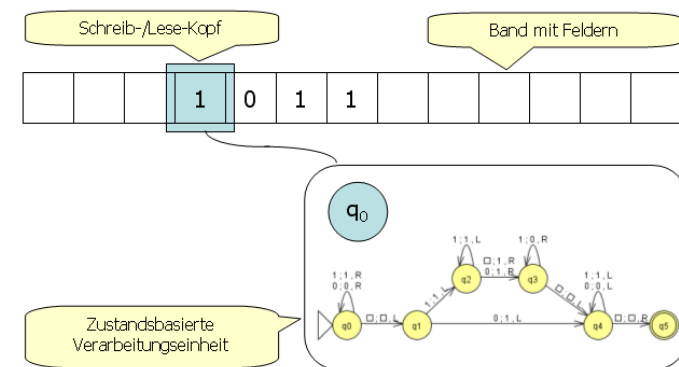
Die Klasse der von (nichtdeterministischen) Kellerautomaten akzeptierten Sprachen ist mit der Klasse der kontextfreien Sprachen identisch.

Die Sprachklasse der von deterministischen Kellerautomaten akzeptierten Sprachen ist eine echte Teilmenge der von nichtdeterministischen Kellerautomaten akzeptierten Sprachen.

Turing-Maschine

- Die Turing-Maschine ist eine weitere Verallgemeinerung des Automatenkonzeptes
- Sie besteht aus einem Zustandsautomaten, einem Schreib- und Lesekopf und einem (unendlich langen) Band
- In jedem Schritt wird
 - ein Zeichen vom Band gelesen,
 - ein Zustandsübergang ausgeführt (abhängig vom gelesenen Zeichen und dem aktuellen Zustand),
 - ein Zeichen auf das Band geschrieben, und
 - der Schreib-/Lese-Kopf nach rechts oder links bewegt.

Turing-Maschine



Quelle: <https://www.inf-schule.de/>

Turing-Maschine

- Die Turing-Maschine ist universelles Modell für Computer
- Alle Berechnungen die mit einem Computer durchgeführt werden können, können (theoretisch) auch mit einer Turing-Maschine durchgeführt werden.
- Die Turing-Maschine ermöglicht somit eine theoretische Analyse von Berechenbarkeit und Komplexität.
- Bezüglich Typ-0 Grammatiken können Wörter mit der Turing-Maschine auf ihre syntaktische Korrektheit geprüft werden.

Komplexitätstheorie

Programmieren und Software-Engineering Homomorphismen, Formale Sprachen und Syntax-Analyse

26. Februar 2026

Komplexitätsklassen

- Der *Nicht-Determinismus* ermöglicht also schwierigere Probleme zu berechnen.
- Die Turing-Maschine **errät (!)** die Lösung in Polynomialzeit.
- Die *Verifikation* der Lösung ist in Polynomieller Zeit möglich
- **Folgerung:**
 - Die Probleme in *NP* sind "schwierig" zu lösen, aber "leicht" zu verifizieren!
 - Deshalb werden sie auch "Guess-and-Check"-Probleme genannt.

Man nennt polynomiell berechenbare Probleme (also Probleme in *P*) auch *effizient* lösbar, Probleme die *nicht* in *P* liegen sind *nicht* effizient lösbar (die Berechnung würde in der Praxis zu lange dauern).

Komplexitätsklassen

Definition (Komplexitätsklasse *P*)

Die Klasse all jener Probleme, die auf einer **deterministischen** Turing-Maschine in **polynomieller** Zeit berechnet werden können.

Definition (Komplexitätsklasse *NP*)

Die Klasse all jener Probleme, die auf einer **nicht-deterministischen** Turing-Maschine in **polynomieller** Zeit berechnet werden können.

Anmerkung: Alle Probleme in *NP* können auf einer *deterministischen* Turing-Maschine in **exponentieller Zeit** berechnet werden.

Zusammenhang der Komplexitätsklassen

Zusammenhang *P* und *NP*

Klarerweise gilt:

$$P \subseteq NP$$

- Die Probleme aus *P* sind jedenfalls in *NP* enthalten. Aber sind diese Klassen tatsächlich verschieden?
- Sind alle Probleme in *NP* gleich schwierig?

NP-Vollständigkeit

- Tatsächlich gibt es Probleme in *NP* mit der Eigenschaft, dass alle anderen Probleme aus *NP* auf sie abgebildet werden können.
- Das bedeutet, mit einem Algorithmus für das *eine* Problem, hat man automatisch ein Lösungsverfahren für das *andere* Verfahren.
- Wichtig ist, dass die Abbildung selbst auch in Polynomialzeit erfolgen kann.

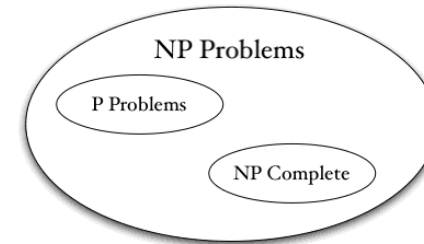
Definition (Komplexitätsklasse NP-Vollständig)

Die Klasse all jener Probleme, die auf einer **nicht-deterministischen** Turing-Maschine in **polynomieller** Zeit berechnet werden können, und zu den schwierigsten Problemen in dieser Problemklasse gehören.

Anmerkung: Der Begriff der Vollständigkeit sagt aus, dass all diese (schwierigsten) Probleme aus *NP* aufeinander abgebildet werden können. Kann man eines dieser Probleme effizient lösen, können mit diesem Algorithmus alle *NP*-vollständigen Probleme gelöst werden!

Entscheidungsproblem

- In der Komplexitätstheorie werden meist sogenannte **Entscheidungsprobleme** betrachtet.
- Die Lösung zu einem Entscheidungsproblem ist stets **ja** oder **nein**.
 - *Beispiel:* Existiert ein Hamilton-Kreis im Graphen *G*?
- In der Praxis sind oft **Optimierungsprobleme** von Interesse.
 - *Beispiel:* Finde die **kürzeste** TSP-Tour?
- Optimierungsprobleme können (indirekt) als Entscheidungsprobleme definiert werden:
 - *Beispiel:* Existiert eine TSP-Tour kürzer als *x*?
- Bei Entscheidungsproblemen spricht man gegebenenfalls von *NP*-Vollständigkeit,
- zugehörige (=verwandte) Optimierungsprobleme nennt man *NP*-schwierig (oder *NP*-schwer).



Quelle: <https://trigonakis.com/>

$P \neq NP$?

Bis heute wurde nicht bewiesen, dass tatsächlich $P \neq NP$ gilt. Dies ist somit eines der größten ungelösten Probleme der theoretischen Informatik.

Obwohl der Beweis für $P \neq NP$ bisher nicht erbracht wurde, nehmen die meisten Wissenschaftler an dass tatsächlich eine Ungleichheit vorliegt. Im unwahrscheinlichen Falle eines konstruktiven Beweise für $P = NP$ wären auf einen Schlag viele schwierige praxisrelevante Probleme effizient lösbar.

Folgerungen

Folgerung:

In der Praxis bedeutet dies, dass *NP*-vollständige Probleme auf konventionellen Computern nur mit exponentiellem Zeitaufwand berechnet werden können.

Viele praxisrelevante Probleme fallen jedoch in diese Komplexitätsklasse. Durch *heuristische* Algorithmen können meist für praktische Anwendungen ausreichend gute *Näherungs-Lösungen*^a berechnet werden.

^anicht die bestmögliche Lösung, aber eine ausreichend gute Lösung. Z.B. Travelling Salesman Problem: eine nur geringfügig längere Rundtour als die bestmögliche Tour.

Typische NP-schwierige Probleme (Übersicht)

- Hamilton-Kreis (Rundreise durch alle Knoten eines Graphen)
- Subset-Sum / Partition (Gleichverteilung von Zahlen)
- 0/1-Knapsack (Rucksackproblem, Optimierung mit Kapazität)
- Bin-Packing (Gegenstände auf Behälter verteilen)
- 3-SAT (Erfüllbarkeit boolescher Formeln)

Es gibt zahlreiche weitere NP-schwere Probleme, diese sind nur einige der bekanntesten Beispiele. Alle diese Probleme sind äquivalent in dem Sinne, dass sie aufeinander abgebildet werden können (NP-vollständig). Die folgenden Folien stellen einige der Problemstellungen überblicksartig vor.

0/1-Knapsack (Rucksack)

Problem (Optimierung):

- Gegeben n Gegenstände mit Gewicht w_i und Wert v_i sowie Kapazität W .
- Wähle eine Teilmenge (jeder Gegenstand höchstens einmal) so, dass die Gesamtmasse $\leq W$ ist und der Gesamtwert maximiert wird.

Beispiel: Für die Gegenstände mit Gewichten $\{2, 3, 4, 6\}$, Werten $\{3, 4, 5, 6\}$ und Kapazität $W = 10$ ist die optimale Lösung die Auswahl der Gegenstände mit Gewicht 2 und 3 und 4 mit Gesamtwert 12.

Als Entscheidungsproblem: Gibt es eine Auswahl mit Wert $\geq V_0$? (NP-vollständig in der Entscheidungsform)

Subset-Sum / Partition

Subset-Sum (Entscheidungsproblem):

- Gegeben eine Menge positiver Ganzzahlen $\{a_1, \dots, a_n\}$ und ein Zielwert T .
- Frage: Gibt es eine Teilmenge, deren Summe genau T ist?

Partition (Spezialfall):

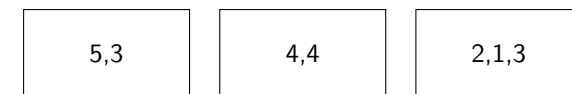
- Frage: Lässt sich die Menge in zwei Teilmengen mit gleicher Summe teilen? (also $T = \frac{1}{2} \sum a_i$)

Beispiel: Für die Menge $\{8, 3, 7, 2\}$ und das Ziel $T = 10$ gibt es zwei solche Teilmenge mit Summe 10, nämlich $\{8, 2\}$ und $\{3, 7\}$. (Antwort: Ja.)

Bin-Packing

Problem:

- Gegeben n Gegenstände mit Größen s_i und Behälter (Bins) mit Kapazität C .
- Ziel (Entscheidung): Kann man alle Gegenstände in höchstens k Bins packen?
- **Beispiel:** Für die Gegenstände mit Größen $\{5, 3, 4, 4, 2, 1, 3\}$ und Bin-Kapazität $C = 8$ gibt es eine Packung in 3 Bins, z.B. $\{5, 3\}$, $\{4, 4\}$, $\{2, 1, 3\}$.



- Heuristiken (First-Fit, Best-Fit) sind praktikabel, beweisbar optimal zu entscheiden ist NP-schwer.

3-SAT (Erfüllbarkeit)

Problem:

- Gegeben sei eine aussagenlogische Formel in konjunktiver Normalform, wobei jeder Minterm genau drei Literale enthalten soll.
- *Beispiel:* $(x_1 \wedge \neg x_2 \wedge x_3) \vee (\neg x_1 \wedge x_5 \wedge x_3) \vee \dots \vee (\neg x_3 \wedge \neg x_5 \wedge x_4)$
- Frage: Gibt es eine Belegung der Variablen, die die Formel wahr macht?

3-SAT ist das klassische NP-vollständige Problem und oft Ausgangspunkt für Reduktionen.